Introduction

The next two sections provide information on Theming and Scrolling. Both important concepts to understand to best use this library. The rest of the manual is reference in nature and split into the following sections:

- Containers
- Controls and Display
- Menus
- Additional Widgets

Each widget has a brief description of its function and includes a screen shot of the widget, description of its constructor, list of properties and any public methods. There is also a section describing the specific Theme used for the widget, If you want to customize it for your purposes this is a great starting point

Check out the Notes section near the end, it might save you a lot of time by avoiding problems. And perhaps of But most importantly, there is a runnable code example that you can copy and it should just work.

Contents

Theming

Every colour, font, and several structural values in this library come from a single file: `sCTkThemes.json`. No widget hardcodes a colour. If you don't like the palette — and you may not, the author is somewhat red/brown colour blind — you change one file and the whole application follows.

Where the file lives

Two locations are checked, in this order:

1. `sCTkThemes.json` **in your application's working directory** — your override.
2. `scustomtkinter/assets/sCTkThemes.json` — the one shipped with the library.

The first one found wins, completely. There is **no merging**: if a local file exists, the bundled one is not consulted at all, for any widget.

That matters more than it sounds. Copy the bundled file, delete the blocks you don't care about, and every widget whose block you removed will fail to construct — not fall back to the library defaults. **Copy the whole file and edit in place.**

```
cp .../scustomtkinter/assets/sCTkThemes.json ./sCTkThemes.json
```

The file is read once, at import time. Changes require a restart.

Block structure

One block per widget class, keyed by the exact class name:

```
{
    "sCTkButtonPrimary": {
        "fg_color": ["#1A4375", "#1F6AA5"],
        "text_color": ["#FFFFFF", "#FFFFFF"],
        "font": ["Arial", 13, "bold"],
        "disabled_map": {
            "fg_color": ["#CBD5E1", "#334155"],
            "text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

The class name is the lookup key and it is **case-sensitive and exact**. A block named `sCTkButton` does nothing for a widget class named `sCTkButtonPrimary`. There is no inheritance between blocks — `sCTkDialSelector` does not inherit from a `sCTkDial` block, and each of the three dial variants carries its own full set of keys even where the values are identical.

Two kinds of key appear in a block:

- **Native CustomTkinter options** — `fg_color`, `corner_radius`, `border_width` and so on. These are passed through to the underlying widget.
- **Keys specific to this library** — `dial_color`, `led_on_color`, `scale_font`, `btn_hover`. These are read by the widget's own drawing code and never reach CustomTkinter.

You mostly don't need to know which is which. It matters in one place: see adding a block for your own widget.

State maps

Nested inside a block, a state map overrides specific keys when the widget is in that state. Anything not listed keeps its normal value.

Map	Applies when
`disabled_map`	The widget is disabled via `state("disabled")` or `configure(state="disabled")`.
`pressed_map`	A button is being held down.
`alarm_map`	A widget is in an alert condition.
`readonly_map`	An entry or spinbox is readonly — arrows still work, typing is blocked.

Most widgets use only `disabled_map`. The button family uses `pressed_map` and `alarm_map`; `sCTkEntryPrimary` / `Secondary` and `sCTkSpinbox` use `readonly_map` to support a genuine three-state model rather than collapsing everything non-disabled into "normal".

A state map is not a full block. Only list the keys that actually change. Widgets deliberately leave `fg_color` out of `disabled_map` in most cases — the background stays put and the border, text, or face carries the signal.

Some keys exist *only* inside a state map, because they have no normal-state equivalent. `sCTkSegmentedButton`'s `selected_text_color` is one: a selected segment's normal text colour comes from `text_color`, and the separate key exists so the selection is still identifiable when the control is greyed out.

Light and dark

Colours are written as a two-element list: `[light_mode, dark_mode]`.

```
"text_color": ["#1A4375", "#FF9100"]
```

Widgets pass these through as pairs rather than resolving them up front, so CustomTkinter's own appearance-mode tracking repaints them when the user switches. You don't need to do anything.

A single string is also accepted, and means the same colour in both modes. The literal `"transparent"` is a CustomTkinter pseudo-value meaning "show whatever is behind me". Note that `"transparent"` **cannot** be used for anything drawn on a raw `tkinter.Canvas` — that includes the dials, the S-meters, and `sCTkScrollArea`'s background, which need a real renderable colour.

Fonts are `[family, size]` or `[family, size, weight]`.

Changing values at runtime

`configure()` accepts theme keys directly, and the override **sticks**:

```
frame.configure(fg_color="red")
```

Two consequences worth understanding:

A single colour replaces the light/dark pair. Passing one value for a key means that property stops following appearance mode — which is what asking for one specific colour means. Pass a two-element tuple if you want it to keep tracking:

```
frame.configure(fg_color=("#FFFFFF", "#111827"))
```

State maps still win in their state. Overriding `border_color` sets the *normal*-state colour. If the widget is disabled, or later becomes disabled, `disabled_map` supplies the border colour as usual. To change what a disabled widget looks like, change the theme file.

Things that will break your theme

This section is the important one. JSON is unforgiving and the failure modes are not always obvious.

Syntax errors take out the entire file

A missing comma, a stray trailing comma before a `}`, an unclosed brace, or a smart quote pasted in from a document — any one of these makes the whole file unparseable. The library catches the error, prints a warning, and **continues with an empty theme registry**. Every widget then fails to construct.

The warning looks like this:

```
sCustomTkinter System Warning -> Could not parse theme layout tracking: ...
```

If you see that line, the problem is a syntax error in your file, not in any widget. Validate before running:

```
python -m json.tool sCTkThemes.json > /dev/null
```

Silence means it parsed. Any editor with JSON support will also flag these as you type — worth using one.

Deleting a key is not the same as leaving it at default

There is no "default" to fall back to. Widgets validate their required keys at construction and raise immediately:

```
KeyError: "'sCTkTabview' theme block is missing 'text_color' at the top level of sCTkThemes.json."
```

That message names the exact key and whether it belongs at the top level or in a state map. This is deliberate. An earlier design substituted a plausible hardcoded colour when a key was missing, and the result was worse than a crash: five separate widgets shipped for a long time rendering in hardcoded colours while their configured theme values were silently ignored, and nobody noticed because the substituted colours *looked* fine. A loud failure that names the key is far better than a widget that quietly ignores you.

If you genuinely don't want a widget's block, don't delete it — you'll break that widget. Change its values instead.

Misspelling a key is worse than deleting it

A misspelled key is not an error. It's an unrecognised key that gets ignored, while the *correct* key is now missing:

```
"text_colour": ["#1A4375", "#FF9100"]
```

That produces a `KeyError` about `text_color` being missing — which is confusing until you spot that your line is right there, spelled British. Check the spelling in this library's own documentation for the widget; it uses American spellings throughout, following CustomTkinter.

A misspelling inside a state map is quieter still: state maps aren't validated as strictly, so a typo there usually means "that property just doesn't change when disabled," with no error at all.

Renaming a block orphans it

Rename `sCTkSlider` to `sCTkSliders` and the block becomes dead data while every slider fails to construct. Block names must match class names exactly.

The reverse also happens: a block for a widget that no longer exists, or was renamed, sits in the file doing nothing. Harmless, but it accumulates.

Adding a key that isn't read does nothing

Adding `"hover_glow_color"` to a block will not make anything glow. Widgets read a fixed set of keys; extra ones are ignored silently. If you want a new visual property, the widget's drawing code has to read it.

Adding a theme block for your own widget

If you subclass `ThemeableWidget`, your block is found automatically by class name. Three things to know:

Custom keys must not reach the native constructor. CustomTkinter widgets reject keyword arguments they don't recognise — `CTkToplevel` in particular raises on *any* unknown key. Filter your resolved keywords down to what the native class actually accepts before calling `super().__init__()`. Every widget in this library that adds custom keys does this; copy the pattern from one close to yours.

Some custom keys are stripped for you. `ThemeableWidget` maintains an internal list of names it removes from the resolved keywords for canvas-drawing widgets — `dial_color`, `text_color`, `pointer_color` and others. If your widget uses one of those names, it will not be in `final_kw`, and you must read it from the raw theme registry instead. This is not obvious and has caused real bugs: an entire widget family rendered in fallback colours for its whole existence because it looked for those keys in the wrong place. If a colour you configured isn't appearing, this is the first thing to check.

Validate your required keys at construction. Follow the pattern used throughout:

```
_REQUIRED_THEME_KEYS = ("fg_color", "text_color")
_REQUIRED_DISABLED_KEYS = ("text_color",)
```

and raise `KeyError` naming the missing key. It costs a dozen lines and turns a class of silent visual bug into an immediate, self-explaining failure.

Scrolling

Scrolling in this library is handled in one place. Whichever widget you use, the wheel and trackpad behaviour comes from a single shared implementation — `ScrollBindingMixin` — so it feels the same everywhere and a fix applies everywhere.

Which widget to use

Widget	Use when
`sCTkScrollableFrame`	You want a scrolling container and don't care where the scrollbar lives. This is the default choice.
`sCTkScrollArea` + `sCTkScrollbar`	You need the scrollbar somewhere the built-in one can't go, or you want to control child event binding explicitly.
`sCTkFileExplorer`, `sCTkTableview`, `sCTkSelector`	These scroll internally. You don't wire anything up.

`sCTkScrollableFrame` builds and manages its own scrollbar. `sCTkScrollArea` deliberately doesn't — you create an `sCTkScrollbar` separately and connect the two with `hook_scrollbar()`. That's the whole reason the pair exists: it lets the bar sit outside the scrolling region, share space with other widgets, or be styled independently.

```
scroll_view = sCTkScrollArea(container)
scroll_view.pack(side="left", fill="both", expand=True)

scrollbar = sCTkScrollbar(container, orientation="vertical")
scrollbar.pack(side="right", fill="y")

scroll_view.hook_scrollbar(scrollbar)

# Content goes into scroll_content, not into the area itself.
for row in data:
    sCTkLabelSecondary(scroll_view.scroll_content, text=row).pack(anchor="w")
```

Content added to `scroll_content` is bound for scrolling automatically, including anything added later. You do not need to call `propagate_scroll_events()` on each item — that method now exists only for widgets placed *outside* the content tree.

How scroll input is handled

Three platforms behave differently, and all three are handled:

Platform	Mechanism
Windows	`<MouseWheel>` with a delta scaled in units of 120
Linux	Discrete `<Button-4>` / `<Button-5>` events — there is no continuous delta
macOS	Its own `<MouseWheel>` scaling, **plus** a separate higher-precision `<TouchpadScroll>` event

macOS trackpads deliver far more events, with far finer values, than a wheel does. Acting on each one is unusably fast, so trackpad deltas accumulate and move the view only once a threshold is crossed. The accumulator resets when you reverse direction, so a change of direction responds immediately rather than having to cancel out what built up going the other way.

Bindings activate on their own and maintain themselves. You never call an activation method, and content added after a widget is placed — the normal case, since you construct, place, then populate — is picked up automatically.

Full detail, including why this takes four separate mechanisms, is on the `ScrollBindingMixin` page.

Tuning scroll speed

Three constants control the feel. They live on `ScrollBindingMixin` as class attributes, so they can be changed globally, per widget class, or per instance.

Constant	Default	Effect
`MAC_SCROLL_SENSITIVITY`	`3`	Amplification for macOS wheel deltas, which are much smaller than Windows' steps.
`MAC_SCROLL_MAX_STEP`	`5`	Ceiling on rows travelled per macOS wheel event.
`TOUCHPAD_ACCUMULATION_THRESHOLD`	`12.0`	Accumulated trackpad movement required before the view moves.

`MAC_SCROLL_MAX_STEP` **is the one you're most likely to want to change.** macOS reports wildly different delta magnitudes depending on hardware: an Apple Magic Mouse sends fine values near 1, while a conventional wheel mouse sends a large value per detent — around 38 in testing. Without a ceiling, the amplification turns one wheel click into 114 rows of travel, which throws a hundred-row list end to end. The clamp lets small deltas scale normally and saturates large ones.

Set it to `3` for the conventional three-rows-per-notch that matches macOS defaults and most applications. Values below `3` also slow the Magic Mouse, since its fine deltas already scale to 3 before the clamp applies.

To change it everywhere, set it once at startup:

```
from scustomtkinter.sctk_scroll_mixin import ScrollBindingMixin
ScrollBindingMixin.MAC_SCROLL_MAX_STEP = 3
```

Or per instance, if one widget wants a different feel:

```
log_view = sCTkScrollableFrame(root)
log_view.MAC_SCROLL_MAX_STEP = 8
```

These values are tuned on macOS. If you're shipping to Windows or Linux and the feel is wrong, these are the knobs.

Disabling scrolling

`sCTkScrollableFrame` and `sCTkFileExplorer` both stop scrolling entirely when disabled — wheel, trackpad, and scrollbar dragging. The bar stays visible but inert; CustomTkinter's scrollbar has no greyed-out appearance to switch to.

`sCTkScrollableFrame` additionally separates two ideas that are easy to confuse:

- `state` is the user-facing enabled/disabled presentation.
- `scroll_enabled` is your own intent about whether this frame should scroll at all.

Scrolling happens only when both allow it, and neither overwrites the other. A frame you deliberately set non-scrolling stays non-scrolling after a disable/enable round trip:

```
frame = sCTkScrollableFrame(master, scroll_enabled=False)
frame.configure(state="disabled")
frame.configure(state="normal")
frame.is_scrolling()          # still False -- your intent survived
```

`disable_scroll()` and `enable_scroll()` are the runtime equivalents, useful when adding a lot of content at once:

```
frame.disable_scroll()
for item in many_items:
    sCTkLabelSecondary(frame, text=item).pack()
frame.enable_scroll()
```

`sCTkScrollArea` has no disabled state.

Nested scrolling regions

Putting one scrolling widget inside another works: the inner one keeps its own bindings and the outer one stops at its boundary, so the wheel scrolls whichever region the pointer is actually over rather than both at once.

The guard recognises `CTkScrollableFrame` and anything built on it — `sCTkScrollableFrame`, `sCTkTableview`, `sCTkSelector`. A scrolling region you build yourself directly on a plain `tkinter.Canvas` is **not** recognised, and would get bound to the outer widget as well as your own handler. If you need that, put it in an `sCTkScrollArea` instead.

Containers

The following widgets are the containers that will hold your user interface. There are some additional constainers that might be of interest that are listed later in the section where we document additional widgets added with sCustomTkinter.

sCTk

The `sCTk` is the primary main window container class wrapper for the `sCustomTkinter` workstation library ecosystem. It acts as a clean, direct pass-through equivalent to its foundational parent container layout class, `customtkinter.CTk`.

📌 Localized Table of Contents

Core Architectural Purpose

The application base frame serves as the core master anchor for your interface tree:

1. **Decoupled User Space:** It eliminates the architectural requirement to maintain raw `import customtkinter` bindings inside your station cockpit panel code.
2. **Framework Alignment:** It standardizes the root initialization sequence pass to match the repository's native object naming conventions (`sCTkFrame`, `sCTkButtonPrimary`, etc.).

Constructor Reference

It maps perfectly onto all native window properties, event loop callbacks, lifecycle handlers, and geometries tracking parameters out-of-the-box.

```
from sCTk import sCTk
from sCTkThemes import apply_sCTkThemes

1. Initialize centralized framework look records natively on system boot
```

```
apply_sCTkThemes()

2. Instantiate your primary root application backplane directly
app = sCTk()
app.geometry("800x600")
app.title("Main Control Rig Backplane")

app.mainloop()
```

Return to Table of Contents

sCTkToplevel

Table of Contents

Overview

`sCTkToplevel` is a themeable subclass of `customtkinter.CTkToplevel`, for secondary windows, modal dialogs, and popups. It adds automatic light/dark theme resolution from `sCTkThemes.json`. This is the simplest widget in the library — no disabled state, no `state()` / `get_state()` at all, and no per-state color-swapping logic, since a top-level window has no interactive "enabled/disabled" concept the way a control does.

Constructor

```
sCTkToplevel(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent window.
`**kwargs`	—	—	Any native `CTkToplevel` argument, or an `fg_color` override — the theme block for this widget currently defines only `fg_color`.

```
settings_window = sCTkToplevel(root)
settings_window.title("Settings")
settings_window.geometry("300x200")
```

Methods

Method	Returns	Description
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, with positional-dict support (e.g. `configure({"fg_color": "red"})`). There's no single-argument property-query support here — unlike every other widget in this library, a bare positional string (e.g. `configure("fg_color")`) currently has no effect at all, since the only positional-argument handling implemented is the dict-merge case.

Theming (`sCTkThemes.json`)

Everything is applied once, at construction — there's no `disabled_map` and no runtime color-swapping logic at all.

```
{
    "sCTkToplevel": {
        "fg_color": ["#F8FAFC", "#0F172A"]
    }
}
```

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkToplevel` directly, construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys native `CTkToplevel` actually accepts. This matters more here than for most widgets — confirmed directly against CustomTkinter's own source, `CTkToplevel.__init__` explicitly validates that no unrecognized keyword survives after its own known-valid keys are popped, and raises immediately if one does. This only matters for the base-class composition pattern — constructing a plain `sCTkToplevel` directly is unaffected either way.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkToplevel, sCTkLabelPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Toplevel Example")

    def open_settings():
        settings_window = sCTkToplevel(root)
        settings_window.title("Settings")
        settings_window.geometry("300x200")
        sCTkLabelPrimary(settings_window, text="Settings go here").pack(expand=True)
```

```
open_button = sCTkButtonPrimary(root, text="Open Settings", command=open_settings)
open_button.pack(pady=20)

root.mainloop()
```

Known Limitations

- No single-argument property-query support (e.g. `configure("fg_color")` does nothing) — consistent with this widget's overall minimalism, but different from every other widget in this library.
- No `state()` / `get_state()` /disabled concept at all — this widget has no visual state to toggle.

Return to Table of Contents

sCTkFrame

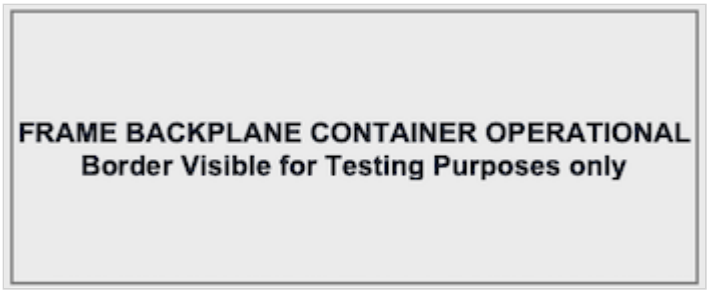
Table of Contents

Overview

`sCTkFrame` is a themeable subclass of `customtkinter.CTkFrame`. It adds automatic light/dark theme resolution from `sCTkThemes.json`. Unlike every other widget in this library, it has no disabled state and no per-state color swapping — frames are containers, not interactive controls, so there's nothing to dim or lock.

Dark Mode: 

Light Mode:



Constructor

```
sCTkFrame(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkFrame` argument, or an override for one of the theme keys listed under Theming.

```
panel = sCTkFrame(control_root)
panel.pack(expand=True, fill="both", padx=20, pady=20)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	No-op. Always returns `"normal"` regardless of what's passed in — deliberate, not a bug, so generic code written against every widget's `state()` / `get_state()` / `configure(state=...)` API doesn't need a special case for frames.
`get_state()`	`str`	Equivalent to calling `state()` with no argument. Always `"normal"`.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` is silently absorbed (a no-op) rather than forwarded to the native widget, which has no real state concept; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, and `border_color` — since neither varies by state here, `default` and `current` are always identical. Queries for any other property name fall through to the native `CTkFrame.configure`.

Theming (`sCTkThemes.json`)

Everything is applied once, at construction — there's no `disabled_map` for this widget, and no runtime color-swapping logic at all.

```
{
    "sCTkFrame": {
```

```
        "border_width": 0,
        "corner_radius": 0,
        "border_color": ["gray", "gray"],
        "fg_color": "transparent"
    }
}
```

With `border_width` at `0`, `border_color` never actually renders visibly regardless of its value — the two are set to the neutral Tkinter color name `"gray"` for both light and dark mode here, but that's moot while the border has no width.

Colors are passed through as raw `(light, dark)` tuples at construction and never touched again, so CustomTkinter's own native appearance-mode tracking handles light/dark repaints on its own — there's no `_set_appearance_mode()` override here, since there's nothing for one to re-trigger. This is the same underlying mechanism validated more deliberately on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family.

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkFrame` directly (rather than placing it as a child), construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys native `CTkFrame` actually accepts (confirmed directly against CustomTkinter's source) to its own native constructor call, so any of your composite's own theme keys that `CTkFrame` wouldn't recognize are filtered out rather than causing a `TypeError`. This only matters for that composition pattern — constructing a plain `sCTkFrame` directly is unaffected either way.

Example

```
#!/usr/bin/python3

from scustomtkinter import sCTkButtonPrimary, sCTkLabelPrimary, sCTk, sCTkFrame


if __name__ == "__main__":

    root = sCTk()
    root.geometry("500x300")
    root.title("sCTkFrame Container Validation Bench")

    # Instantiate your custom theme-compliant frame element chassis
    base_container = sCTkFrame(root, border_width=2)
    base_container.pack(expand=True, fill="both", padx=30, pady=30)
#
#     # Add a simple sub-element child widget to verify structural clipping layouts
    lbl_marker = sCTkLabelPrimary(base_container, text="FRAME BACKPLANE CONTAINER OPERATIONAL\n"+
                                  "Border Visible for Testing Purposes only")
    lbl_marker.pack(expand=True)

    root.mainloop()
```

Known Limitations

- `state()` / `get_state()` / `configure(state=...)` are all no-ops by design — there's no way to visually disable a frame through this API, since the widget has no disabled state at all.
- Calling `configure("fg_color")` or `configure("border_color")` returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `border_color`, and falls through to the native widget's `configure()` for anything else.

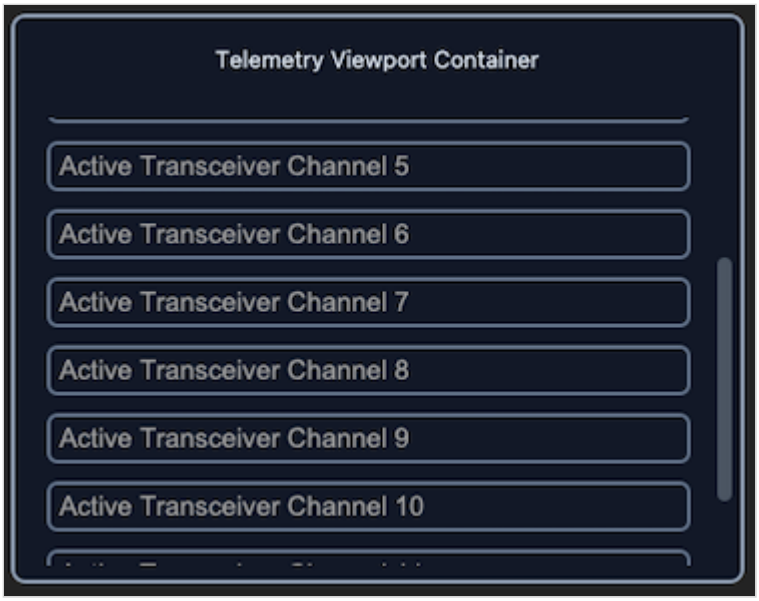
Return to Table of Contents

sCTkScrollableFrame

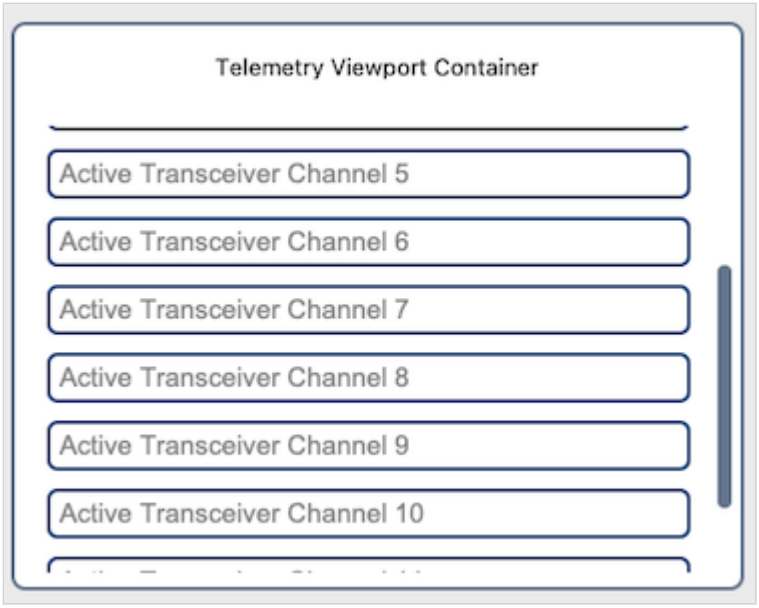
Table of Contents

Overview

`sCTkScrollableFrame` is a themeable subclass of `customtkinter.CTkScrollableFrame`. It adds automatic light/dark theme resolution from `sCTkThemes.json`, plus carefully-tuned cross-platform mouse wheel and macOS trackpad scroll handling that native CustomTkinter doesn't reliably provide on its own.

Dark Mode: 

Light Mode:



Unlike `sCTkFrame`, this widget **does** have a disabled state. That's justified here where it isn't for a plain frame: this widget owns real behavior to disable, not just colors. Disabling dims the border and scrollbar and stops all scrolling — wheel, trackpad, and scrollbar drag alike.

Disabling does **not** cascade to child widgets. That remains the caller's responsibility, exactly as with the labeled frame variants — loop over `get_children()` and call `.configure(state=...)` on each one.

Constructor

```
sCTkScrollableFrame(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.

Parameter	Type	Default	Description
`state`	`str`	`"normal"`	`"normal"` or `"disabled"`. See Scrolling and State.
`scroll_enabled`	`bool`	`True`	Whether this frame should respond to scroll input at all.
`**kwargs`	—	—	Any native `CTkScrollableFrame` argument (e.g. `label_text`, `orientation`), or an override for one of the theme keys listed under Theming.

```
log_viewport = sCTkScrollableFrame(dashboard, width=380, height=250, label_text="Telemetry Log")
log_viewport.pack(padx=20, pady=20, fill="both", expand=True)
# Scrolling works immediately -- no activation call needed.
```

Scrolling and State

Scroll bindings are **automatic** and self-maintaining. No activation call is needed, and content added after the widget is placed is picked up on its own.

Scroll handling comes from `ScrollBindingMixin`, the library's single shared implementation — that page covers the four activation mechanisms, the debounced content rebind, the platform models, the nested-frame guard, and the tuning constants (`MAC_SCROLL_SENSITIVITY`, `MAC_SCROLL_MAX_STEP`, `TOUCHPAD_ACCUMULATION_THRESHOLD`). This widget supplies four hooks: `_scroll_target()` resolves the parent canvas via `winfo_parent()`, since it's wrapped by a native `CTkScrollableFrame` that owns the canvas; `_scroll_layers()` assembles the frame, canvas, that canvas's parent, the scrollbar, and the content tree; `_scroll_permitted()` returns `is_scrolling()`; and `_scroll_drag_targets()` returns the internal scrollbar.

It also passes `_parent_frame` as the mixin's `extra_map_widget`, because this widget is a canvas-window child and may never receive `<Map>` itself.

`state` **and** `scroll_enabled` **are two independent axes**, deliberately not collapsed into one. `state` is the user-facing enabled/disabled presentation; `scroll_enabled` is the developer's own intent about whether this frame should scroll at all. Effective scrolling is the AND of the two:

`scroll_enabled`	`state`	**Scrolls?**
`True`	`"normal"`	yes
`True`	`"disabled"`	no
`False`	`"normal"`	no
`False`	`"disabled"`	no

Because state changes never write to `scroll_enabled`, intent survives a round trip. A frame explicitly set non-scrolling stays non-scrolling after `state="disabled"` → `state="normal"`, rather than being silently switched on by the state change.

This is also why `cget("scroll_enabled")` reports stored **intent** while `is_scrolling()` reports the live **effective** result. A frame with `scroll_enabled=True` that has been disabled returns `True` from the former and `False` from the latter.

Temporarily suspending scroll is a supported pattern, useful during bulk content updates where rebinding on every widget added would be wasted work:

```
frame = sCTkScrollableFrame(master)
frame.disable_scroll()
frame.pack(fill="both", expand=True)
for item in many_items:
    sCTkLabelSecondary(frame, text=item).pack()
frame.enable_scroll()
```

Calling `disable_scroll()` before placement correctly suppresses automatic activation rather than being overridden by it — every activation path routes through the same effective-state check. Passing `scroll_enabled=False` to the constructor achieves the same starting state without the separate call.

Methods

Method	Returns	Description
`configure(**kwargs)` / `config(**kwargs)`	`None`	Standard configuration. Accepts `state` and `scroll_enabled` alongside any native option. Both are this library's own properties and are removed before reaching native `CTkScrollableFrame.configure()`, which rejects unrecognized keywords.
`configure(name)`	`tuple`	Pygubu-style single-argument query for `fg_color`, `label_fg_color`, `scrollbar_button_color`, `border_color`, `state`, and `scroll_enabled`. For the color keys the `default` and `current` positions are identical; for `state` and `scroll_enabled` they can differ, since those carry live runtime values.
`cget(name)`	`Any`	Extended to know about `state` and `scroll_enabled`; everything else passes through to the native widget.
`enable_scroll()`	`None`	Turns scroll handling back on. Equivalent to `configure(scroll_enabled=True)`. Safe to call repeatedly.
`disable_scroll()`	`None`	Turns scroll handling off — wheel, trackpad, and scrollbar drag. Equivalent to

Method	Returns	Description
		`configure(scroll_enabled=False)` .
`is_scrolling()`	`bool`	The live effective scroll state — the AND of `scroll_enabled` and `state` . Distinct from `cget("scroll_enabled")` ; see above.
`get_state()`	`str`	Current state, `"normal"` or `"disabled"` . Mirrors the same accessor on `sCTkFrameLabeledPrimary` / `Secondary` .
`winfo_children(include_private=False)`	`list`	By default, filters out children whose exact class name is `"CTkScrollbar"` , `"CTkCanvas"` , or `"Canvas"` — internal furniture this widget creates for its own scrolling machinery. **Confirmed correct by direct, live testing** — printing `get_children()` alongside the widget's internal `_parent_frame.winfo_children()` confirmed the real content widgets are found correctly by this method, and are *not* reachable via `_parent_frame` at all (they're nested deeper, inside the internal scrolling canvas). Pass `include_private=True` for the raw, unfiltered list.
`get_children()`	`list`	Equivalent to `winfo_children(include_private=False)` .
`get_all_children()`	`list`	Equivalent to `winfo_children(include_private=True)` .
`_finalize_split_bindings()`	`None`	**Retained for compatibility; no longer required.** Calling this after placement was once mandatory, and calling it after rebuilding content was the way to bind newly-created rows. The debounced `<Configure>` rebind now handles both automatically. Existing callers are harmless — the underlying toggle is idempotent — but new code shouldn't need it. It respects the current effective state rather than forcing scrolling on.

Platform handling, the nested-frame guard, and how disabling actually blocks scrolling are all documented on the `ScrollBindingMixin` page. Read it before changing anything about scroll behavior here — several of the mechanisms look like needless complications and are not.

Theming (`sCTkThemes.json`)

```
{
    "sCTkScrollableFrame": {
        "border_width": 1.5,
        "border_color": ["#64748B", "#94A3B8"],
        "corner_radius": 8,
        "fg_color": ["#FFFFFF", "#111827"],
        "label_fg_color": "transparent",
        "scrollbar_fg_color": ["#FFFFFF", "#111827"],
        "scrollbar_button_color": ["#64748B", "#4B5563"],
        "scrollbar_button_hover_color": ["#1A4375", "#2471A3"],
        "disabled_map": {
            "border_color": ["#CBD5E1", "#374151"],
            "scrollbar_button_color": ["#CBD5E1", "#1F2937"],
            "scrollbar_button_hover_color": ["#CBD5E1", "#1F2937"]
        }
    }
}
```

`label_fg_color` is deliberately `"transparent"`, so the internal title-row label blends with the frame's own `fg_color` via CustomTkinter's native parent-to-child color propagation, rather than showing its own distinct background.

`disabled_map` **is required, not optional.** Construction raises `KeyError` immediately if `border_color`, `scrollbar_button_color`, or `scrollbar_button_hover_color` is missing from either the top-level block or `disabled_map`. This is the same fail-loud principle used across this project — a theme gap surfaces at construction with a message naming exactly what's missing, rather than being papered over with a guessed color.

The hover color needs a disabled entry because a disabled scrollbar is inert (dragging is blocked), and one that still lit up on hover would falsely advertise itself as draggable. Setting it to the same value as the disabled `scrollbar_button_color`, as above, means it simply doesn't react.

Only the keys that genuinely change when disabled are required in `disabled_map`. `fg_color` is deliberately **not** among them: the content background stays put when disabled, and the border and the now-inert scrollbar carry the visual signal on their own.

Validation is scoped to direct construction. A subclass inheriting this class (such as `sCTkTableview`) reaches this constructor with `final_kw` built from *its own* theme block — `ThemeableWidget`'s run-once guard means the parent never rebuilds it. Validating this widget's keys against a subclass's block would demand scrollbar colors from, say, the `sCTkTableview` block and raise on every construction. Subclasses own their own theme contract and validate it themselves, so this check runs only for the concrete class.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family.

Runtime color overrides persist. `configure()` records any of the tracked theme keys — `fg_color`, `border_color`, `label_fg_color`, `scrollbar_button_color`, `scrollbar_button_hover_color` — into the widget's stored defaults *before* repainting, so an override survives the repaint, later state changes, and appearance-mode switches. This matches CustomTkinter's own semantics, where `configure(fg_color=...)` sticks.

Two consequences worth knowing. Passing a single color replaces the theme's `(light, dark)` tuple for that key, so **that property stops following light/dark** — which is what asking for one specific color means. And `disabled_map` still wins while disabled: an override sets the *normal*-state color.

`scroll_enabled` is deliberately excluded from this write-back, so the Pygubu query can report construction-time default and live value separately.

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkScrollableFrame` directly, construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys native `CTkScrollableFrame` actually accepts (confirmed directly against CustomTkinter's source, which has no fallback `**kwargs` at all — every parameter is explicitly named, so this matters more here than for most widgets).

Example

```
from scustomtkinter import sCTk, sCTkButtonPrimary, sCTkEntryPrimary, sCTkScrollableFrame

if __name__ == "__main__":
    root = sCTk()
    root.title("ScrollableFrame Example")
    root.geometry("450x420")

    log_viewport = sCTkScrollableFrame(root, width=380, height=250, label_text="Telemetry Log")
    log_viewport.pack(padx=20, pady=20, fill="both", expand=True)

    for i in range(12):
        entry = sCTkEntryPrimary(log_viewport, placeholder_text=f"Channel {i + 1}")
        entry.pack(padx=10, pady=5, fill="x")

    # No activation call needed -- scrolling is live as soon as the widget
    # is placed.

    def toggle_lock():
        target = "disabled" if log_viewport.get_state() == "normal" else "normal"
        log_viewport.configure(state=target)
        toggle_btn.configure(text="Enable All" if target == "disabled" else "Disable All")

        # Disabling the frame dims it and stops its scrolling, but does NOT
        # cascade to children -- do that explicitly.
        for child in log_viewport.get_children():
            if hasattr(child, "configure"):
                try:
                    child.configure(state=target)
                except Exception:
                    pass

    toggle_btn = sCTkButtonPrimary(root, text="Disable All", command=toggle_lock)
    toggle_btn.pack(side="bottom", pady=15)

    root.mainloop()
```

Known Limitations

- **Disabling does not cascade to children.** The frame dims and stops scrolling, but child widgets are unaffected — disabling their content is the caller's responsibility, as shown in the example above.
- **The scrollbar is made inert, not hidden.** When disabled it can't be dragged and doesn't respond to hover, but it stays visible. CustomTkinter's scrollbar has no native disabled state to lock, so there's no greyed-out appearance to switch to either. Hiding it entirely is a separate technique, used elsewhere in this project (`sCTkFrameLabeledPrimary / Secondary`) via color-matching and zero width.
- `winfo_children()` **'s default filtering is a class-name check, not an identity check** — a plain, un-themed `customtkinter.CTkCanvas / CTkScrollbar / Canvas` added directly as a real child (not internal furniture) would be incorrectly filtered out too, since its class name matches. Themed `sCTk`-prefixed widgets are unaffected.
- `_parent_frame` **'s** `width / height` **don't reflect the real configured size** — confirmed by direct testing: reading `width / height` through the outer widget correctly returns the real value, but the same properties read through the internal `_parent_frame` attribute always report `0`, regardless of the widget's actual size. `fg_color`, `border_color`, and `border_width` are reliable through either path; `width / height` are not. There's no current code path in this widget that relies on `_parent_frame` for sizing, so this is a trap for future changes, not an active bug.
- **The debounced rebind also runs on genuine resizes.** `<Configure>` doesn't distinguish "a child was added" from "the window was dragged", so resizing rebinds too. It's one coalesced pass rather than one per event, but on a very large content tree it is not free.
- **The nested-frame boundary guard is reasoned, not yet live-tested.** The logic mirrors native CustomTkinter's own guard and is straightforward, but an actual nested scrollable frame (or an `sCTkSelector / sCTkTableview` placed inside another scrollable frame) hasn't been exercised against it yet.
- **A separate** `Canvas` **+ scrollbar placed inside this frame is not guarded.** The nested-frame boundary check keys on `CTkScrollableFrame` specifically. An independent scrolling region built directly on a plain `Canvas` would still be walked into and bound to this frame's handler, stacking an unwanted scroll behavior on top of its own. Guarding this would need an explicit opt-out convention, since a plain `Canvas` has no generic way to declare itself an independent scroll region.
- **Single-argument color queries return** `str(value)`, where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. A known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.

Return to Table of Contents

Controls and Display

These are the basic everyday widgets that you will use frequently. There are some additional selections below where we document the extra widgets included in sCustomTkinter.

sCTkButtonPrimary

Table of Contents

Overview

`sCTkButtonPrimary` is a themeable subclass of `customtkinter.CTkButton` — the most prominent of the library's three button tiers (see also `sCTkButtonSecondary`, `sCTkButtonTertiary`). It adds automatic light/dark theme resolution from `sCTkThemes.json`, a four-state visual model (not just enabled/disabled, but also pressed and alarm), and Pygubu Designer property introspection.

Dark Mode: 

Light Mode:

Constructor

```
sCTkButtonPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkButton` argument (e.g. `text`, `command`, `width`, `height`, `font`, `corner_radius`), or an override for one of the theme keys listed under Theming. Anything not supplied falls back to the `sCTkButtonPrimary` block of `sCTkThemes.json`. Unlike `sCTkComboBox` / `sCTkSegmentedButton`, there's no special extraction step here — `command` and every other native argument flow straight through to construction.

```
save_button = sCTkButtonPrimary(
    master=control_panel,
    text="Save Changes",
```

```
    command=on_save_clicked,
)
save_button.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only the literal string `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value matches neither branch and leaves the state unchanged (no error raised). Disabling correctly blocks both clicks and hover color changes — confirmed by direct, repeated testing.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`set_pressed(pressed)`	`None`	Forces the visual "pressed" look on or off. No-op while disabled or while in alarm state.
`set_alarm_state(active)`	`None`	Forces a high-visibility warning/alarm look on or off. No-op while disabled. Turning alarm **on** clears any active "pressed" state, since alarm takes visual precedence — see Theming for the full precedence order.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, `border_color`, `text_color`, and `hover_color` — with `current` reflecting whichever state (disabled/alarm/pressed/normal) is presently active. Queries for any other single property name fall through to the native `CTkButton.configure`.

Theming (`sCTkThemes.json`)

Four visual states, not two, with a fixed precedence when more than one could apply: **disabled > alarm > pressed > normal**. Only the highest-precedence active state's colors are ever shown — e.g. a button that's both "pressed" and in "alarm" shows alarm colors, and setting alarm while pressed automatically clears the pressed flag.

- **Applied once, at construction** — every key in the widget's theme block, including `width`, `height`, `font`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state change** — `fg_color`, `hover_color`, `text_color`, and `border_color` are recomputed from whichever map matches the current precedence every time you call `state()`,

`set_pressed()`, or `set_alarm_state()`. `border_width`, `corner_radius`, and `font` are **not** re-applied on state changes — they don't vary between states, so they're set once at construction and left alone.

```
{
    "sCTkButtonPrimary": {
        "width": 140,
        "height": 34,
        "font": ["Arial", 15, "normal"],
        "fg_color": ["#1A4375", "#2471A3"],
        "hover_color": ["#112A4B", "#1F618D"],
        "text_color": ["#FFFFFF", "#FFFFFF"],
        "corner_radius": 6,
        "disabled_map": {
            "fg_color": ["#E5E7EB", "#374151"],
            "hover_color": ["#E5E7EB", "#374151"],
            "text_color": ["#94A3B8", "#64748B"]
        },
        "pressed_map": {
            "fg_color": ["#3B5984", "#2E4A75"],
            "hover_color": ["#3B5984", "#2E4A75"],
            "text_color": ["#FFFFFF", "#FFFFFF"]
        },
        "alarm_map": {
            "fg_color": ["#990000", "#E74C3C"],
            "hover_color": ["#990000", "#E74C3C"],
            "text_color": ["#FFFFFF", "#FFFFFF"]
        }
    }
}
```

Note there's no `border_color` anywhere in this block — this button style has no themed border by design (it's a solid-fill button). The widget checks for `border_color` in every state's color swap for consistency with the other themed widgets, but that lookup always resolves to nothing here and is simply skipped.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, the same approach already confirmed working on `sCTkComboBox` and `sCTkSegmentedButton` — so they should correctly follow system/app appearance-mode changes automatically. That specific behavior hasn't been separately re-confirmed for this widget's light/dark toggle, only for its disable/enable cycle.

Disabling this button uses CustomTkinter's native `state="disabled"`, not a manual workaround — that distinction matters here specifically because an earlier version of this widget instead manually unbound mouse events while leaving the native state at `"normal"`, and that approach was directly tested and found to **not** actually block clicks. Native `state="disabled"` is what's required.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
```

```
root.title("ButtonPrimary Example")

base = sCTkFrame(root)
base.pack(expand=True, fill="both", padx=20, pady=20)

save_button = sCTkButtonPrimary(base, text="Save", command=lambda: print("Saved!"))
save_button.pack(pady=10)

def toggle_alarm():
    save_button.set_alarm_state(not save_button.is_alarm)

alarm_toggle = sCTkButtonPrimary(base, text="Toggle Alarm Look", command=toggle_alarm)
alarm_toggle.pack(pady=10)

def toggle_disabled():
    target = "disabled" if save_button.get_state() == "normal" else "normal"
    save_button.state(target)
    disable_toggle.configure(text="Enable Save" if target == "disabled" else "Disable Save")

disable_toggle = sCTkButtonPrimary(base, text="Disable Save", command=toggle_disabled)
disable_toggle.pack(pady=10)

root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged. No exception is raised.
- Calling `configure("fg_color")` (or `"border_color"` / `"text_color"` / `"hover_color"`) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color — e.g. `"('#1A4375', '#2471A3')"` instead of a plain hex string. This is a known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project, not specific to this widget.
- Passing a positional dict to `configure()` is supported and merges into the update; a positional property-name string returns the Tkinter-style query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

Return to Table of Contents

sCTkButtonSecondary

Table of Contents

Overview

`sCTkButtonSecondary` is a themeable subclass of `customtkinter.CTkButton` — a lower-emphasis sibling of `sCTkButtonPrimary` (see also `sCTkButtonTertiary`). It adds automatic light/dark theme resolution from `sCTkThemes.json`, a three-state visual model (normal, disabled, and pressed — no "alarm" state, unlike Primary), and Pygubu Designer property introspection.

Dark Mode: 

Light Mode:

Constructor

```
sCTkButtonSecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkButton` argument (e.g. `text`, `command`, `width`, `height`, `font`, `corner_radius`), or an override for one of the theme keys listed under Theming. As with `sCTkButtonPrimary`, there's no special extraction step — `command` and every other native argument flow straight through to construction.

```
cancel_button = sCTkButtonSecondary(
    master=control_panel,
    text="Cancel",
    command=on_cancel_clicked,
)
cancel_button.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only the literal string `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value matches neither branch and leaves the state unchanged. Disabling uses

Method	Returns	Description
		CTk's native `state="disabled"`, confirmed by direct testing to correctly block both clicks and hover color changes.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`set_pressed(pressed)`	`None`	Forces the visual "pressed" look on or off. No-op while disabled.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, `border_color`, `text_color`, and `hover_color`, with `current` reflecting whichever state (disabled/pressed/normal) is presently active. Queries for any other property name fall through to the native `CTkButton.configure`.

Theming (`sCTkThemes.json`)

Three visual states, with precedence **disabled > pressed > normal** when both could apply.

- **Applied once, at construction** — every key in the widget's theme block, including `font` and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every state change** — `fg_color`, `hover_color`, `border_color`, and `text_color` are recomputed from whichever map matches the current state every time you call `state()` or `set_pressed()`. `border_width`, `corner_radius`, and `font` are **not** re-applied on state changes — they don't vary between states.

```
{
    "sCTkButtonSecondary": {
        "font": ["Arial", 15, "normal"],
        "fg_color": ["#E5E7EB", "#374151"],
        "hover_color": ["#D1D5DB", "#4B5563"],
        "text_color": ["#1F2937", "#F9FAFB"],
        "border_width": 2,
        "border_color": ["#9CA3AF", "#4B5563"],
        "corner_radius": 6,
        "disabled_map": {
            "fg_color": ["#F3F4F6", "#1F2937"],
            "hover_color": ["#F3F4F6", "#1F2937"],
            "border_color": ["#E5E7EB", "#374151"],
            "text_color": ["#94A3B8", "#64748B"]
        },
        "pressed_map": {
            "fg_color": ["#CBD5E1", "#1F2937"],
            "hover_color": ["#CBD5E1", "#1F2937"],
            "border_color": ["#475569", "#94A3B8"],
            "text_color": ["#0F172A", "#FFFFFF"]
        }
    }
```

```
}
```

Unlike `sCTkButtonPrimary` (which has no themed border at all, being a solid-fill button), this style does define `border_color` at every tier — normal, pressed, and disabled all have their own distinct border color.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and `sCTkButtonPrimary`.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonSecondary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("ButtonSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    cancel_button = sCTkButtonSecondary(base, text="Cancel", command=lambda: print("Cancelled"))
    cancel_button.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if cancel_button.get_state() == "normal" else "normal"
        cancel_button.state(target)
        disable_toggle.configure(text="Enable Cancel" if target == "disabled" else "Disable Cancel")

    disable_toggle = sCTkButtonSecondary(base, text="Disable Cancel", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- Calling `configure("fg_color")` (or `"border_color"` / `"text_color"` / `"hover_color"`) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

Return to Table of Contents

sCTkButtonTertiary

Table of Contents

Overview

`sCTkButtonTertiary` is a themeable subclass of `customtkinter.CTkButton` — the lowest-emphasis of the library's three button tiers (see also `sCTkButtonPrimary`, `sCTkButtonSecondary`), styled as an outline button: border and text only, no filled background. It adds automatic light/dark theme resolution from `sCTkThemes.json`, a three-state visual model (normal, disabled, pressed), and Pygubu Designer property introspection.

Dark Mode: 

Light Mode:

Constructor

```
sCTkButtonTertiary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkButton` argument, or an override for one of the theme keys listed under Theming. No special extraction step — `command` and every other native argument flow straight through to construction.

```
learn_more_button = sCTkButtonTertiary(
    master=control_panel,
    text="Learn More",
    command=on_learn_more_clicked,
)
learn_more_button.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value leaves the state unchanged. Uses CTk's native `state="disabled"`, confirmed by direct testing to correctly block clicks and hover color changes.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`set_pressed(pressed)`	`None`	Forces the visual "pressed" look on or off. No-op while disabled.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `border_color`, `text_color`, and `hover_color`. Queries for any other property name fall through to the native `CTkButton.configure`.

Theming (`sCTkThemes.json`)

Three visual states, with precedence **disabled > pressed > normal**.

```
{
    "sCTkButtonTertiary": {
        "font": ["Arial", 15, "normal"],
        "fg_color": "transparent",
        "text_color": ["#3B8ED0", "#1F6AA5"],
        "corner_radius": 6,
        "border_width": 1.25,
        "border_color": ["#64748B", "#94A3B8"],
        "hover_color": ["#E2E8F0", "#1E293B"],
        "disabled_map": {
            "border_color": ["#E5E7EB", "#374151"],
            "text_color": ["#94A3B8", "#64748B"]
        },
        "pressed_map": {
            "fg_color": ["#E2E8F0", "#1E293B"],
            "border_color": ["#112A4B", "#1F618D"],
            "text_color": ["#112A4B", "#1F618D"]
        }
    }
}
```

A few design decisions specific to this outline style, worth knowing before editing this block:

- `fg_color` **is the literal string** `"transparent"` **, not a color pair** — this is a border-and-text-only button by design.
- `disabled_map` **has no** `fg_color` **or** `hover_color` **entries, deliberately.** Since only keys present in a map get swapped, omitting these means the button stays transparent when disabled instead of gaining an unwanted solid gray fill — a filled button (Primary/Secondary) wants that fill; this one doesn't.
- `pressed_map` **has no** `hover_color` **entry either.** Rather than leaving hover color unset while pressed, the widget explicitly falls back to the normal-state `hover_color` in that case.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and `sCTkButtonPrimary`.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonTertiary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("ButtonTertiary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    learn_more_button = sCTkButtonTertiary(base, text="Learn More", command=lambda: print("Clicked"))
    learn_more_button.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if learn_more_button.get_state() == "normal" else "normal"
        learn_more_button.state(target)
        disable_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    disable_toggle = sCTkButtonTertiary(base, text="Disable Learn More", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) silently leaves the state unchanged.
- Calling `configure("fg_color")` (or `"border_color"` / `"text_color"` / `"hover_color"`) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

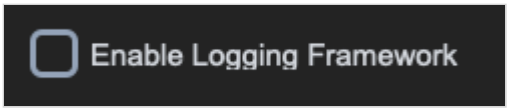
Return to Table of Contents

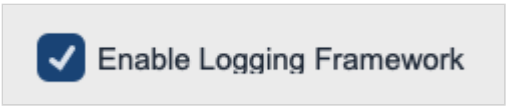
sCTkCheckBox

Table of Contents

Overview

`sCTkCheckBox` is a themeable subclass of `customtkinter.CTkCheckBox`. It adds automatic light/dark theme resolution from `sCTkThemes.json`, including the checkmark color itself, and a distinct enabled/disabled visual state.

Dark Mode: 

Light Mode: 

Constructor

```
sCTkCheckBox(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkCheckBox` argument (e.g. `text`, `command`, `variable`, `onvalue`, `offvalue`), or an override for one of the theme keys listed under Theming. No special extraction step — everything flows straight through to construction.

```
agree_checkbox = sCTkCheckBox(
    master=form_panel,
    text="I agree to the terms",
    command=on_agreement_toggled,
)
agree_checkbox.pack(anchor="w", padx=20, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value leaves the state unchanged.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `border_color`, `text_color`, `hover_color`, and `checkmark_color`. Queries for any other property name fall through to the native `CTkCheckBox.configure`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `hover_color`, `text_color`, `checkmark_color`, `border_width`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkCheckBox": {
        "font": ["Arial", 15, "normal"],
        "border_width": 3,
        "border_color": ["#64748B", "#94A3B8"],
        "fg_color": ["#1A4375", "#2471A3"],
        "hover_color": ["#112A4B", "#1F618D"],
        "text_color": ["#1F2937", "#D1D5DB"],
        "checkmark_color": ["#FFFFFF", "#FFFFFF"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#64748B"],
            "fg_color": ["#E5E7EB", "#374151"],
            "border_color": ["#CBD5E1", "#475569"],
            "checkmark_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

`checkmark_color` was added as a real theme key during this project's audit — previously the theme block didn't define it at all, so the checkmark silently always used CTkCheckBox's native default color regardless of what theme was active, even though the widget's own code was already set up to read it. When disabled, the checkmark dims to match `disabled_map.text_color`'s gray rather than staying bright against a grayed-out box.

Note there's no `hover_color` entry in `disabled_map` — the widget's native disabled state is expected to suppress hover interaction entirely (consistent with the same behavior confirmed on other themed widgets in this project), so a disabled-specific hover color was judged unnecessary; this hasn't been independently re-confirmed for this specific widget.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and `sCTkButtonPrimary`, though not separately re-confirmed for this widget's light/dark toggle specifically.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkCheckBox, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("CheckBox Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    agree_checkbox = sCTkCheckBox(
        base, text="I agree to the terms", command=lambda: print("Toggled")
    )
    agree_checkbox.pack(anchor="w", pady=10)

    def toggle_disabled():
        target = "disabled" if agree_checkbox.get_state() == "normal" else "normal"
        agree_checkbox.state(target)
        disable_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Checkbox", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) silently leaves the state unchanged.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

- Color reapplication after a `state()` change is deferred by one event-loop tick (`after_idle`), as a precaution carried over from a confirmed race condition on `sCTkButtonPrimary` . In virtually all normal usage this is imperceptible, but code that inspects colors in the same tick as a `state()` call may see the pre-change values.

Return to Table of Contents

sCTkEntryPrimary

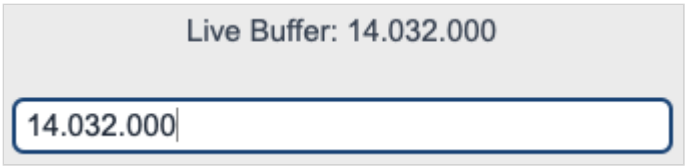
Table of Contents

Overview

`sCTkEntryPrimary` is a themeable subclass of `customtkinter.CTkEntry` — the higher-emphasis of the library's two entry-field tiers (see also `sCTkEntrySecondary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a genuine three-state visual model: normal, readonly, and disabled.

Dark Mode: 

Light Mode:



All three states use CTk's native `state` option (`"normal"` , `"readonly"` , `"disabled"`). `normal` / `disabled` are confirmed correct by direct testing, consistent with every other widget in this library. `readonly` was added specifically to support `sCTkSpinbox` 's own readonly mode correctly (its entry can't be typed into directly, but the increment/decrement arrows stay clickable) — matching real `ttk.Spinbox` semantics, which distinguish readonly (arrows still work) from disabled (nothing works). Confirmed directly against CustomTkinter's own source: native `CTkEntry` already has full, deliberate support for a `"readonly"` state distinct from `"disabled"` — including a placeholder-text rule worth knowing about (see Known Limitations).

Constructor

```
sCTkEntryPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	None	Parent container.
`**kwargs`	—	—	Any native `CTkEntry` argument (e.g. `placeholder_text` , `width`), or an override for one of the theme keys listed under Theming. `state` is extracted and applied after construction rather than passed to the native constructor.

```
freq_entry = sCTkEntryPrimary(
    master=control_panel,
    placeholder_text="Enter frequency (MHz)",
)
freq_entry.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(state_string=None)`	`str`	Gets or sets the widget's visual state. `"normal"` / `"enabled"` / `"active"` all map to `"normal"` ; `"readonly"` maps to `"readonly"` ; `"disabled"` maps to `"disabled"` . All three use CTk's native `state` option.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state` , `fg_color` , `text_color` , `border_color` , and `placeholder_text_color` . Queries for any other property name fall through to the native `CTkEntry.configure` .

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font` and `corner_radius` , is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color` , `border_color` , `text_color` , and `placeholder_text_color` are recomputed from the theme's normal values, its `disabled_map` , or its `readonly_map` every time you call `state()` .

```
{
    "sCTkEntryPrimary": {
```

```
        "font": ["Arial", 15, "normal"],
        "border_width": 1.5,
        "border_color": ["#1A4375", "#64748B"],
        "fg_color": ["#FFFFFF", "#111827"],
        "text_color": ["#1F2937", "#F9FAFB"],
        "placeholder_text_color": ["#94A3B8", "#64748B"],
        "corner_radius": 6,
        "disabled_map": {
            "fg_color": ["#F3F4F6", "#1F2937"],
            "border_color": ["#CBD5E1", "#475569"],
            "text_color": ["#94A3B8", "#64748B"]
        },
        "readonly_map": {
            "fg_color": ["#F8FAFC", "#1F2937"],
            "border_color": ["#64748B", "#94A3B8"],
            "text_color": ["#1F2937", "#F9FAFB"],
            "placeholder_text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

`readonly_map` **requires all four keys** (`fg_color`, `border_color`, `text_color`, `placeholder_text_color`) whenever `readonly` is actually requested — if any are missing, `state("readonly")` raises immediately rather than falling back to a guessed color. This check only runs when readonly is used, so existing code that never requests it is unaffected regardless of whether `readonly_map` is present.

The design intent behind the values above: `text_color` in `readonly_map` deliberately matches `normal`'s `text_color` exactly — readonly means "you can still read this clearly, you just can't edit it," a different message from disabled's "this is inactive." `border_color` is the primary visual cue distinguishing readonly from normal, using a muted "locked" tone distinct from both normal's vivid border and disabled's washed-out one.

`placeholder_text_color` is a genuinely distinct, themed value — not a fallback to `text_color`. This follows CustomTkinter's own convention: in the library's stock `dark-blue` theme, `text_color` is `["gray14", "gray84"]` while `placeholder_text_color` is a visibly more muted `["gray52", "gray62"]`. The value here reuses the muted gray already established throughout this theme file for disabled states — deliberate, but worth knowing if you'd rather placeholder text and disabled text look distinguishable from each other.

Note: CTkEntry has no separate font for placeholder text — it always shares the single `font` property with typed text. This is a real limitation of the underlying widget, not a gap in this theme file; there's no way to make placeholder text use a different font.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
#!/usr/bin/python3

import customtkinter as ctk
```

```
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelPrimary, sCTkEntryPrimary


if __name__ == "__main__":

    root = sCTk()
    root.geometry("450x260")
    root.title("sCTkEntryPrimary Testing Deck")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    # Label notice layer to monitor buffer array activity
    lbl_monitor = sCTkLabelPrimary(base, text="Console monitor active...")
    lbl_monitor.pack(pady=10)

    # Instantiate your custom Primary helper field
    input_field = sCTkEntryPrimary(base, placeholder_text="Enter configuration metadata...")
    input_field.pack(expand=False, fill="x", padx=40, pady=10)

    # Monitor keystrokes live
    input_field.bind("<KeyRelease>", lambda e: lbl_monitor.configure(text=f"Live Buffer: {input_field

    def toggle_operational_state():
        """Toggles the helper input field between normal active and dimmed disabled profiles."""
        current_mode = input_field.get_state()
        target = "disabled" if current_mode == "normal" else "normal"

        # Explicitly testing the dual-routing capability via configure()
        input_field.configure(state=target)
        btn_toggle.configure(
            text="Lock Helper Input (Set 'disabled')" if target == "normal" else "Unlock Helper Input
        print(f"Logged Verification Hook -> input_field.get_state() = {input_field.get_state()}")

    btn_toggle = sCTkButtonPrimary(base, text="Lock Helper Input (Set 'disabled')", command=toggle_op
    btn_toggle.pack(side="bottom", pady=15)

    # Run the interactive boot tracking logs
    print("--- BOOT INITIALIZATION PASSTHROUGH ---")
    input_field.state("disabled")
    print("state (Disabled Pass) =", input_field.get_state())  # Output: disabled

    input_field.state("normal")
    print("state (Normal Pass)   =", input_field.get_state())  # Output: normal
    print("========================================\n")

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"`, `"readonly"`, and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) leaves the internal state flag unchanged, though colors are still harmlessly re-applied.
- `readonly` **never deactivates placeholder text, even on focus** — confirmed directly against CustomTkinter's own source: native `CTkEntry`'s internal placeholder logic explicitly skips clearing the placeholder whenever `state` is `"readonly"`. This makes sense (there's no reason to clear a placeholder for typing on a field that

can't be typed into), but it means a readonly field showing placeholder text will keep showing it indefinitely, regardless of focus.
- The disable/enable-cycle cursor-position fix (`_reset_cursor_if_showing_placeholder`) is also applied on transitions into `readonly` , as a precaution — but this specific transition (unlike normal↔disabled, which is directly confirmed by testing) has not been independently verified. Given the point above, this is likely lower-risk than it might otherwise seem, since a readonly field showing placeholder text stays in that state continuously rather than toggling.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

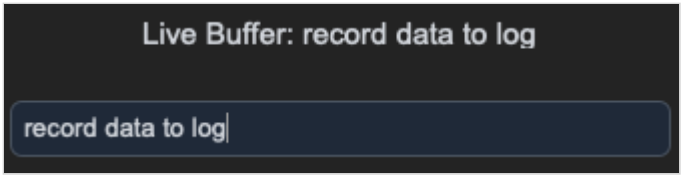
Return to Table of Contents

sCTkEntrySecondary

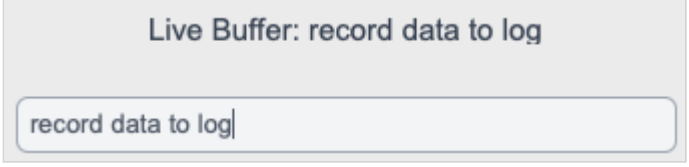
Table of Contents

Overview

`sCTkEntrySecondary` is a themeable subclass of `customtkinter.CTkEntry` — the lower-emphasis of the library's two entry-field tiers (see also `sCTkEntryPrimary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a genuine three-state visual model: normal, readonly, and disabled.

Dark Mode: 

Light Mode:



All three states use CTk's native `state` option (`"normal"` , `"readonly"` , `"disabled"`) — see `sCTkEntryPrimary` 's documentation for the full rationale and the readonly-specific placeholder behavior worth knowing about.

Constructor

```
sCTkEntrySecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkEntry` argument, or an override for one of the theme keys listed under Theming. `state` is extracted and applied after construction rather than passed to the native constructor.

```
notes_entry = sCTkEntrySecondary(
    master=control_panel,
    placeholder_text="Optional notes",
)
notes_entry.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(state_string=None)`	`str`	Gets or sets the widget's visual state. `"normal"` / `"enabled"` / `"active"` all map to `"normal"` ; `"readonly"` maps to `"readonly"` ; `"disabled"` maps to `"disabled"` . All three use CTk's native `state` option.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state` , `fg_color` , `text_color` , `border_color` , and `placeholder_text_color` . Queries for any other property name fall through to the native `CTkEntry.configure` .

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font` and `corner_radius` , is merged with any matching keyword arguments and applied when the widget is built.

- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `text_color`, and `placeholder_text_color` are recomputed from the theme's normal values, its `disabled_map`, or its `readonly_map` every time you call `state()`.

```
{
    "sCTkEntrySecondary": {
        "font": ["Arial", 13, "normal"],
        "border_width": 1,
        "border_color": ["#9CA3AF", "#4B5563"],
        "fg_color": ["#F3F4F6", "#1F2937"],
        "text_color": ["#4B5563", "#D1D5DB"],
        "placeholder_text_color": ["#94A3B8", "#64748B"],
        "corner_radius": 6,
        "disabled_map": {
            "fg_color": ["#F3F4F6", "#0B0F19"],
            "border_color": ["#CBD5E1", "#374151"],
            "text_color": ["#94A3B8", "#64748B"]
        },
        "readonly_map": {
            "fg_color": ["#F3F4F6", "#1F2937"],
            "border_color": ["#64748B", "#6B7280"],
            "text_color": ["#4B5563", "#D1D5DB"],
            "placeholder_text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

`readonly_map` **requires all four keys** whenever `readonly` is actually requested — see `sCTkEntryPrimary`'s docs for the full requirement and design rationale (`text_color` deliberately matches normal exactly; `fg_color` stays close to normal too, since Secondary's normal state is already fairly subtle and there wasn't much room to differentiate further without it starting to look disabled instead).

Same rationale as `sCTkEntryPrimary`: `placeholder_text_color` is a genuinely distinct, themed value, following CustomTkinter's own convention of giving placeholder text a visibly more muted color than typed text. CTkEntry has no separate font for placeholder text — it always shares the single `font` property with typed text; that's a real limitation of the underlying widget, not a gap in this theme file.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkEntrySecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("EntrySecondary Example")
```

```
base = sCTkFrame(root)
base.pack(expand=True, fill="both", padx=20, pady=20)

notes_entry = sCTkEntrySecondary(base, placeholder_text="Optional notes")
notes_entry.pack(fill="x", pady=10)

def toggle_disabled():
    target = "disabled" if notes_entry.get_state() == "normal" else "normal"
    notes_entry.state(target)
    disable_toggle.configure(text="Enable Field" if target == "disabled" else "Disable Field")

disable_toggle = sCTkButtonPrimary(base, text="Disable Field", command=toggle_disabled)
disable_toggle.pack(pady=10)

root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"`, `"readonly"`, and `"normal"` / `"enabled"` / `"active"`; any other value leaves the internal state flag unchanged, though colors are still harmlessly re-applied.
- `readonly` **never deactivates placeholder text, even on focus** — confirmed directly against CustomTkinter's own source; see `sCTkEntryPrimary`'s docs for the full explanation.
- The disable/enable-cycle cursor-position fix is also applied on transitions into `readonly`, as a precaution not independently verified the way normal↔disabled was — see `sCTkEntryPrimary`'s docs for why this is likely lower-risk than it sounds.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.

Return to Table of Contents

sCTkLabelPrimary

Table of Contents

Overview

`sCTkLabelPrimary` is a themeable subclass of `customtkinter.CTkLabel` — the most prominent of the library's three label tiers (see also `sCTkLabelSecondary`, `sCTkLabelTertiary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Since labels have no native interactivity to block, "disabled" here is purely a text-color dim — there's no click-blocking concern the way there is for buttons or checkboxes.

Dark Mode: 

Light Mode:

Constructor

```
sCTkLabelPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	`state` is pulled out and applied after construction rather than passed to the native widget. Everything else is any native `CTkLabel` argument (e.g. `text`, `font`), or an override for one of the theme keys listed under Theming.

```
console_header = sCTkLabelPrimary(
    master=control_panel,
    text="Main Console",
)
console_header.pack(expand=True, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only the literal string `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value matches neither branch and leaves the state unchanged.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`cget("state")`	`str`	Returns the current state, same as `get_state()`. Intercepted specially because native `CTkLabel` has no real `"state"` option

Method	Returns	Description
		to query — without this override, `cget("state")` would raise.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, and `text_color`. Queries for any other property name fall through to the native `CTkLabel.configure`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkLabelPrimary": {
        "font": ["Arial", 18, "bold"],
        "fg_color": "transparent",
        "text_color": ["#111827", "#F9FAFB"],
        "disabled_map": {
            "text_color": ["#64748B", "#94A3B8"]
        }
    }
}
```

`text_color` is required in whichever map is active — if it's missing, `_update_current_visual_state()` raises immediately rather than falling back to a default. This is deliberate: this project's design is for `ThemeableWidget`-based widgets to fail hard on incomplete theme data, not paper over it with a fallback color.

Primary's disabled color is intentionally the least muted of the three label tiers — by design, a disabled `sCTkLabelPrimary` should still read as more prominent than a disabled `sCTkLabelSecondary` or `sCTkLabelTertiary`, echoing the hierarchy the three tiers already have when enabled.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelPrimary, sCTkButtonPrimary
```

```
if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("LabelPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    header = sCTkLabelPrimary(base, text="Main Console")
    header.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if header.get_state() == "normal" else "normal"
        header.state(target)
        disable_toggle.configure(text="Enable Header" if target == "disabled" else "Disable Header")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Header", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- Calling `configure("fg_color")` or `configure("text_color")` returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `text_color`, and falls through to the native widget's `configure()` for anything else.

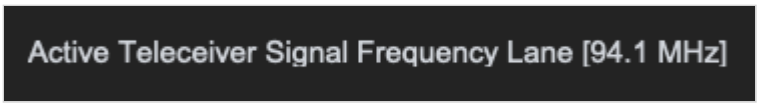
Return to Table of Contents

sCTkLabelSecondary

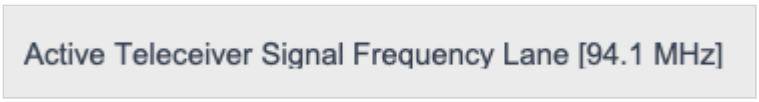
Table of Contents

Overview

`sCTkLabelSecondary` is a themeable subclass of `customtkinter.CTkLabel` — the mid-emphasis tier of the library's three label styles, between `sCTkLabelPrimary` and `sCTkLabelTertiary`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Since labels have no native interactivity to block, "disabled" here is purely a text-color dim.

Dark Mode: 

Light Mode:



Constructor

```
sCTkLabelSecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	`state` is pulled out and applied after construction rather than passed to the native widget. Everything else is any native `CTkLabel` argument (e.g. `text`, `font`), or an override for one of the theme keys listed under Theming.

```
panel_sub_label = sCTkLabelSecondary(
    master=control_panel,
    text="Status: Active",
)
panel_sub_label.pack(expand=True, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only the literal string `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value matches neither branch and leaves the state unchanged.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`cget("state")`	`str`	Returns the current state, same as `get_state()`. Intercepted specially because native `CTkLabel` has no real `"state"` option to query — without this override, `cget("state")` would raise.

Method	Returns	Description
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, and `text_color`. Queries for any other property name fall through to the native `CTkLabel.configure`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkLabelSecondary": {
        "font": ["Arial", 15, "normal"],
        "fg_color": "transparent",
        "text_color": ["#374151", "#D1D5DB"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

Unlike `sCTkLabelPrimary` / `sCTkLabelTertiary`'s history, this widget's `_update_current_visual_state()` was already never falling back to a default — but it also wasn't raising, either; a missing `text_color` would just silently never get set. As of this project's audit, all three label widgets now raise explicitly if `text_color` is missing from whichever map is active, per this project's design of failing hard on incomplete theme data rather than working around it.

Secondary's disabled color sits deliberately in the middle of the three label tiers' disabled states — less muted than Tertiary's, more muted than Primary's — mirroring the emphasis hierarchy the three tiers already have when enabled.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelSecondary, sCTkButtonPrimary

if __name__ == "__main__":
```

```
root = sCTk()
root.geometry("400x250")
root.title("LabelSecondary Example")

base = sCTkFrame(root)
base.pack(expand=True, fill="both", padx=20, pady=20)

status_label = sCTkLabelSecondary(base, text="Status: Active")
status_label.pack(pady=10)

def toggle_disabled():
    target = "disabled" if status_label.get_state() == "normal" else "normal"
    status_label.state(target)
    disable_toggle.configure(text="Enable Label" if target == "disabled" else "Disable Label")

disable_toggle = sCTkButtonPrimary(base, text="Disable Label", command=toggle_disabled)
disable_toggle.pack(pady=10)

root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- Calling `configure("fg_color")` or `configure("text_color")` returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `text_color`, and falls through to the native widget's `configure()` for anything else.

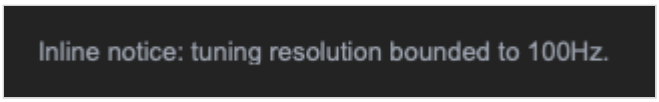
Return to Table of Contents

sCTkLabelTertiary

Table of Contents

Overview

`sCTkLabelTertiary` is a themeable subclass of `customtkinter.CTkLabel` — the lowest-emphasis of the library's three label tiers (see also `sCTkLabelPrimary`, `sCTkLabelSecondary`), intended for inline descriptions, sub-legends, or auxiliary notices. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Since labels have no native interactivity to block, "disabled" here is purely a text-color dim.

Dark Mode: 

Light Mode:

Constructor

```
sCTkLabelTertiary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	`state` is pulled out and applied after construction rather than passed to the native widget. Everything else is any native `CTkLabel` argument (e.g. `text`, `font`), or an override for one of the theme keys listed under Theming.

```
panel_legend = sCTkLabelTertiary(
    master=control_panel,
    text="Note: values update every 5 seconds.",
)
panel_legend.pack(expand=True, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only the literal string `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Any other value matches neither branch and leaves the state unchanged.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`cget("state")`	`str`	Returns the current state, same as `get_state()`. Intercepted specially because native `CTkLabel` has no real `"state"` option

Method	Returns	Description
		to query — without this override, `cget("state")` would raise.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, and `text_color`. Queries for any other property name fall through to the native `CTkLabel.configure`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkLabelTertiary": {
        "font": ["Arial", 13, "normal"],
        "fg_color": "transparent",
        "text_color": ["#4B5563", "#9CA3AF"],
        "disabled_map": {
            "text_color": ["#CBD5E1", "#475569"]
        }
    }
}
```

`text_color` is required in whichever map is active — if it's missing, `_update_current_visual_state()` raises immediately rather than falling back to a default. This is deliberate: this project's design is for `ThemeableWidget`-based widgets to fail hard on incomplete theme data, not paper over it with a fallback color.

Tertiary's disabled color is intentionally the most muted of the three label tiers — by design, a disabled `sCTkLabelTertiary` should read as the least prominent of the three even while disabled, echoing the hierarchy the three tiers already have when enabled.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkLabelTertiary, sCTkButtonPrimary
```

```
if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("LabelTertiary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    legend = sCTkLabelTertiary(base, text="Note: values update every 5 seconds.")
    legend.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if legend.get_state() == "normal" else "normal"
        legend.state(target)
        disable_toggle.configure(text="Enable Legend" if target == "disabled" else "Disable Legend")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Legend", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value (including typos) matches neither branch and silently leaves the state unchanged.
- Calling `configure("fg_color")` or `configure("text_color")` returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `text_color`, and falls through to the native widget's `configure()` for anything else.

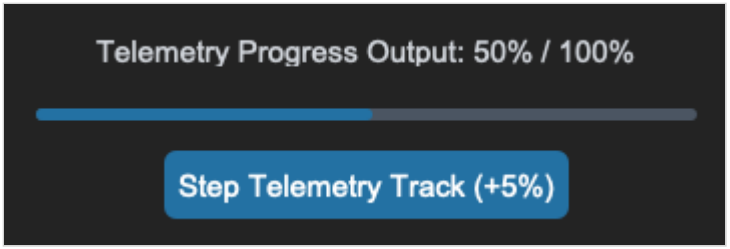
Return to Table of Contents

sCTkProgressBar

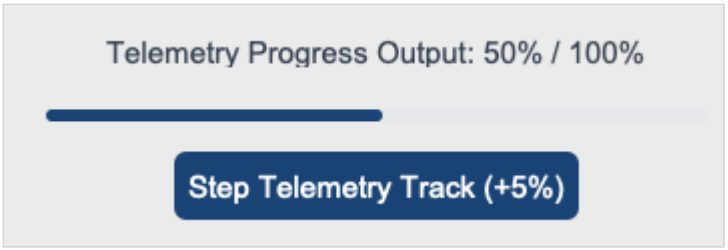
Table of Contents

Overview

`sCTkProgressBar` is a themeable subclass of `customtkinter.CTkProgressBar`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a purely visual "disabled" state — progress bars have no click behavior to block, so disabling one only dims its colors.

Dark Mode: 

Light Mode:



Constructor

```
sCTkProgressBar(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	`state` is pulled out and applied after construction. Everything else is any native `CTkProgressBar` argument, or an override for one of the theme keys listed under Theming.

```
signal_meter = sCTkProgressBar(master=control_panel)
signal_meter.pack(fill="x", padx=40, pady=10)
signal_meter.set(0.65)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's visual "disabled" state. Unlike most widgets in this library, any string is accepted and stored as-is (lowercased) — there's no validation against a fixed set of values. Only the literal `"disabled"` actually changes colors; anything else is treated as "not disabled".

Method	Returns	Description
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`cget("state")`	`str`	Returns the current state, same as `get_state()`. Intercepted specially because native `CTkProgressBar` has no real `"state"` option to query — without this override, `cget("state")` would raise.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()` rather than the native option; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, `progress_color`, and `border_color`. Queries for any other property name fall through to the native `CTkProgressBar.configure`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `width`, `height`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `progress_color`, `border_width`, and `corner_radius` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkProgressBar": {
        "width": 200,
        "height": 6,
        "fg_color": ["#E5E7EB", "#4B5563"],
        "progress_color": ["#1A4375", "#2471A3"],
        "corner_radius": 100,
        "disabled_map": {
            "fg_color": ["#CBD5E1", "#374151"],
            "progress_color": ["#94A3B8", "#4B5563"]
        }
    }
}
```

There's no `border_color` anywhere in this theme block, even though the repaint loop checks for one — this style simply has no themed border, the same situation as `sCTkButtonPrimary`'s `border_color`.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkProgressBar, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("ProgressBar Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    meter = sCTkProgressBar(base)
    meter.pack(fill="x", pady=10)
    meter.set(0.65)

    def toggle_disabled():
        target = "disabled" if meter.get_state() == "normal" else "normal"
        meter.state(target)
        disable_toggle.configure(text="Enable Meter" if target == "disabled" else "Disable Meter")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Meter", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` performs no validation at all — any string you pass is stored verbatim; only `"disabled"` actually changes the rendered colors.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

Return to Table of Contents

sCTkRadioButton

Table of Contents

- Known Limitations

Overview

`sCTkRadioButton` is a themeable subclass of `customtkinter.CTkRadioButton`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state, using CustomTkinter's native `state="disabled"` — confirmed by direct testing to correctly block clicks.

Dark Mode: 

Light Mode:

Constructor

```
sCTkRadioButton(master=None, variable=None, value=None, command=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`variable`	`tkinter.Variable`	`None`	Shared variable used for mutual exclusion within a group.
`value`	any	`None`	This button's value — the button shows as selected when `variable` equals this.
`command`	`callable`	`None`	Called when the button is selected.
`**kw`	—	—	Any native `CTkRadioButton` argument, or an override for one of the theme keys listed under Theming.

```
mode_var = tkinter.StringVar(value="AM")
am_radio = sCTkRadioButton(control_panel, text="AM", variable=mode_var, value="AM")
fm_radio = sCTkRadioButton(control_panel, text="FM", variable=mode_var, value="FM")
am_radio.pack(anchor="w")
fm_radio.pack(anchor="w")
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: `variable` / `value` / `command` / `state` are each routed individually. Rebinding `variable` / `value` to a new group after construction is confirmed correct by direct testing — it properly tears down the old variable's binding and establishes real mutual exclusion in the new group, with no leftover coupling to the original group. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `border_color`, `text_color`, and `hover_color`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `hover_color`, `text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkRadioButton": {
        "font": ["Arial", 15, "normal"],
        "text_color": ["#374151", "#D1D5DB"],
        "border_width_unchecked": 4,
        "border_width_checked": 6,
        "border_color": ["#64748B", "#94A3B8"],
        "fg_color": ["#1A4375", "#2471A3"],
        "hover_color": ["#112A4B", "#1F618D"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#64748B"],
            "fg_color": ["#CBD5E1", "#374151"],
            "border_color": ["#CBD5E1", "#475569"]
        }
    }
}
```

`border_width_unchecked` and `border_width_checked` are real, top-level-only theme keys (not in `disabled_map`) that control the button's border thickness based on whether it's currently the selected button in its group — thicker when checked, to show the filled dot. They're applied once at construction and left alone afterward; the native widget switches between them internally based on the checked/unchecked state, so no repaint-time logic is needed here.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import tkinter
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkRadioButton, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("RadioButton Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    mode_var = tkinter.StringVar(value="AM")
    am_radio = sCTkRadioButton(base, text="AM", variable=mode_var, value="AM")
    am_radio.pack(anchor="w", pady=5)
    fm_radio = sCTkRadioButton(base, text="FM", variable=mode_var, value="FM")
    fm_radio.pack(anchor="w", pady=5)

    def toggle_disabled():
        target = "disabled" if am_radio.get_state() == "normal" else "normal"
        am_radio.state(target)
        fm_radio.state(target)
        disable_toggle.configure(text="Enable Group" if target == "disabled" else "Disable Group")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Group", command=toggle_disabled)
    disable_toggle.pack(pady=15)

    root.mainloop()
```

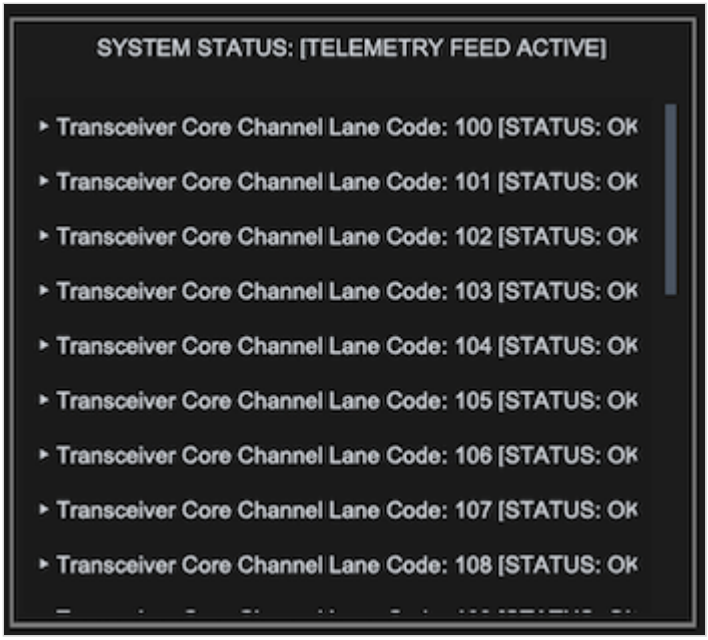
Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value matches neither branch, though colors are still harmlessly re-applied.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for `state` / `fg_color` / `border_color` / `text_color` / `hover_color`, and falls through to the native widget's `configure()` for anything else.

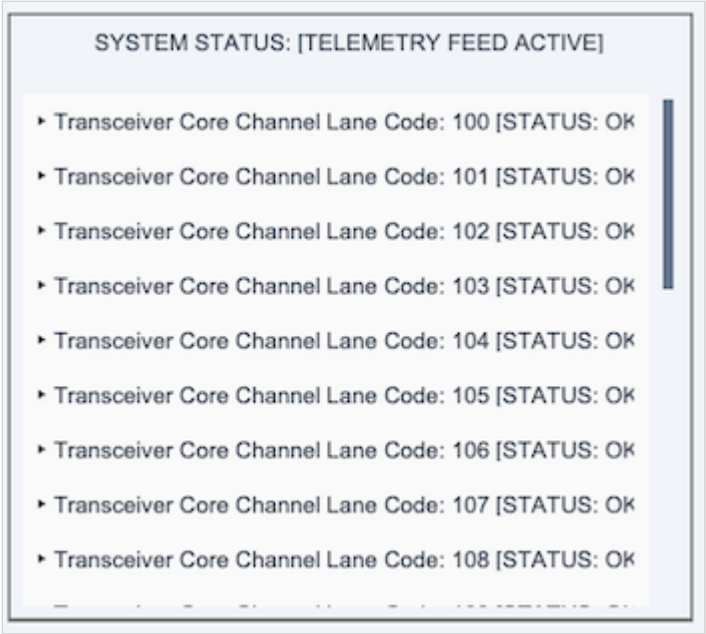
Return to Table of Contents

sCTkScrollbar

`sCTkScrollbar` is a themeable scrollbar — a subclass of `ctk.CTkScrollbar` with color resolution from `sCTkThemes.json` and orientation-aware default sizing. It's designed to pair with `sCTkScrollArea`, which needs an external scrollbar, but works anywhere a `CTkScrollbar` would.

Dark Mode: 

Light Mode:



This widget contains no scroll-handling logic. It's a scrollbar: it renders a draggable bar and reports its position. Wheel and trackpad handling belongs to the scrolling container — see `ScrollBindingMixin`. An earlier version of this page credited the scrollbar with an "inertial micro-delta aggregator"; that logic lives in `sCTkScrollArea`, not here.

Table of Contents

Constructor

```
scrollbar = sCTkScrollbar(master=None, orientation="vertical", **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`orientation`	`"vertical"` / `"horizontal"`	`"vertical"`	Layout direction. Sets a default `width` of 14 when vertical, or `height` of 14 when horizontal.
`**kwargs`	—	—	Any native `CTkScrollbar` argument, or a theme-key override (see Theming).

Methods

Method	Returns	Description
`configure(**kwargs)` / `config(**kwargs)`	`None`	Standard configuration. Overrides of `button_color` and `button_hover_color` **persist** — see below.
`configure(name)`	`tuple`	Pygubu-style single-argument query for `button_color` and `button_hover_color`. Any other name passes through to the native widget.

Runtime color overrides persist. `configure()` records the tracked theme keys into the widget's stored defaults *before* repainting, so an override survives the repaint and later appearance-mode switches. This matches CustomTkinter's own semantics, where `configure(button_color=...)` sticks.

This was previously broken. `_apply_custom_theme_colors()` runs on every `configure()` call and re-pushes both colors from the stored defaults — and since `configure()` never wrote to those defaults, `configure(button_color="red")` applied red and then had it overwritten on the very next line. Runtime color overrides silently did nothing.

Two consequences worth knowing: passing a single color replaces the theme's `(light, dark)` tuple for that key, so **that property stops following light/dark** — which is what asking for one specific color means.

The single-argument query was also previously broken. The implementation tested `if args and isinstance(args, dict)`, but `args` is always a tuple, so that branch was dead and there was no query branch at all — `configure("button_color")` silently returned `None` instead of a property tuple.

Theming (`sCTkThemes.json`)

```
{
    "sCTkScrollbar": {
        "corner_radius": 4,
        "fg_color": "transparent",
        "button_color": ["#64748B", "#4B5563"],
        "button_hover_color": ["#1A4375", "#2471A3"]
    }
}
```

`button_color` is the bar itself; `button_hover_color` is the bar under the cursor. `fg_color` is the track behind it.

`button_color` **and** `button_hover_color` **are required.** Construction raises `KeyError` naming the missing one. These previously carried hardcoded fallbacks, so a theme block missing either would silently substitute a plausible guess rather than failing loudly.

`orientation` may also be supplied from the theme block. It's read from the resolved keywords rather than the raw constructor dict, so it's picked up whichever way it arrives — an earlier version read the raw dict *after* `ThemeableWidget` had processed it, which risked a horizontal scrollbar silently getting a default `width` instead of `height` .

Colors are passed through as raw `(light, dark)` tuples rather than resolved ahead of time, so they follow appearance-mode changes automatically.

There is no `disabled_map` **, and no disabled state.** CustomTkinter's scrollbar has none to lock. Containers that need an inert scrollbar block dragging at the binding level instead and dim the bar themselves — see `ScrollBindingMixin` .

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import (sCTk, sCTkFrame, sCTkLabelSecondary,
                            sCTkScrollbar, sCTkScrollArea)

if __name__ == "__main__":
    root = sCTk()
    root.geometry("480x420")
    root.title("sCTkScrollbar Validation Bench")

    main_layout = sCTkFrame(root, border_width=2)
    main_layout.pack(expand=True, fill="both", padx=15, pady=15)

    scrollbar = sCTkScrollbar(main_layout, orientation="vertical")
    scrollbar.pack(side="right", fill="y", padx=(5, 10), pady=10)

    content_chassis = sCTkFrame(main_layout, border_width=0, fg_color="transparent")
    content_chassis.pack(side="left", fill="both", expand=True, padx=(10, 0), pady=10)
```

```
scroll_view = sCTkScrollArea(content_chassis)
scroll_view.pack(fill="both", expand=True)

for i in range(25):
    sCTkLabelSecondary(
        scroll_view.scroll_content,
        text=f"Transceiver channel {100 + i} [OK]"
    ).pack(anchor="w", padx=10, pady=4)

scroll_view.hook_scrollbar(scrollbar)

root.mainloop()
```

Known Limitations

- **No disabled state** — see Theming.
- **Only** `button_color` **and** `button_hover_color` **are tracked** for the persist-on- `configure()` behavior. Other properties still repaint from the theme.

Return to Table of Contents

sCTkSegmentedButton

Table of Contents

Overview

`sCTkSegmentedButton` is a themeable subclass of `customtkinter.CTkSegmentedButton` — a horizontal strip of connected text buttons where selecting one automatically unselects the others, similar to a row of radio buttons. It adds automatic light/dark theme resolution from `sCTkThemes.json`, a distinct enabled/disabled visual state, and per-segment text-color handling for the currently selected segment.

Dark Mode: 

Light Mode: 

Constructor

```
sCTkSegmentedButton(master=None, values=None, variable=None, command=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`values`	`list[str]`	native default (empty)	The segment labels.
`variable`	`tkinter.StringVar`	`None`	Optional variable bound to the current selection.
`command`	`callable`	`None`	Called with the selected value when the user picks a segment.
`**kwargs`	—	—	Any native `CTkSegmentedButton` argument, or an override for `fg_color` / `selected_color`.

> **Not settable via constructor:** `unselected_color`, `unselected_hover_color`, `border_width`, `border_color`, and `selected_color_padding` are intentionally stripped out before the native widget is built, regardless of what you pass in. They're only ever pulled from `sCTkThemes.json` — see Known Limitations for why setting them later doesn't stick either.

```
mode_selector = sCTkSegmentedButton(
    master=control_panel,
    values=["Alpha", "Beta", "Gamma"],
    command=on_mode_changed,
)
mode_selector.pack(fill="x", padx=40, pady=10)
```

Colors are first applied roughly 15ms after construction, not immediately — the widget's individual segment buttons don't exist yet at the moment `sCTkSegmentedButton.__init__` returns, so the initial color pass is deferred with `self.after(15, ...)`. In virtually all normal usage this is unnoticeable, but code that inspects a segment's color in the same tick as construction may see it before this pass runs.

Methods

Method	Returns	Description
`get()`	`str`	Currently selected value (native `CTkSegmentedButton` behavior).
`set(value)`	`None`	Selects a segment programmatically and repaints (native `set()`, plus a theme-color refresh).
`state(mode=None)`	`str | None`	Gets or sets the widget's enabled/disabled visual state. Only the literal string `"disabled"` (case-insensitive) is treated as disabled

Method	Returns	Description
		— **any other value, including typos, is treated as** `"normal"` with no error raised. Called with no argument, returns the current state as a lowercase string.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	—	Standard widget configuration. Passing `state=...` routes through `state()` rather than the native `state` option. A single positional argument (string or dict) is accepted, but — unlike `sCTkComboBox` / `sCTkCheckBox` — a single property-name string does **not** return a Pygubu-style query tuple here; it's forwarded directly to the native widget instead. (Broader Pygubu query-behavior gaps across the library are a known follow-up item, not specific to this widget.)

Theming (`sCTkThemes.json`)

Same two-tier model as the other themed widgets:

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built, *except* the five keys listed under Constructor, which are deliberately excluded and applied only through the mechanism below.
- **Re-applied on** `state()` **change, on selection, and on click** — `fg_color` , `selected_color` , `unselected_color` , `unselected_hover_color` , and each segment's `text_color` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()` , `set()` , or click a segment.

Colors are stored and passed through as raw `(light, dark)` tuples rather than being resolved to a single value ahead of time, which means they correctly follow system/app appearance-mode changes automatically — including while the widget is disabled. Confirmed by direct testing: toggling light/dark mode on a disabled, pre-selected widget repaints it immediately, with no manual intervention needed.

```
{
    "sCTkSegmentedButton": {
        "fg_color": ["#4F75A2", "#2B4C7E"],
        "selected_color": ["#1A4375", "#3A6FA2"],
        "unselected_hover_color": ["#3A5C85", "#3A5F8C"],
        "text_color": ["#FFFFFF", "#FFFFFF"],
        "disabled_map": {
            "fg_color": ["#B2B9BC", "#222527"],
            "selected_color": ["#70777B", "#45494D"],
            "text_color": ["#94A3B8", "#64748B"],
            "selected_text_color": ["#1F2937", "#FFFFFF"]
        }
    }
}
```

A couple of design decisions worth knowing if you're editing this block:

- **Unselected segments always match `fg_color`, by design** — there's no independent `unselected_color` key. Segments are meant to blend into the widget's own background color rather than appear individually distinct or transparent.
- **Hover is fully suppressed while disabled**, at the native widget level — confirmed by direct testing. There's no `disabled_map` entry for hover colors because a disabled segment never fires a hover event in the first place; any color set there would never be visible.
- **The selected segment keeps a distinct, more prominent text color while disabled** (`disabled_map.selected_text_color`), so you can still tell which option was chosen even though the whole control is grayed out. Every other segment's disabled text color comes from the plain `disabled_map.text_color`.

Every key above is required, at the top level or in `disabled_map` as shown. Construction raises `KeyError` naming the missing one.

The colour lookups previously carried hardcoded fallbacks. Those were unreachable given the theme block as shipped — every key they guarded was present — but that was a property of the *theme file*, not the code. Deleting a key would have silently activated a fallback, producing a plausible-looking wrong colour instead of the loud failure every other widget now gives. They're gone.

`selected_hover_color` has also been removed from the block. It was present in the theme but read by no code path at all — dead data, the same situation `pointer_color` was in for the dial family. A selected segment's hover behaviour comes from CustomTkinter's own default, since this widget never set it.

Every color in this widget comes from `sCTkThemes.json` — there are no hardcoded hex values left in the widget's own source.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonPrimary, sCTkSegmentedButton

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x300")
    root.title("SegmentedButton Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    widget = sCTkSegmentedButton(
        base,
        values=["Alpha", "Beta", "Gamma"],
        command=lambda choice: print(f"Selected: {choice}"),
    )
    widget.pack(expand=True, fill="none", padx=10, pady=10)
    widget.set("Beta")

    def toggle_widget_state():
        target = "disabled" if widget.get_state() == "normal" else "normal"
        widget.configure(state=target)
        btn_toggle.configure(text="Enable" if target == "disabled" else "Disable")
```

```
btn_toggle = sCTkButtonPrimary(base, text="Disable", command=toggle_widget_state)
btn_toggle.pack(side="bottom", pady=15)

root.mainloop()
```

Known Limitations

- `state()` treats any value other than `"disabled"` (case-insensitive) as `"normal"` — including typos like `"disbaled"`. No exception is raised and no warning is logged.
- `unselected_color`, `unselected_hover_color`, `border_width`, `border_color`, and `selected_color_padding` cannot be set via constructor kwargs (see Constructor); the color-related ones only take effect through the theme file or the widget's own repaint logic.
- Calling `configure("some_property_name")` with a single property name does not return a Pygubu-style query tuple the way `sCTkComboBox` / `sCTkCheckBox` do; it's forwarded to the native widget's `configure()`, which — per the wider Pygubu-query investigation set aside earlier in this project — does not support single-argument property queries at all for most properties.

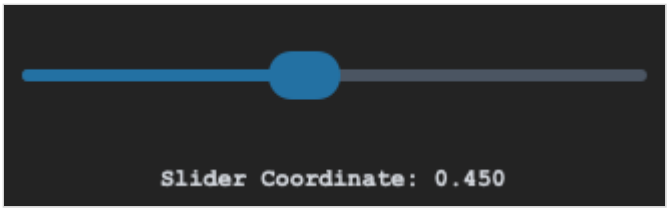
Return to Table of Contents

sCTkSlider

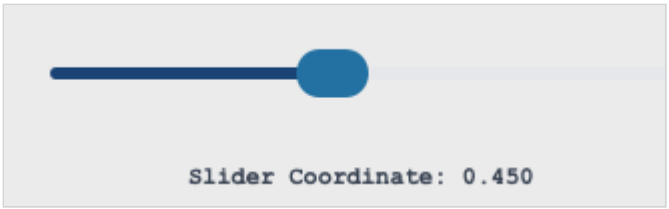
Table of Contents

Overview

`sCTkSlider` is a themeable subclass of `customtkinter.CTkSlider`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. Unlike every other widget in this library, its state isn't tracked in a separate instance attribute — it reads and writes CustomTkinter's own native `state` property directly, treating it as the single source of truth.

Dark Mode: 

Light Mode:



Constructor

```
sCTkSlider(master=None, command=None, variable=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`command`	`callable`	`None`	Called with the current value as the slider is dragged.
`variable`	`tkinter.Variable`	`None`	Optional variable bound to the current value.
`**kw`	—	—	Any native `CTkSlider` argument (e.g. `from_` , `to` , `number_of_steps`), or an override for one of the theme keys listed under Theming.

```
volume_slider = sCTkSlider(
    master=control_panel,
    command=on_volume_changed,
)
volume_slider.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Queries read directly from the native widget's own `state` property rather than a parallel attribute. Setting forwards to `configure(state=mode)` , which reaches the native widget's own state handling — confirmed by direct testing to correctly block interaction.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.

Method	Returns	Description
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: `command` / `variable` are routed individually; `state` is **not** specially intercepted — it flows straight through to the native widget's own `configure()`, which is what makes this widget's disable mechanism correct. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `progress_color`, `button_color`, and `button_hover_color`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `width`, `height`, `button_length`, and `border_width`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` / `configure(state=...)` change** — `fg_color`, `progress_color`, `button_color`, and `button_hover_color` are recomputed from the theme's normal values or its `disabled_map`.

```
{
    "sCTkSlider": {
        "width": 200,
        "height": 24,
        "button_length": 12,
        "border_width": 9,
        "fg_color": ["#E5E7EB", "#4B5563"],
        "progress_color": ["#1A4375", "#2471A3"],
        "button_color": ["#2471A3", "#2471A3"],
        "button_hover_color": ["#112A4B", "#1F618D"],
        "disabled_map": {
            "fg_color": ["#CBD5E1", "#374151"],
            "progress_color": ["#CBD5E1", "#4B5563"],
            "button_color": ["#94A3B8", "#4B5563"]
        }
    }
}
```

`button_color` is the same value for both light and dark mode here — a deliberate accent color that doesn't shift with appearance mode. `disabled_map` has no `button_hover_color` entry; while disabled, the widget explicitly forces `button_hover_color` to match `button_color` instead, since hover can't trigger once natively disabled anyway.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```python
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkSlider, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Slider Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    volume_slider = sCTkSlider(base, command=lambda v: print(f"Value: {v:.2f}"))
    volume_slider.pack(fill="x", pady=10)

    def toggle_disabled():
        target = "disabled" if volume_slider.get_state() == "normal" else "normal"
        volume_slider.state(target)
        disable_toggle.configure(text="Enable Slider" if target == "disabled" else "Disable Slider")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Slider", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for four specific properties, and falls through to the native widget's `configure()` for anything else.

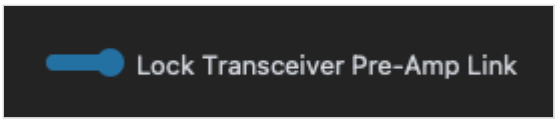
Return to Table of Contents

sCTkSwitch

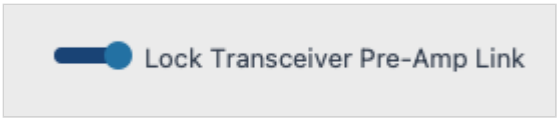
Table of Contents

Overview

`sCTkSwitch` is a themeable subclass of `customtkinter.CTkSwitch`. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state that dims every color property, not just the label text. A previously separate widget, `sCTkSwitchAlt`, existed specifically to work around limitations that have since been resolved directly in this widget and has been retired.

Dark Mode: 

Light Mode:



Disabling combines two mechanisms: CustomTkinter's native `state="disabled"`, and a bindtag-based click interceptor that prepends a dedicated binding returning `"break"` on click. This is more robust than a simple event-unbind, since it intercepts clicks regardless of which internal level the native click handler is actually bound at.

Constructor

```
sCTkSwitch(master=None, onvalue=1, offvalue=0, command=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`onvalue`	any	`1`	Value reported when the switch is on.
`offvalue`	any	`0`	Value reported when the switch is off.
`command`	`callable`	`None`	Called on toggle. May accept the new value as a single argument, or no arguments at all — both styles are supported.
`**kw`	—	—	`state` is pulled out explicitly. Everything else is either a native `CTkSwitch` argument or a theme-key override (see the "sCTkSwitch" block in Theming).

```
notify_switch = sCTkSwitch(
    master=control_panel,
    text="Enable notifications",
    command=on_notify_toggled,
)
notify_switch.pack(anchor="w", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`cget("state")` / `cget("command")`	varies	Both intercepted specially, since they're tracked on the instance rather than delegated to the native widget.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration. Note the signature is `configure(require_redraw=None, **kwargs)`, matching real CTk's own convention, rather than `*args` — calling `configure("state")` positionally returns a Tkinter-style query tuple; a positional dict is merged into the update.

Exceptions from your `command` propagate normally. An earlier version silently swallowed every exception a command raised, hiding real bugs completely; this is now fixed, confirmed by direct testing. Tkinter's own default callback-exception handling reports propagated exceptions to the console without crashing the running application.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` / `configure()` change** — all five color properties (`fg_color`, `progress_color`, `button_color`, `button_hover_color`, `text_color`) are recomputed from the theme's normal values or its `disabled_map`. This full dimming is confirmed working by direct testing.

```
{
    "sCTkSwitch": {
        "font": ["Arial", 14, "normal"],
        "fg_color": ["#1A4375", "#1F6AA5"],
        "progress_color": ["#1A4375", "#1F6AA5"],
        "button_color": ["#CBD5E1", "#CBD5E1"],
        "button_hover_color": ["#E5E7EB", "#94A3B8"],
        "text_color": ["#1F2937", "#F9FAFB"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#64748B"],
            "fg_color": ["#94A3B8", "#526071"],
            "progress_color": ["#64748B", "#526071"],
            "button_color": ["#CBD5E1", "#94A3B8"],
            "button_hover_color": ["#CBD5E1", "#94A3B8"]
        }
    }
}
```

`button_color` uses the same light-mode value for both normal and hover states by design — it was retuned from an earlier pure-white value, which had too little contrast against light backgrounds in general (not a code bug; CustomTkinter already resolves the widget's background to match its real parent correctly on its own). The disabled-state track colors (`fg_color` / `progress_color`) were similarly retuned for the same reason — the original values were close enough to typical light backgrounds that a disabled switch's track could become hard to see at all.

All five color properties are required to be present in both the top-level block and `disabled_map` — if any are missing, the widget raises immediately rather than substituting a hardcoded color.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they correctly follow system/app appearance-mode changes automatically — confirmed by direct testing, including while disabled.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkSwitch, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Switch Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    notify_switch = sCTkSwitch(base, text="Enable notifications", command=lambda v: print(f"Value: {v
    notify_switch.pack(anchor="w", pady=10)

    def toggle_disabled():
        target = "disabled" if notify_switch.get_state() == "normal" else "normal"
        notify_switch.state(target)
        disable_toggle.configure(text="Enable Switch" if target == "disabled" else "Disable Switch")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Switch", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

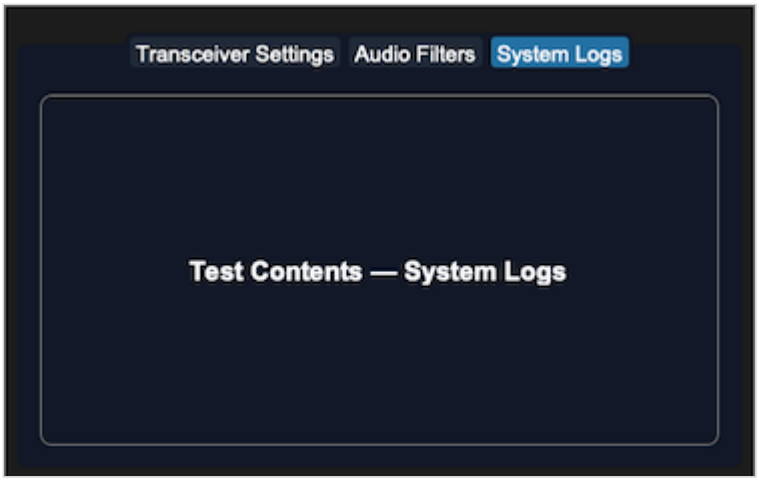
Known Limitations

- If your `command` accepts exactly one argument and raises a `TypeError` for a reason unrelated to argument count, the wrapper's fallback logic can't tell the difference from "this command doesn't accept an argument" — it will retry calling your command with no arguments, which then fails with a second, different `TypeError` (a missing-argument error) layered on top of your real bug. Python's exception chaining keeps both visible in the console, so the real bug isn't hidden, just noisier than ideal.
- Calling `configure("propname")` for a property name other than `"state"` is forwarded to the native widget's `configure()`, which does not support single-argument property queries — a known limitation shared with the wider Pygubu query investigation set aside elsewhere in this project.

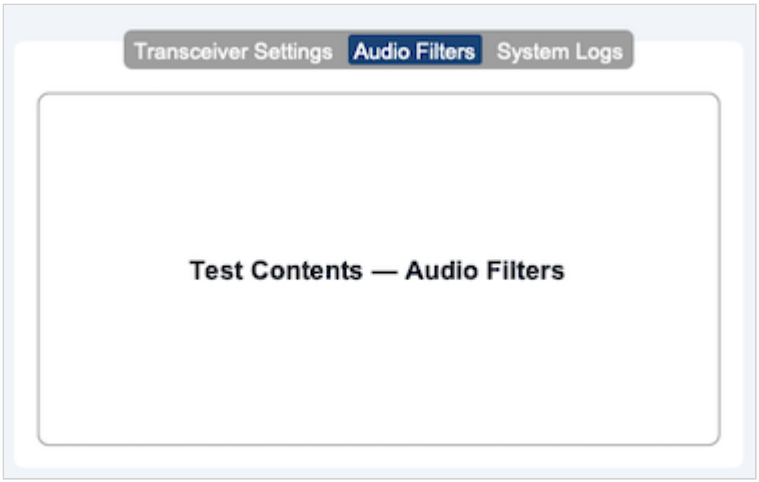
Return to Table of Contents

sCTkTabview

`sCTkTabview` is a themeable multi-page tab container — a subclass of `customtkinter.CTkTabview` with automatic light/dark theme resolution from `sCTkThemes.json`, a disabled state, and Pygubu Designer support.

Dark Mode: 

Light Mode:



Its one structural difference from the native widget: `add()` and `tab()` return an `sCTkFrame`, not a `ctk.CTkFrame`. See Tab Pages.

Table of Contents

Constructor

```
tabview = sCTkTabview(master=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`state`	`str`	`"normal"`	`"normal"` or `"disabled"`. Also settable via `configure(state=...)` or `state()`.
`**kw`	—	—	Any native `CTkTabview` argument, or an override for one of the theme keys listed under Theming.

Construction **raises** `KeyError` **immediately** if the theme block is incomplete — see Theming.

Tab Pages

Native `CTkTabview.add()` constructs a plain `ctk.CTkFrame` for each tab and grids it inline, with no hook to substitute a different class. Rather than reimplement `add()` against CustomTkinter's internals — or mutate the created frame's `__class__` at runtime, a fragile pattern deliberately retired elsewhere in this project — this widget embeds an `sCTkFrame` **inside** the native tab frame and hands that back instead.

The native frame stays exactly where `CTkTabview` put it and keeps doing its own show/hide/grid work untouched; it just becomes an invisible outer shell. The wrapper is transparent with no border of its own, so the tab looks identical — the difference is purely structural: everything placed in a tab now has an `sCTk` widget as its parent.

```
# add() returns the page directly -- no separate tab() call needed.
page = widget.add("Transceiver Settings")

inner_panel = sCTkFrame(page, border_width=1)
inner_panel.pack(expand=True, fill="both", padx=10, pady=10)
```

`tab(name)` returns the same object on every call, and creates the wrapper on first use — so a tab created by any other path (`insert()`, or `CTkTabview`'s own machinery) still comes back correctly wrapped. If you specifically need the native outer shell, `ctk.CTkTabview.tab(widget, name)` still reaches it.

Pygubu Designer Tab Insertion

Nesting children within the Pygubu Designer layout pane requires adherence to CustomTkinter's native tab allocation slots.

1. **Chassis placement:** Locate the custom widget container on your workbench tree panel and place an instance of `sCTkTabview` into your frame layout.
2. **Tab component selection:** In the Pygubu Designer widget selector tree, expand the CustomTkinter widget set and locate the native element named `CTkTabview.Tab`.
3. **Parent nesting assignment:** Drop the `CTkTabview.Tab` element directly onto the parent `sCTkTabview` widget slot in your inspector tree layout.
4. **Repeat allocation:** Repeat for each additional page slot. Tabs can then be named individually using the workspace property sidebars.

Note that tabs created this way are native `CTkTabview.Tab` slots. Calling `widget.tab(name)` on one still returns a wrapped `sCTkFrame`, since wrapping happens lazily on first access.

Methods

Method	Returns	Description
`add(name)`	`sCTkFrame`	Creates a tab and returns its content page. Return type differs from native `CTkTabview.add()`.
`tab(name)`	`sCTkFrame`	Returns a tab's content page, creating the wrapper on first use. Stable across calls.
`delete(name)`	—	Deletes a tab, tearing down its page wrapper first so no stale entry is left behind.
`state()` / `state(mode)`	`str`	Getter with no argument; setter with `"normal"` or `"disabled"`. Dims text, flattens the tab bar, and locks tab selection.
`get_state()`	`str`	Equivalent to `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	`None`	Standard configuration. Accepts `state` alongside any native option.
`configure("state")`	`tuple`	Pygubu-style single-argument query, returning `(name, name, name, default, current)`.
`cget(name)`	`Any`	Extended to know about `state`; everything else passes through to the native widget.

All four state paths — `state()`, `get_state()`, `cget("state")`, and `configure(state=...)` — operate on the same underlying value and agree with each other.

On `bind()`: native `CTkTabview.bind()` raises `NotImplementedError`. This widget overrides it to route through `tkinter.Frame.bind` instead, so Pygubu Designer click handling doesn't crash the workspace.

Theming (`sCTkThemes.json`)

```
{
    "sCTkTabview": {
        "font": ["Arial", 15, "normal"],
        "segmented_button_height": 36,
        "fg_color": ["#FFFFFF", "#111827"],
        "text_color": ["#FFFFFF", "#FFFFFF"],
        "segmented_button_fg_color": ["#9E9E9E", "#111827"],
        "segmented_button_selected_color": ["#1A4375", "#2471A3"],
        "segmented_button_selected_hover_color": ["#112A4B", "#1F618D"],
        "segmented_button_unselected_color": ["#9E9E9E", "#1F2937"],
```

```
        "segmented_button_unselected_hover_color": ["#7D7D7D", "#374151"],
        "disabled_map": {
            "segmented_button_fg_color": ["#FFFFFF", "#111827"],
            "segmented_button_selected_color": ["#CBD5E1", "#374151"],
            "segmented_button_unselected_color": ["#CBD5E1", "#374151"],
            "text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

Every key above is required. Construction raises `KeyError` naming exactly what's missing, rather than substituting a guessed color. This is the fail-loud principle used across the project — an earlier version fell back to hardcoded literals for all ten colors and the font, and because those guesses looked plausible, a broken or partial theme block was invisible.

The split between the two blocks:

Keys	Required in
`text_color` , `segmented_button_fg_color` , `segmented_button_selected_color` , `segmented_button_unselected_color`	top level **and** `disabled_map`
`segmented_button_selected_hover_color` , `segmented_button_unselected_hover_color` , `font` , `segmented_button_height`	top level only

The two hover colors deliberately have no `disabled_map` entry. A disabled tab bar must not light up under the cursor, so when disabled, hover collapses to the corresponding non-hover disabled color. There is no meaningful "dimmed hover" distinct from "dimmed", so requiring a separate key would only invite them to drift apart. `font` and `segmented_button_height` are top level only because neither changes with state.

`font` and `segmented_button_height` are both intercepted before native construction and forwarded to the internal segmented button. This is not optional: `CTkTabview` names every parameter explicitly with no `**kwargs` catch-all, so any key it doesn't recognize raises `ValueError` from its constructor. They're applied once rather than on every repaint, since neither varies by state. See Known Limitations regarding what `segmented_button_height` actually achieves.

Validation is scoped to direct construction. A subclass reaches this constructor with `final_kw` built from *its own* theme block — `ThemeableWidget` 's run-once guard means it is never rebuilt — so validating these keys against a subclass's block would raise on every construction. Subclasses own their own theme contract.

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTkLabelPrimary, sCTk, sCTkTabview

if __name__ == "__main__":
    root = sCTk()
```

```
root.geometry("640x480")
root.title("sCTkTabview Container Validation Bench")
root.configure(fg_color=("#F1F5F9", "#1C1C1C"))

base = sCTkFrame(root, border_width=2)
base.pack(expand=True, fill="both", padx=20, pady=20)

widget = sCTkTabview(base)
widget.pack(expand=True, fill="both", padx=10, pady=10)

for page_name in ["Transceiver Settings", "Audio Filters", "System Logs"]:
    # add() returns the sCTkFrame content page directly.
    page_viewport = widget.add(page_name)

    inner_frame = sCTkFrame(page_viewport, border_width=1, corner_radius=8)
    inner_frame.pack(expand=True, fill="both", padx=10, pady=10)

    test_label = sCTkLabelPrimary(inner_frame, text=f"Test Contents — {page_name}")
    test_label.pack(expand=True, fill="none", padx=20, pady=20)

def toggle_tab_lock():
    target = "disabled" if widget.state() == "normal" else "normal"
    widget.configure(state=target)
    btn_lock.configure(
        text="Unlock Tabview Navigation" if target == "disabled" else "Lock Tabview (Set 'disable
    print(f"state()={widget.state()}  cget={widget.cget('state')}")

def toggle_temp_page():
    if "Scratch Pad" in widget._sctk_pages:
        widget.delete("Scratch Pad")
        btn_temp.configure(text="Add Runtime Page")
    else:
        page = widget.add("Scratch Pad")
        sCTkLabelPrimary(page, text="Created at runtime").pack(expand=True, padx=20, pady=20)
        btn_temp.configure(text="Delete Runtime Page")

def toggle_skin_preference():
    ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

control_tray = sCTkFrame(root, fg_color="transparent")
control_tray.pack(side="bottom", fill="x", padx=20, pady=(0, 15))

btn_lock = sCTkButtonPrimary(control_tray, text="Lock Tabview (Set 'disabled')", command=toggle_ta
btn_lock.pack(side="left", expand=True, padx=4)

btn_temp = sCTkButtonPrimary(control_tray, text="Add Runtime Page", command=toggle_temp_page)
btn_temp.pack(side="left", expand=True, padx=4)

btn_skin = sCTkButtonPrimary(control_tray, text="Toggle UI Light/Dark Appearance", command=toggle_
btn_skin.pack(side="right", expand=True, padx=4)

root.mainloop()
```

Known Limitations

- `segmented_button_height` **is currently a no-op, retained for possible future use.** The value is applied to the internal segmented button and `cget("height")` reports it back accurately, but the visible tab strip does **not** grow

to match. `CTkTabview` grids the segmented button into a row whose `minsize` comes from its own private spacing constants, and deliberately overlaps the button with the page frame below to produce the connected-tab look. A taller button is clipped by that row rather than expanding it. Confirmed by direct testing: a height of 128 reported back correctly and produced no visible change.

The key is deliberately kept, and kept **required**, rather than removed. It costs nothing, it keeps the theme contract stable, and it's already wired end-to-end — so if a future CustomTkinter release exposes the strip height, or the internals approach below is revisited, only the application step changes. Do not treat it as broken and delete it; changing the number is expected to do nothing today.

Making the strip actually taller would mean writing `CTkTabview` 's private `_top_spacing` / `_top_button_overhang` attributes and re-running its `_configure_grid()` — a dependency on CustomTkinter internals that could break on any upstream release. Deliberately not done.

Note this is a `CTkTabview` layout constraint, **not** a limitation of the segmented button: a standalone `sCTkSegmentedButton` honors `height` normally.

- **Disabling does not cascade to children.** It dims the tab bar and locks tab selection, but widgets placed inside a page are unaffected — disabling them is the caller's responsibility.
- `add()` **and** `tab()` **return a different type than the native widget.** Code doing an `isinstance` check against `ctk.CTkFrame` , or reaching for CTkFrame-specific internals on a tab page, would notice. `ctk.CTkTabview.tab(widget, name)` still reaches the native shell.
- **The internal segmented button is a native** `CTkSegmentedButton` , not `sCTkSegmentedButton` . It is created inside `CTkTabview.__init__` and re-themed afterwards by pushing colors onto it. Replacing it with the themed variant would let it theme itself and remove most of that code, but the swap hasn't been made.
- **Each tab page carries one extra frame layer** — the native shell plus the `sCTkFrame` wrapper inside it. Transparent and borderless, so invisible, but present in the widget tree.
- `text_color` **is applied by reaching into the segmented button's private** `_buttons_dict` , since CustomTkinter exposes no public way to set per-button text color on a segmented button. This depends on a CustomTkinter internal and could break on a future release.
- **Colors are resolved to a single value** via `_resolve_color()` rather than passed through as raw `(light, dark)` tuples, so appearance-mode changes rely on this widget's own `_set_appearance_mode()` hook re-running the theme pass rather than on CustomTkinter's native tracking.

Return to Table of Contents

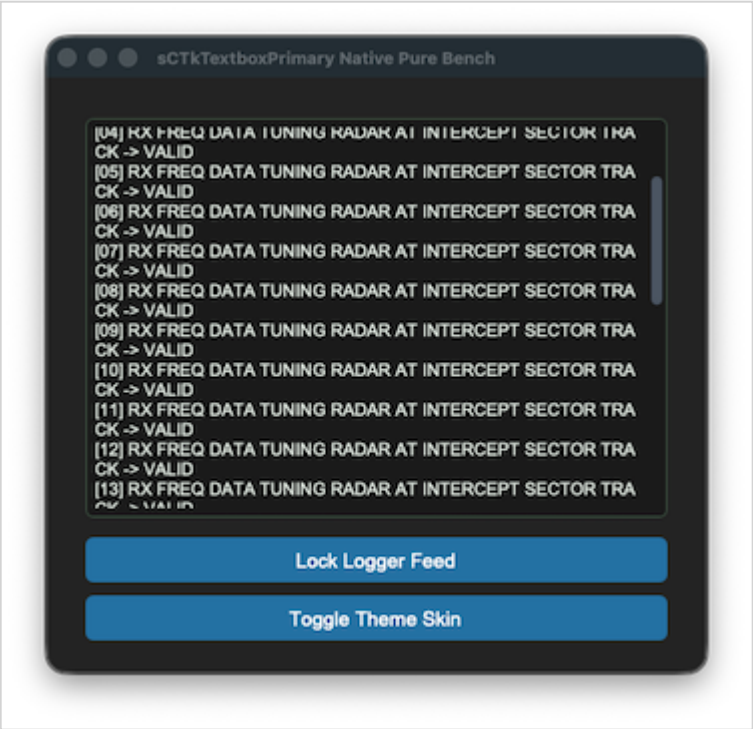
sCTkTextboxPrimary

Table of Contents

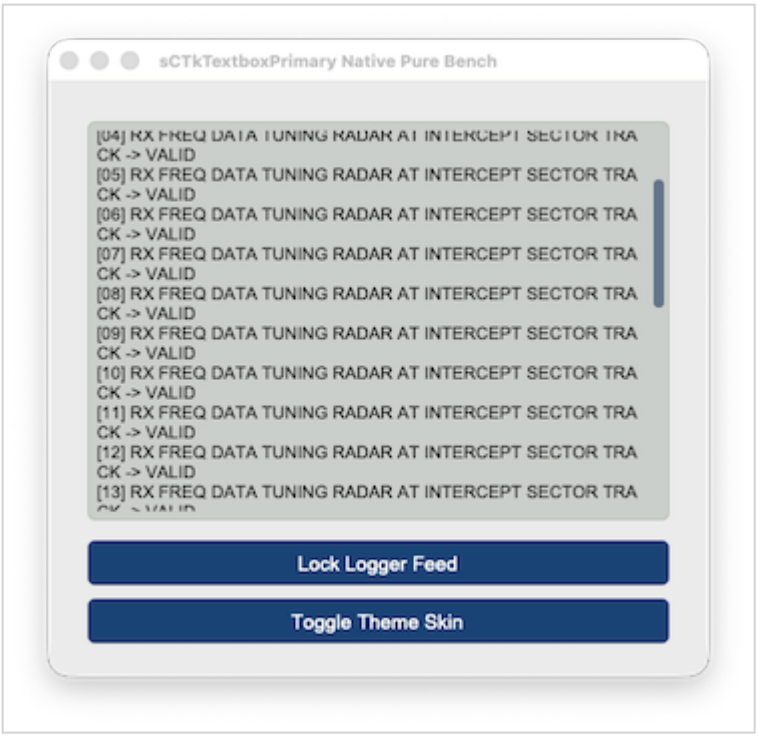
Overview

`sCTkTextboxPrimary` is a themeable subclass of `customtkinter.CTkTextbox` — a multi-line text area, the higher-emphasis of the library's two textbox tiers (see also `sCTkTextboxSecondary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state, using CustomTkinter's native `state="disabled"`.

Dark Mode:



Light Mode:



If `fg_color` resolves to `"transparent"` at construction, the widget copies its parent's actual `fg_color` instead, since `CTkTextbox` doesn't render true transparency the way canvas-based widgets can.

Constructor

```
sCTkTextboxPrimary(master=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kw`	—	—	`state` is pulled out and applied after construction. Everything else is any native `CTkTextbox` argument, or an override for one of the theme keys listed under Theming.

```
log_area = sCTkTextboxPrimary(master=control_panel)
log_area.pack(fill="both", expand=True, padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(state_string=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it. Uses CTk's native `state="disabled"`, consistent with the other widgets in this library confirmed to correctly block interaction this way.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` updates the tracked state and triggers a repaint. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `text_color`, `border_color`, `scrollbar_button_color`, and `scrollbar_button_hover_color`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, `border_width`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `text_color`, `scrollbar_button_color`, and `scrollbar_button_hover_color` are recomputed from the theme's normal values or its `disabled_map`. This includes manually re-theming the widget's internal scrollbar, which isn't automatically covered by a single `configure()` call.

```
{
    "sCTkTextboxPrimary": {
        "font": ["Arial", 13, "normal"],
        "border_width": 1,
        "corner_radius": 6,
        "border_color": ["#b5beb6", "#3d5242"],
        "fg_color": ["#cbcfcb", "#1a1a1a"],
        "text_color": ["#1c1d1c", "#e3ece4"],
        "scrollbar_button_color": ["#64748B", "#4B5563"],
        "scrollbar_button_hover_color": ["#1A4375", "#2471A3"],
        "disabled_map": {
            "fg_color": ["#E5E7EB", "#111827"],
            "border_color": ["#CBD5E1", "#1F2937"],
            "text_color": ["#94A3B8", "#64748B"],
            "scrollbar_button_color": ["#E5E7EB", "#1F2937"],
            "scrollbar_button_hover_color": ["#E5E7EB", "#1F2937"]
        }
    }
}
```

`scrollbar_button_color` and `scrollbar_button_hover_color` are required to be present in whichever map is active — if either is missing, the widget raises immediately rather than substituting a hardcoded color.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkTextboxPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x350")
    root.title("TextboxPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    log_area = sCTkTextboxPrimary(base)
    log_area.pack(fill="both", expand=True, pady=10)
    log_area.insert("1.0", "Log output appears here...")

    def toggle_disabled():
        target = "disabled" if log_area.get_state() == "normal" else "normal"
        log_area.state(target)
        disable_toggle.configure(text="Enable Log" if target == "disabled" else "Disable Log")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Log", command=toggle_disabled)
    disable_toggle.pack(pady=10)
```

```
root.mainloop()
```

Known Limitations

- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for six specific properties, and falls through to the native widget's `configure()` for anything else.

Return to Table of Contents

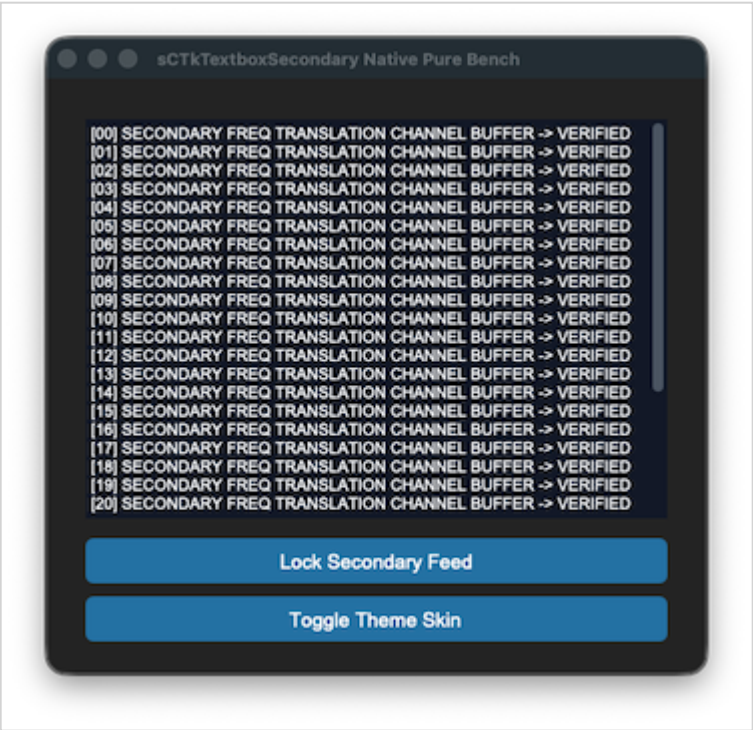
sCTkTextboxSecondary

Table of Contents

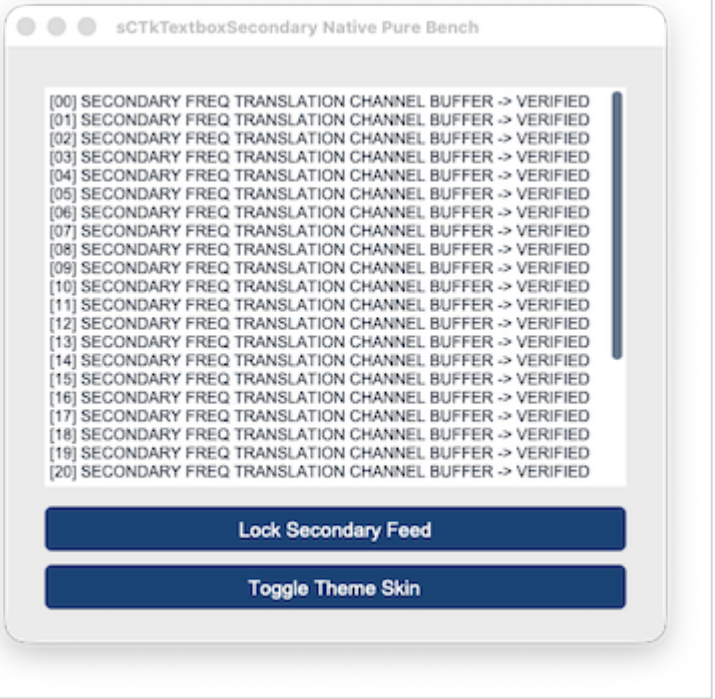
Overview

`sCTkTextboxSecondary` is a themeable subclass of `customtkinter.CTkTextbox` — a multi-line text area, the lower-emphasis of the library's two textbox tiers (see also `sCTkTextboxPrimary`). It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state, using CustomTkinter's native `state="disabled"`.

Dark Mode:



Light Mode:



If `fg_color` resolves to `"transparent"` at construction, the widget copies its parent's actual `fg_color` instead — same rationale as `sCTkTextboxPrimary` 's identical fallback.

Constructor

```
sCTkTextboxSecondary(master=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kw`	—	—	`state` is pulled out and applied after construction. Everything else is any native `CTkTextbox` argument, or an override for one of the theme keys listed under Theming.

```
notes_area = sCTkTextboxSecondary(master=control_panel)
notes_area.pack(fill="both", expand=True, padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(state_string=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it; `"normal"`, `"enabled"`, or `"active"` all enable it.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` updates the tracked state and triggers a repaint. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `text_color`, `border_color`, `scrollbar_button_color`, and `scrollbar_button_hover_color`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, is merged with any matching keyword arguments and applied when the widget is built. Note `border_width` and `corner_radius` are both `0` for this style — no visible border, unlike `sCTkTextboxPrimary`.
- **Re-applied on every `state()` change** — `fg_color`, `text_color`, `scrollbar_button_color`, and `scrollbar_button_hover_color` are recomputed from the theme's normal values or its `disabled_map`, including manually re-theming the internal scrollbar.

```
{
    "sCTkTextboxSecondary": {
        "font": ["Arial", 12, "normal"],
        "border_width": 0,
        "corner_radius": 0,
        "fg_color": ["#FFFFFF", "#111827"],
        "text_color": ["#1F2937", "#F9FAFB"],
        "scrollbar_button_color": ["#64748B", "#4B5563"],
```

```
        "scrollbar_button_hover_color": ["#1A4375", "#2471A3"],
        "disabled_map": {
            "fg_color": ["#E5E7EB", "#111827"],
            "text_color": ["#94A3B8", "#64748B"],
            "scrollbar_button_color": ["#E5E7EB", "#1F2937"],
            "scrollbar_button_hover_color": ["#E5E7EB", "#1F2937"]
        }
    }
}
```

`scrollbar_button_color` and `scrollbar_button_hover_color` are required to be present in whichever map is active — if either is missing, the widget raises immediately rather than substituting a hardcoded color.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkTextboxSecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x350")
    root.title("TextboxSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    notes_area = sCTkTextboxSecondary(base)
    notes_area.pack(fill="both", expand=True, pady=10)
    notes_area.insert("1.0", "Notes go here...")

    def toggle_disabled():
        target = "disabled" if notes_area.get_state() == "normal" else "normal"
        notes_area.state(target)
        disable_toggle.configure(text="Enable Notes" if target == "disabled" else "Disable Notes")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Notes", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.

- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for six specific properties, and falls through to the native widget's `configure()` for anything else.

Return to Table of Contents

Menus

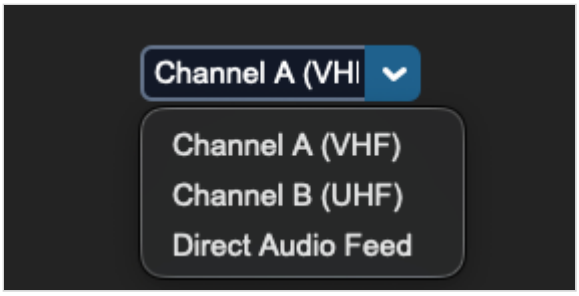
Not a lot of choices here, but they should suffice.

sCTkComboBox

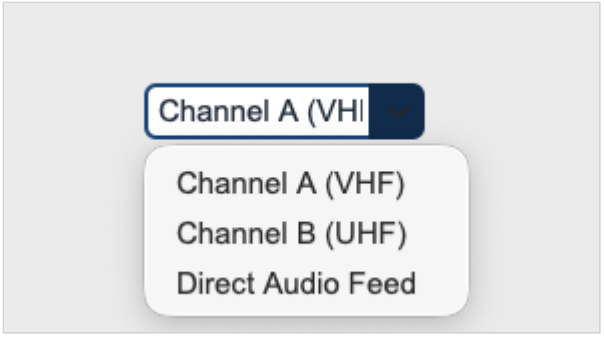
Table of Contents

Overview

`sCTkComboBox` is a themeable subclass of `customtkinter.CTkComboBox`. It adds automatic light/dark theme resolution from `sCTkThemes.json`, a distinct enabled/disabled visual state (separate from, but synchronized with, the widget's native interactive lock), and Pygubu Designer property introspection support.

Dark Mode: 

Light Mode:



Constructor

```
sCTkComboBox(master=None, values=None, command=None, variable=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`values`	`list[str]`	`[""]`	Dropdown options. If the first entry is a non-empty string, it is selected automatically on creation.
`command`	`callable`	`None`	Called with the selected value when the user picks an item.
`variable`	`tkinter.StringVar`	`None`	Optional variable bound to the current selection.
`**kwargs`	—	—	Any native `CTkComboBox` argument (e.g. `width`, `height`, `font`, `corner_radius`), or an override for one of the theme keys listed under Theming. Anything not supplied falls back to the `sCTkComboBox` block of `sCTkThemes.json`.

```
frequency_dropdown = sCTkComboBox(
    master=control_panel,
    values=["Channel A (VHF)", "Channel B (UHF)", "Direct Audio Feed"],
    command=on_frequency_channel_changed,
)
frequency_dropdown.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`get()`	`str`	Currently selected text (native `CTkComboBox` behavior).
`set(value)`	`None`	Sets the displayed text (native `CTkComboBox` behavior).
`state(mode=None)`	`str | None`	Gets or sets the widget's enabled/disabled visual state. Accepts `"normal"`, `"enabled"`, or `"active"` (all equivalent) and `"disabled"`. Any other value is silently ignored. Called with no argument, returns the current state as a lowercase string.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, with two additions: passing `state=...` routes to `state()` rather than the native Tkinter `state` option; calling `configure("propname")` with a single

Method	Returns	Description
		property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, `border_color`, `text_color`, and `hover_color`. Queries for any other single property name fall through to the native `CTkComboBox.configure`.

Theming (`sCTkThemes.json`)

Theme values are used in two different ways, worth keeping separate:

- **Applied once, at construction** — every key in the widget's theme block (including `corner_radius`) is merged with any matching keyword arguments you pass in and applied when the widget is built.
- **Re-applied on every `state()` change** — only `fg_color`, `border_color`, `text_color`, `button_color`, `button_hover_color`, `dropdown_fg_color`, `dropdown_text_color`, `dropdown_hover_color`, `border_width`, and `font` are swapped between the theme's normal values and its `disabled_map` values when you call `state("disabled")` / `state("normal")`. Toggling state also sets the native Tkinter `state` flag, so a disabled combo box is both visually muted and non-interactive.

```
{
    "sCTkComboBox": {
        "fg_color": ["#FFFFFF", "#1E1E1E"],
        "border_color": ["#94A3B8", "#4B5563"],
        "text_color": ["#111827", "#F9FAFB"],
        "button_color": ["#1A4375", "#1F6AA5"],
        "button_hover_color": ["#112A4B", "#194A7A"],
        "dropdown_fg_color": ["#FFFFFF", "#1F2937"],
        "dropdown_text_color": ["#374151", "#F3F4F6"],
        "dropdown_hover_color": ["#F3F4F6", "#374151"],
        "border_width": 2,
        "corner_radius": 6,
        "disabled_map": {
            "fg_color": ["#F3F4F6", "#111111"],
            "border_color": ["#CBD5E1", "#333333"],
            "text_color": ["#94A3B8", "#4B5563"],
            "button_color": ["#E5E7EB", "#222222"]
        }
    }
}
```

> **Note:** `disabled_map` above has no entries for `dropdown_fg_color`, `dropdown_text_color`, `dropdown_hover_color`, or `button_hover_color`. Since only keys present in `disabled_map` are swapped, those four properties keep their *enabled*-state color when the widget is disabled. Add entries for them here if you want the dropdown portion to visually mute along with the rest of the widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkButtonPrimary, sCTkComboBox

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x300")
    root.title("ComboBox Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    widget = sCTkComboBox(
        base,
        values=["Channel A (VHF)", "Channel B (UHF)", "Direct Audio Feed"],
        command=lambda choice: print(f"Selected: {choice}"),
    )
    widget.pack(expand=True, fill="none", padx=10, pady=10)

    def toggle_widget_state():
        target = "disabled" if widget.get_state() == "normal" else "normal"
        widget.configure(state=target)
        btn_toggle.configure(text="Enable" if target == "disabled" else "Disable")

    btn_toggle = sCTkButtonPrimary(base, text="Disable", command=toggle_widget_state)
    btn_toggle.pack(side="bottom", pady=15)

    root.mainloop()
```

Known Limitations

- `state()` silently ignores any value other than `normal`, `enabled`, `active`, or `disabled` — no exception is raised and no warning is logged.
- Passing a positional `dict` to `configure()` (e.g. `configure({"fg_color": "red"})`) is not merged into the update; only keyword arguments are applied. Use `configure(**your_dict)` instead.

Return to Table of Contents

sCTkOptionMenuPrimary

Table of Contents

Overview

`sCTkOptionMenuPrimary` is a themeable subclass of `customtkinter.CTkOptionMenu` — a dropdown option-selection button. It adds automatic light/dark theme resolution from `sCTkThemes.json` and a distinct enabled/disabled visual state. See also `sCTkOptionMenuSecondary`, a composite bordered variant with a different internal architecture.

Dark Mode: 

Light Mode: 

Constructor

```
sCTkOptionMenuPrimary(master=None, values=None, command=None, variable=None, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`values`	`list[str]`	native default	The dropdown options.
`command`	`callable`	`None`	Called with the selected value when the user picks an item.
`variable`	`tkinter.StringVar`	`None`	Optional variable bound to the current selection.
`**kw`	—	—	Any native `CTkOptionMenu` argument, or an override for one of the theme keys listed under Theming.

```
mode_menu = sCTkOptionMenuPrimary(
    master=control_panel,
    values=["AM", "FM", "SSB"],
    command=on_mode_changed,
)
mode_menu.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it; `"normal"`,

Method	Returns	Description
		`"enabled"`, or `"active"` all enable it. Uses CTk's native `state="disabled"`, consistent with the other widgets in this library confirmed to correctly block interaction this way, though not independently re-tested for this specific widget.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: `values` / `command` / `variable` are routed individually; `state=...` routes through `state()`; calling `configure("propname")` with a single property name returns a Tkinter-style `(name, name, name, default, current)` tuple for `state`, `fg_color`, `button_color`, `button_hover_color`, and `text_color`. Queries for any other property name fall through to the native `CTkOptionMenu.configure`.
`update_list(new_values, default_index=0)`	`None`	Replaces the dropdown's options and resets the visible selection. If `new_values` is empty, the widget is set to a single blank option. If `default_index` is out of range, falls back to index `0` rather than raising.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `font`, `dropdown_font`, and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `button_color`, `button_hover_color`, `text_color`, `dropdown_fg_color`, `dropdown_text_color`, and `font` are recomputed from the theme's normal values or its `disabled_map` every time you call `state()`.

```
{
    "sCTkOptionMenuPrimary": {
        "font": ["Arial", 15, "normal"],
        "dropdown_font": ["Arial", 15, "normal"],
        "fg_color": ["#1A4375", "#2471A3"],
        "button_color": ["#112A4B", "#1F618D"],
        "button_hover_color": ["#0D1F38", "#1A5276"],
        "text_color": ["#FFFFFF", "#FFFFFF"],
        "corner_radius": 6,
        "dropdown_fg_color": ["#FFFFFF", "#1F2937"],
        "dropdown_text_color": ["#1F2937", "#F9FAFB"],
        "dropdown_hover_color": ["#E5E7EB", "#374151"],
        "disabled_map": {
            "fg_color": ["#CBD5E1", "#374151"],
            "button_color": ["#CBD5E1", "#374151"],
            "text_color": ["#94A3B8", "#64748B"]
        }
    }
```

```
}
```

`disabled_map` doesn't cover `button_hover_color`, `dropdown_fg_color`, `dropdown_text_color`, or `dropdown_hover_color` — consistent with every other themed widget in this library: once natively disabled, hover and dropdown-open interactions can't fire in the first place, so there's nothing for a disabled-state color on those properties to ever visibly apply to.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkOptionMenuPrimary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("OptionMenuPrimary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    mode_menu = sCTkOptionMenuPrimary(
        base, values=["AM", "FM", "SSB"], command=lambda choice: print(f"Selected: {choice}")
    )
    mode_menu.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if mode_menu.get_state() == "normal" else "normal"
        mode_menu.state(target)
        disable_toggle.configure(text="Enable Menu" if target == "disabled" else "Disable Menu")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Menu", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value matches neither branch, though colors are still harmlessly re-applied.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for

anything else.

Return to Table of Contents

sCTkOptionMenuSecondary

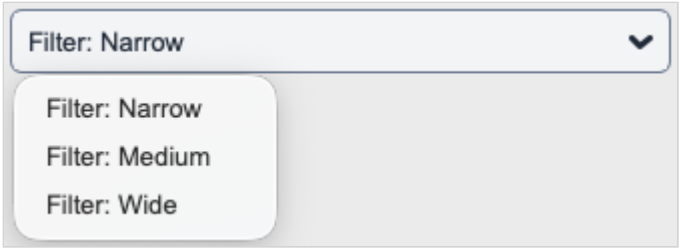
Table of Contents

Overview

`sCTkOptionMenuSecondary` is a themeable, composite bordered dropdown option-selection menu. Unlike every other widget in this library, it is **not** a direct subclass of the widget it wraps — it's a `customtkinter.CTkFrame` containing a plain, native `customtkinter.CTkOptionMenu` inside it, giving the dropdown a themed border the native widget has no way to draw on its own. See also `sCTkOptionMenuPrimary`, a simpler direct-subclass variant.

Dark Mode: 

Light Mode:



Because configuring the outer widget affects the frame (border, background, size) while the dropdown itself is a separate inner object, most of this widget's behavior comes from keeping those two pieces in sync — see Theming for how that split works.

Constructor

```
sCTkOptionMenuSecondary(master=None, width=160, height=28, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`width`	`int`	`160`	Frame width, used unless overridden by a kwarg or the theme.
`height`	`int`	`28`	Frame height, used unless overridden by a kwarg or the theme.
`**kw`	—	—	`values` (list[str]), `command` (callable), and `variable` (tkinter.StringVar) are forwarded to the inner dropdown. Theme keys that belong to the inner menu rather than the outer frame — `font`, `dropdown_font`, `text_color`, `dropdown_fg_color`, `dropdown_text_color`, `dropdown_hover_color`, `button_hover_color` — are automatically routed there; everything else applies to the outer frame. See Theming.

```
band_menu = sCTkOptionMenuSecondary(
    master=control_panel,
    values=["80m", "40m", "20m", "10m"],
    command=on_band_changed,
)
band_menu.pack(fill="x", padx=40, pady=10)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's enabled/disabled state. Only `"disabled"` (case-insensitive) disables it. Passes `state="disabled"` to the **inner dropdown**, not the outer frame (which has nothing interactive to lock), consistent with the other widgets in this library confirmed to correctly block interaction this way.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`get()`	`str`	Delegates to the inner dropdown's `get()`.
`set(value)`	`None`	Delegates to the inner dropdown's `set()`.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard configuration for the **outer frame**, plus: `values` / `command` / `variable` are routed to the **inner dropdown**, not the frame; `state=...` routes through `state()`; calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `border_color`, `text_color`, `width`, and `height`. Queries for any other property name fall through to the native `CTkFrame.configure`.

Method	Returns	Description
`update_list(new_values, default_index=0)`	`None`	Replaces the inner dropdown's options and resets the visible selection. Empty list falls back to a blank option; out-of-range `default_index` falls back to `0`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is split between the outer frame and the inner dropdown (see the constructor table above for which keys go where), then applied when each is built.
- **Re-applied on every `state()` change** — the outer frame's `border_color`, `fg_color`, `border_width`, and `corner_radius` are recomputed from the theme's normal values or `disabled_map`; the inner dropdown's `fg_color`, `button_color`, and `text_color` are recomputed the same way. `font`, `dropdown_font`, `dropdown_fg_color`, `dropdown_text_color`, `dropdown_hover_color`, and `button_hover_color` are **not** re-applied on state changes — they're static properties of the inner dropdown, set once and left alone.

```
{
    "sCTkOptionMenuSecondary": {
        "border_width": 1.25,
        "corner_radius": 6,
        "border_color": ["#64748B", "#94A3B8"],
        "fg_color": ["#F3F4F6", "#0B0F19"],
        "font": ["Arial", 13, "normal"],
        "dropdown_font": ["Arial", 13, "normal"],
        "text_color": ["#1F2937", "#F9FAFB"],
        "button_hover_color": ["#94A3B8", "#374151"],
        "dropdown_fg_color": ["#FFFFFF", "#1F2937"],
        "dropdown_text_color": ["#1F2937", "#F9FAFB"],
        "dropdown_hover_color": ["#E5E7EB", "#374151"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#64748B"],
            "border_color": ["#CBD5E1", "#374151"],
            "fg_color": ["#E5E7EB", "#0B0F19"]
        }
    }
}
```

`fg_color` and `text_color` are required to be present in whichever map is active — if either is missing, the widget raises immediately rather than substituting a hardcoded color, per this project's design of failing hard on incomplete theme data (see `sCTkLabelPrimary / Secondary / Tertiary` for the precedent). An earlier version of this widget used hardcoded hex fallbacks for both, and separately had a real bug where the theme's actual `button_hover_color` was computed correctly and then immediately overwritten with `fg_color` — both are fixed as of this project's audit.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Example

```
import customtkinter as ctk
from scustomtkinter import sCTk, sCTkFrame, sCTkOptionMenuSecondary, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("OptionMenuSecondary Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    band_menu = sCTkOptionMenuSecondary(
        base, values=["80m", "40m", "20m", "10m"], command=lambda choice: print(f"Selected: {choice}"
    )
    band_menu.pack(pady=10)

    def toggle_disabled():
        target = "disabled" if band_menu.get_state() == "normal" else "normal"
        band_menu.state(target)
        disable_toggle.configure(text="Enable Menu" if target == "disabled" else "Disable Menu")

    disable_toggle = sCTkButtonPrimary(base, text="Disable Menu", command=toggle_disabled)
    disable_toggle.pack(pady=10)

    root.mainloop()
```

Known Limitations

- `state()` only recognizes `"disabled"` and `"normal"` / `"enabled"` / `"active"`; any other value matches neither branch, though colors are still harmlessly re-applied.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- Passing a positional dict to `configure()` merges into the update; a positional property-name string returns the query tuple described above for five specific properties, and falls through to the native widget's `configure()` for anything else.
- Because this widget wraps rather than subclasses its inner control, `configure()` on the outer widget and `configure()` on `self._menu` (the inner dropdown) are genuinely different calls affecting different objects — code that expects a single unified `configure()` surface (as every other widget in this library provides) needs to be aware of this split.

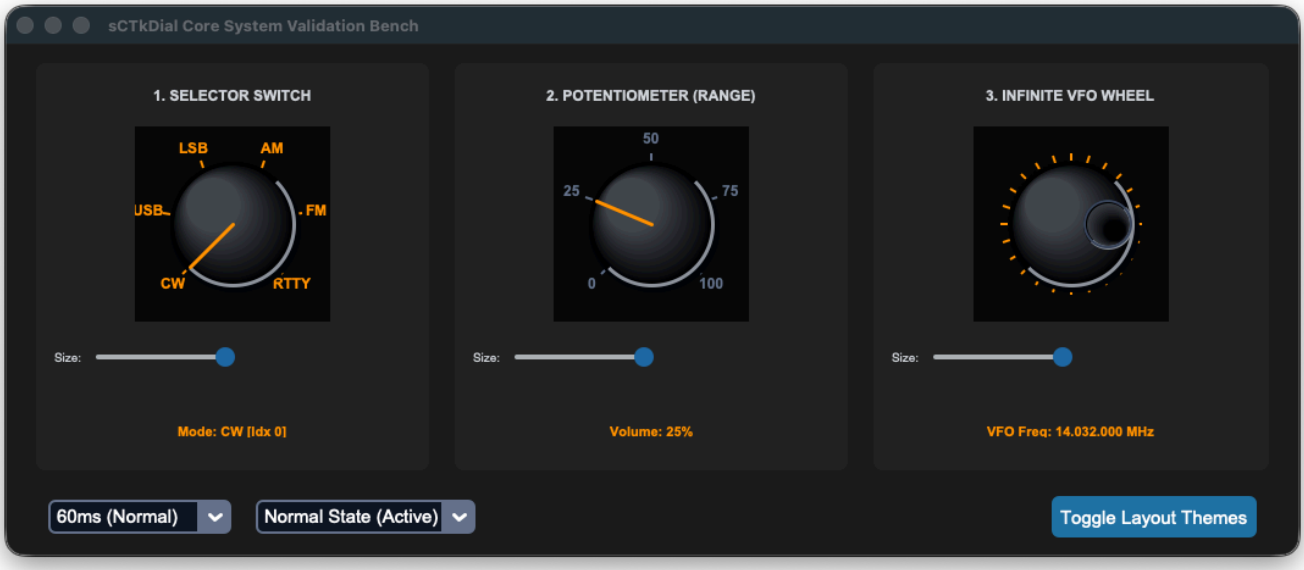
Return to Table of Contents

Additional Widgets Provided by sCustomTkinter

These are all the extra widgets that were added to the stock set provided with `CustomTkinter`.

sCTKDialBase

Abstract base class for the rotary dial family. It owns the canvas rendering, the mouse and scroll interaction model, the theme contract, and the state machine shared by `sCTkDialContinuous`, `sCTkDialSelector`, and `sCTkDialRange`.



This page is the reference for how dials are drawn and themed. The three variant pages describe only what differs.

Note the spelling: the class is `sCTKDialBase` with a capital K. It is never instantiated directly and has no theme block of its own — each concrete subclass resolves its own block by class name.

Table of Contents

Knob rendering

The knob is drawn as a stack of concentric ovals stepping from `dial_shadow_color` at the rim to `dial_highlight_color` off-centre, each ring shifted toward a light source in the upper left. Tk's canvas has no gradient primitive and **no alpha channel**, so this is the only way to get a domed surface — every colour is a solid fill computed by interpolation, never a translucent overlay.

Two arcs finish it: `dial_rim_light_color` across the upper-left edge and `dial_rim_shadow_color` across the lower-right.

The rim light does most of the work on dark knobs. A black knob's shading range is clamped at the bottom — you cannot go darker than black at the edge — so it has roughly half the dynamic range of a light one. Remove the bright rim arc and a dark knob collapses back to a flat disc. This is why the highlight and shadow colours are explicit theme keys rather than derived from `dial_color` by a fixed lighten/darken: a percentage that produces a visible rim on black blows out on aluminium, and vice versa.

Tuning constants live on the base class, so they can be overridden per subclass or per instance:

Constant	Default	Meaning
`KNOB_SHADE_STEPS`	`18`	Ring count. Below ~12 the steps read as contour bands; above ~24 costs more than it shows.
`KNOB_SHADE_SHRINK`	`0.55`	How far the stack shrinks from rim to centre, as a fraction of radius.
`KNOB_LIGHT_OFFSET`	`0.55`	How far each ring drifts toward the light.
`DIMPLE_RADIUS_FRAC`	`0.36`	Finger dimple radius, as a fraction of knob radius.
`DIMPLE_RIM_CLEARANCE_FRAC`	`0.06`	Gap between dimple edge and rim, same units.
`POINTER_WIDTH`	`3.0`	Pointer line width in pixels.
`POINTER_RIM_INSET`	`3`	How far short of the rim the pointer stops.

The dimple and clearance are **fractions, not pixels**. An earlier version used a fixed 14px inset with a fixed 14.5px radius, so the dimple was lost on a large dial and swallowed a small one.

Recesses shade opposite to domes. The knob body is a dome, lit on the upper left. The dimple is a hole, so its shading inverts — shadowed on the upper-left interior wall, lit on the lower right. Drawn with the body's light direction it reads as a raised bump instead of something you can put a finger in.

Theme contract

Every concrete dial requires these at the **top level** of its theme block:

`fg_color`, `text_color`, `shadow_color`, `dial_color`, `dial_highlight_color`, `dial_shadow_color`, `dial_rim_light_color`, `dial_rim_shadow_color`

and these inside `disabled_map`:

`text_color`, `dial_color`

Plus one variant-specific key each — see the individual pages.

Construction raises `KeyError` naming the missing key and where it belongs. This replaced a pattern of `.get(key) or ("#hex", "#hex")` throughout the draw routine, which silently substituted a plausible guess and made an incomplete theme block look merely slightly-off rather than broken.

`fg_color` is deliberately **not** required in `disabled_map`: the background does not dim when disabled, the knob face and text carry the signal. This also fixes a latent bug — the old code read `fg_color` from `disabled_map` for *both* the dial face and the background, so once that key existed the knob would have rendered the same colour as the surface behind it and vanished. The two only looked different because the map was empty and their hardcoded fallbacks happened to differ.

The flat `disabled_text_color` / `disabled_dial_color` / `disabled_dimple_glow` keys used by earlier theme files are **retired**. They now live in `disabled_map` under their normal names, matching every other widget in the library.

Reading theme colours

> **Custom drawing colours must be read from the raw theme registry, not from `final_kw`.** This is a trap that produces plausible-looking wrong colours rather than an error, and it went unnoticed in this widget family for its entire existence.

`ThemeableWidget` maintains a `CUSTOM_VECTOR_KEYS` set — `dial_color`, `shadow_color`, `text_color`, `pointer_color`, `pointer_glow_color`, `diameter` and others — which it strips out of `final_kw` for vector widgets, so they never reach the native `CTkFrame` constructor and raise `ValueError`. That stripping is correct and necessary.

What was wrong was reading those colours back out of `final_kw` afterwards. They were never in there. Every fallback in the old draw code was therefore *always* taken, and the configured values for `dial_color`, `shadow_color`, `text_color` and `pointer_glow_color` were decorative — the dials rendered in hardcoded colours regardless of what the theme said. Applying fail-loud validation is what surfaced it.

The base class now builds `_local_defaults` from the raw registry block, with `final_kw` layered on top so non-vector keys and constructor overrides keep their precedence:

```
raw_block = _tw.GLOBAL_THEME_REGISTRY.get(self.__class__.__name__) or {}
raw_colors = {k: v for k, v in raw_block.items() if not isinstance(v, dict)}
self._local_defaults = ThemeableWidget._convert_lists_to_tuples(raw_colors)
self._local_defaults.update(self.final_kw)
```

The registry is reached as a **module attribute**, not a direct name import, because `load_initial_framework_themes()` rebinds that global on load — `from ... import GLOBAL_THEME_REGISTRY` captures the empty dict that exists at import time.

Shared API

Member	Type	Description
`state(mode=None)`	method	Getter with no argument; setter with `"normal"` or `"disabled"`. Unbinds clicks, wheel and trackpad input, and repaints from `disabled_map`.
`get_state()`	method	Equivalent to `state()` with no argument.
`configure(state=...)`	method	Same effect as `state()`. Both routes are supported.
`configure(name)`	method	Pygubu-style single-argument query.
`config`	alias	Bound to `configure` **on every class in the family**.
`diameter`	`int`	Square bounding size; sets canvas width and height together.
`divisions`	`int`	Tick count drawn around the outer ring.

`config = configure` **is declared separately on each class, and must be.** Tkinter binds `.config` to `.configure` as its own class attribute — it does not track whichever `configure()` a subclass defines. Without a per-class line, `.config(...)` skips every override and lands on the native widget, bypassing divisions/command/diameter handling and the theme repaint entirely. This was missing from all four dial classes; the same bug was confirmed on `sCTkSegmentedButton` earlier in this project's audit. An inherited alias would not help — it would point at the *parent's* `configure()`.

Redraw model

Two entry points, deliberately separate:

- `_draw_dial_base()` rebuilds everything. Call on geometry, theme or state change.
- `_redraw_indicator()` redraws only the dimple or pointer line, leaving the knob body, ticks and labels alone. Call on a **value** change.

The body is now roughly twenty shaded ovals plus ticks and labels, none of which changes as the dial turns. Rebuilding all of it per detent would make the shading cost real; the split makes it free while tuning. Same pattern as `sCTkSMeter._execute_needle_draw()`, which redraws its needle against a static face.

`_redraw_indicator()` falls back to a full pass if the body isn't on the canvas — first paint, or after a resize wiped it — so a partial update can't leave the dial blank.

Known limitations

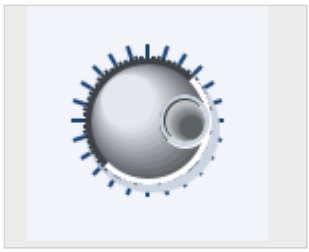
- **Constructor overrides don't work for vector-guarded colours.** `sCTkDialContinuous(master, dial_color="#FF0000")` is silently ignored, because `ThemeableWidget` strips those names from its `kwargs` loop as well as from the theme block. Changing this means touching `themeable_widget.py`.
- **The five shading keys aren't in** `CUSTOM_VECTOR_KEYS`**,** so they do land in `final_kw`. Harmless here — the dial's own `FRAME_VALID_KEYS` whitelist filters them before the native constructor — but a widget that forwarded `final_kw` wholesale would raise.

- **Knurling teeth and the canvas background fallback remain hardcoded.** The teeth are a shadow effect rather than a palette choice; the background fallback is the "a raw canvas needs a renderable colour" case accepted elsewhere in this library.
- **Ticks and labels are not affected by the body shading** — they sit outside the knob radius and draw flat in `text_color`.
- **Scroll handling is duplicated across the three subclasses.** `_process_mac_touchpad_scroll` and `_process_scroll_wheel` are near-identical in each, differing only in the line that applies the step. This is not a candidate for `ScrollBindingMixin`: a dial steps discretely with a cooldown and has no `yview_scroll` target. It belongs in this base class with one overridable step method.

sCTkDialContinuous

Table of Contents

An infinite flywheel tuning encoder module tracking signed velocity delta step increments across an endless 360-degree rotation path (ideal for high-fidelity radio VFO controls, audio mixers, and multi-channel squelch encoders).

Dark Mode:  Light Mode: 

API Property Reference

Property / Feature	Type / Signature	Description
Instantiation	*Constructor*	`sCTkDialContinuous(master)` *(Infinite Tuning Wheel Encoder)*
File Mapping	*Inheritance Tree*	Inherits vector math mechanics and 3D knob rendering directly out of `sCTkDial.py`.
`_scroll_cooldown_seconds`	`float`	Throttle limiting touchpad refresh rates to stabilize fast tuning rolls.

Property / Feature	Type / Signature	Description
`set_position_index(delta)`	`Method (int)`	Manually advances the 3D dimple coordinates via an integer step.
`left_click_callback`	`Callable / None`	**Custom Accelerated Click Hook:** Overrides standard single-step decrements to execute accelerated jumping intervals when clicking the left canvas edge.
`right_click_callback`	`Callable / None`	**Custom Accelerated Click Hook:** Overrides standard single-step increments to execute accelerated jumping intervals when clicking the right canvas edge.
State	`dial.state("disabled")` **OR** `dial.configure(state="disabled")`	**Dual-Routing State Pipeline:** Handles both syntaxes natively. Freezes canvas mouse-wheel scrolling, disables click jump hooks, and shifts visual themes out of `disabled_map` guidelines via a strict sequential re-binding engine.

Constructor

Initialize an infinite flywheel encoder instance. Keyword properties layer safely over centralized configuration defaults and are automatically sanitized by the `ThemeableWidget` mixin layer before the native constructor fires.

```
# Instantiate the themed infinite VFO wheel element
tuning_dial = sCTkDialContinuous(
    master=frame_continuous,
    divisions=24,
    diameter=130,
    command=on_vfo_dial_rotated,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
```

Callback Signature & Usage

Dispatches a raw signed directional integer step change directly to runtime listeners upon rotation changes.

Command

```
# Fires automatically on valid mouse scrolling, touchpad rolling, or click-drag actions
def on_vfo_dial_rotated(clicks_delta: int):
    # Clockwise rotation yields positive steps (+1); Counter-clockwise yields negative steps (-1)
    global current_frequency_hz
    current_frequency_hz += clicks_delta * 100
```

Centralized Stylesheet Setup (`sCTkThemes.json`)

```
{
    "sCTkDialContinuous": {
        "fg_color": ["#F1F5F9", "#0A0A0A"],
        "text_color": ["#1A4375", "#FF9100"],
        "shadow_color": ["#CBD5E1", "#02040A"],
        "dial_color": ["#9E9E9E", "#2A2F3D"],
        "dial_highlight_color": ["#E4E8EC", "#42454B"],
        "dial_shadow_color": ["#5C6165", "#050507"],
        "dial_rim_light_color": ["#FFFFFF", "#8E949C"],
        "dial_rim_shadow_color": ["#3E4245", "#000000"],
        "pointer_glow_color": ["#CBD5E1", "#3A455C"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#4B5563"],
            "dial_color": ["#E2E8F0", "#1A1D24"],
            "pointer_glow_color": ["#CBD5E1", "#334155"]
        }
    }
}
```

Every key above is required — construction raises `KeyError` naming any that are missing. See the base class page for the shared contract.

`pointer_glow_color` is **specific to this variant**: it colours the ring around the finger dimple, and only this dial draws one. It is required in both the top level and `disabled_map`. Selector and Range require `pointer_color` instead.

The dark-mode values above give a black anodised knob. For a brushed-aluminium look, raise `dial_shadow_color` and `dial_highlight_color` toward the light end and brighten the rim.

Other notes

- **Knob rendering:** the body is a shaded dome and the indicator is a recessed finger dimple, sized at 36% of the knob radius with 6% rim clearance — a VFO operator puts a finger in it to spin the dial quickly. Both scale with the knob. See the base class page.
- `.config()` **now works.** This class previously had no `config = configure` alias, so `.config(...)` bypassed every override and landed on the native widget. If existing code called it expecting no effect, it will now have one.
- **Theme colours are live for the first time.** Colours were previously read from `final_kw`, which never contained them, so every dial rendered in hardcoded fallbacks regardless of the theme file. See reading theme colours.
- **Latching Override Independence:** Infinite flywheel dimples loop continuously around the chassis ring, ignoring arc boundary restrictions.

- **Custom Accelerated Steps:** Attaching optional click callbacks allows click events to jump values by wider intervals (e.g., jumping 2 full indices per tap via `set_position_index(2)`) rather than dropping onto the baseline single-step tracking paths.
- **Automated Lifecycle Handshake:** Triggers `self._finalize_themeable_lifecycle()` at the absolute end of the constructor initialization track to cleanly pass instance registration hooks straight back up to Pygubu parent controllers.

Implementation Example & Test Harness

Below is a complete, self-contained test execution script demonstrating how to properly embed an `sCTkDialContinuous` alongside custom click jump hooks and an interactive VFO digital frequency display counter readout.

```
#!/usr/bin/python3
# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for Dial Continuous
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelSecondary, sCTkDialContinuous



# Global state trackers for the interactive bench loop
current_frequency_hz = 14032000


def refresh_frequency_display():
    """Formats integers into a clean MHz telemetry layout readout string."""
    freq_str = f"{current_frequency_hz:08d}"
    formatted_freq = f"{freq_str[-8:-6]}.{freq_str[-6:-3]}.{freq_str[-3:]}"
    if formatted_freq.startswith("."):
        formatted_freq = formatted_freq[1:]

    if lbl_vfo_display.winfo_exists():
        lbl_vfo_display.configure(text=f"VFO Freq: {formatted_freq} MHz")


def on_vfo_dial_rotated(clicks_delta):
    """Event-driven callback tracking signed velocity delta step changes."""
    global current_frequency_hz
    current_frequency_hz += clicks_delta * 100
    current_frequency_hz = max(0, current_frequency_hz)
    refresh_frequency_display()


def my_custom_left_click():
    """Accelerated Jump: Moves 2 complete indexing steps left per click tap."""
    if tuning_dial.cget("state") == "disabled":
        return
    tuning_dial.set_position_index(-2)  # Jump 2 steps left natively


def my_custom_right_click():
    """Accelerated Jump: Moves 2 complete indexing steps right per click tap."""
```

```
    if tuning_dial.cget("state") == "disabled":
        return
    tuning_dial.set_position_index(2)  # Jump 2 steps right natively


def toggle_operational_state():
    """Toggles interaction channels and visual states back and forth."""
    current_mode = tuning_dial.cget("state")
    target = "disabled" if current_mode == "normal" else "normal"

    tuning_dial.configure(state=target)
    lbl_vfo_display.configure(state=target)
    btn_toggle.configure(text="Lock Dial (Set 'disabled')" if target == "normal" else "Unlock Dial (S
    print(f"Logged Verification Hook -> tuning_dial.get_state() = {tuning_dial.get_state()}")


if __name__ == "__main__":
    root = sCTk()
    root.title("sCTkDialContinuous Test Deck")
    root.geometry("380x360")

    base = sCTkFrame(root, corner_radius=8)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    lbl_title = sCTkLabelSecondary(base, text="3. Continuous VFO WHEEL", font=("Arial", 12, "bold"))
    lbl_title.pack(pady=(12, 2))

    tuning_dial = sCTkDialContinuous(
        base,
        divisions=24,
        diameter=130,
        command=on_vfo_dial_rotated,
        left_click_callback=my_custom_left_click,
        right_click_callback=my_custom_right_click
    )
    tuning_dial.pack(pady=10)

    lbl_vfo_display = sCTkLabelSecondary(base, text="VFO Freq: 14.032.000 MHz", font=("Arial", 11, "b
    lbl_vfo_display.pack(pady=10)

    btn_toggle = sCTkButtonPrimary(base, text="Lock Dial (Set 'disabled')", command=toggle_operationa
    btn_toggle.pack(side="bottom", pady=15)

    print("--- BOOT INITIALIZATION PASSTHROUGH ---")
    print(f"Initial Dial State = {tuning_dial.get_state().upper()}")
    print("========================================\n")

    root.mainloop()
```

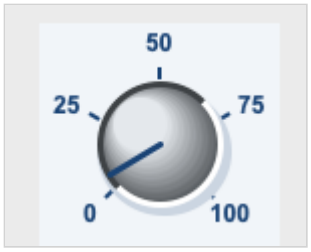
Return to Table of Contents

sCTkDialRange

Table of Contents

A concrete rotary encoder range variant designed for hard-bounded linear controls (e.g., AF/RF volume gain level sliders, squelch limits, or power thresholds). It enforces absolute mechanical dead stops at outer thresholds, preventing directional wraparound loops.

Dark Mode: 

Light Mode: 

API Property Reference

Property / Feature	Type / Signature	Description
Instantiation	*Constructor*	`sCTkDialRange(master)` *(Bounded Linear Range Dial)*
File Mapping	*Inheritance Tree*	Streamlined and compiled programmatically inside `sCTkDial.py` and `ThemeableWidget.py` .
`from_ / min_value`	`int`	Lower boundary threshold (default 0) enforcing absolute counter-clockwise dead stops.
`to / max_value`	`int`	Upper boundary threshold (default 100) enforcing absolute clockwise dead stops.
`divisions`	`int`	Quantized subdivision tick line count painted geometrically across the arc limit sweep.
`_scroll_cooldown_seconds`	`float`	Throttle limiting touchpad refresh rates to stabilize fast range adjustments.
`get() / set(val)`	`Methods -> int`	Unified index query mechanisms to get or force selected integer values.

Property / Feature	Type / Signature	Description
`left_click_callback`	`Callable / None`	**Custom Accelerated Click Hook:** Overrides standard single-step decrements to execute accelerated jumping intervals when clicking the left canvas edge.
`right_click_callback`	`Callable / None`	**Custom Accelerated Click Hook:** Overrides standard single-step increments to execute accelerated jumping intervals when clicking the right canvas edge.
State	`dial.state("disabled")` **OR** `dial.configure(state="disabled")`	**Dual-Routing State Pipeline:** Handles both syntaxes natively. Freezes canvas mouse-wheel scrolling, disables click jump hooks, and shifts visual themes out of `disabled_map` guidelines via a strict sequential re-binding engine.

Constructor

Initialize a custom bounded linear range potentiometer instance. Custom parameters passed from Pygubu builder allocations (like string `translator` tracks or `data_pool` environments) are automatically intercepted, processed, and purged early by the `ThemeableWidget` mixin layer before the native constructor fires. Bounding geometry sizes and limits scale out of central stylesheet registries.

```
# Instantiate an AF Volume gain potentiometer control dial
volume_potentiometer = sCTkDialRange(
    master=control_panel,
    from_=0,
    to=30,
    divisions=6,
    arc_angle=270,
    command=on_volume_level_changed,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
```

Callback Signature & Usage

Dispatches the current absolute active integer value directly to runtime tracking listeners upon position changes.

Command

```
# Fires automatically on valid mouse scrolling, touchpad rolling, or click-drag actions
def on_volume_level_changed(active_value: int):
    # active_value is hard constrained between your from_ and to boundary integers
    print(f"Active Selected Option Value position tracker = {active_value}")
```

Centralized Stylesheet Setup (`sCTkThemes.json`)

```
{
    "sCTkDialRange": {
        "fg_color": ["#F1F5F9", "#0A0A0A"],
        "text_color": ["#1A4375", "#64748B"],
        "shadow_color": ["#CBD5E1", "#02040A"],
        "dial_color": ["#9E9E9E", "#2A2F3D"],
        "dial_highlight_color": ["#E4E8EC", "#42454B"],
        "dial_shadow_color": ["#5C6165", "#050507"],
        "dial_rim_light_color": ["#FFFFFF", "#8E949C"],
        "dial_rim_shadow_color": ["#3E4245", "#000000"],
        "pointer_color": ["#1A4375", "#FF9100"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#4B5563"],
            "dial_color": ["#E2E8F0", "#1A1D24"]
        }
    }
}
```

Every key above is required — construction raises `KeyError` naming any that are missing. See the base class page for the shared contract.

`pointer_color` is **specific to this variant and its Selector sibling**, and colours the pointer line. It was present in the theme file for a long time but read by no code path at all — the pointer drew in `text_color` instead. It is now live, so the pointer can differ from the tick labels. It has no `disabled_map` entry; a disabled pointer falls back to the disabled `text_color` .

Other notes

- **Knob rendering:** the body is a shaded dome, marked with a plain straight line from dead centre out to just short of the rim. An earlier version drew an arrowhead and a raised centre cap; both are gone, along with the cap's two hardcoded outline colours. See the base class page.
- `.config()` **now works.** This class previously had no `config = configure` alias, so `.config(...)` bypassed every override and landed on the native widget. If existing code called it expecting no effect, it will now have one.
- **Theme colours are live for the first time.** Colours were previously read from `final_kw` , which never contained them, so every dial rendered in hardcoded fallbacks regardless of the theme file. See reading theme colours.
- **Bypassing the BaseUI Middleman:** This component inherits cleanly and directly from native CustomTkinter classes and `ThemeableWidget` , completely bypassing the intermediate template layout files entirely to avoid argument deadlocks.

- **Automated Lifecycle Handshake:** At the absolute bottom of the initialization track, the constructor triggers `self._finalize_themeable_lifecycle()` to safely notify top-level Pygubu container managers that the widget is compiled.
- **Absolute Threshold Dead Stops:** Unlike continuous or selector models, scrolling past upper or lower boundaries clips inputs securely using `max(self._from, min(self._to, value))`, blocking accidental overflow.

Implementation Example & Test Harness

Below is a complete, self-contained test execution script demonstrating how to properly embed an `sCTkDialRange` alongside custom click jump hooks and an active volume gain control panel display tracker.

```
#!/usr/bin/python3
# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for Dial Range
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelSecondary, sCTkDialRange


if __name__ == "__main__":

    root = sCTk()
    root.geometry("450x350")
    root.title("Ranged Potentiometer Telemetry Bench")

    base = sCTkFrame(root, corner_radius=8)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    # 1. Live feedback display lane tracking
    lbl_volume = sCTkLabelSecondary(base, text="AF Volume: 15 %", font=("Arial", 11, "bold"))
    lbl_volume.pack(pady=15)


    def my_custom_left_click():
        """Accelerated Jump: Drops 3 units per click tap."""
        if volume_pot.get_state() == "disabled": return
        volume_pot.set(volume_pot.get() - 3)


    def my_custom_right_click():
        """Accelerated Jump: Jumps 3 units per click tap."""
        if volume_pot.get_state() == "disabled": return
        volume_pot.set(volume_pot.get() + 3)


    # 2. Instantiate with explicit limits and tracking labels
    volume_pot = sCTkDialRange(
        base,
        from_=0,
        to=100,
        divisions=5,
        arc_angle=270,
        command=lambda val: lbl_volume.configure(text=f"AF Volume: {int((val / 100) * 100)} %"),
        left_click_callback=my_custom_left_click,
```

```
    right_click_callback=my_custom_right_click
)
volume_pot.pack(expand=True, fill="none", padx=10, pady=10)
volume_pot.set(5)  # Initialize baseline startup volume index


# 3. Dynamic panel interactive state toggle test layout
def toggle_pot_lock():
    current_mode = volume_pot.get_state()
    target = "disabled" if current_mode == "normal" else "normal"

    volume_pot.configure(state=target)
    btn_toggle.configure(text="UNLOCK VOLUME DECK" if target == "disabled" else "LOCK POTENTIOMETI
    print(f"Logged Verification Hook -> volume_pot.get_state() = {volume_pot.get_state()}")


btn_toggle = sCTkButtonPrimary(base, text="LOCK POTENTIOMETER", command=toggle_pot_lock)
btn_toggle.pack(side="bottom", pady=15)

# Standard test assertions routine verification sequences
print("--- BOOT INITIALIZATION PASSTHROUGH ---")
volume_pot.state("disabled")
print("state (Disabled Pass) =", volume_pot.get_state())  # Output: disabled

volume_pot.state("normal")
print("state (Normal Pass)   =", volume_pot.get_state())  # Output: normal
print("========================================\n")

root.mainloop()
```

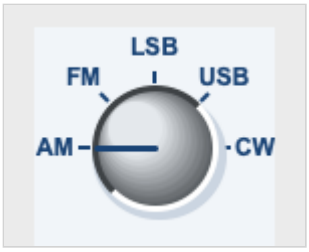
Return to Table of Contents

sCTkDialSelector

Table of Contents

A concrete rotary encoder switch variant designed for stepped selector controls (e.g., band configurations, operating modes, or filter sub-selections). It uses an explicit bounding arc configuration and outputs a clean integer mapping parameter tracking list item indices natively.

Dark Mode: 

Light Mode: 

API Property Reference

Property / Feature	Type / Signature	Description
Instantiation	*Constructor*	`sCTkDialSelector(master)` *(Stepped Arc Selector Dial)*
File Mapping	*Inheritance Tree*	Streamlined and compiled programmatically inside `sCTkDial.py` and `ThemeableWidget.py` .
`labels`	`list [str]`	Ordered array list mapping string tags directly above calculated step lines. Supports raw comma-separated strings inside layout inspectors.
`arc_angle`	`float`	Angular geometric limit (default 270) restricting the pointer range sweep layout.
`_scroll_cooldown_seconds`	`float`	Throttle limiting touchpad refresh rates to stabilize fast selector rolls.
`get() / set(idx)`	`Methods -> int`	Unified index query mechanisms to get or force selected positions.
`left_click_callback`	`Callable / None`	**Custom Accelerated Click Hook:** Overrides standard single-step decrements to execute accelerated jumping intervals when clicking the left canvas edge.
`right_click_callback`	`Callable / None`	**Custom Accelerated Click Hook:** Overrides standard single-step increments to execute accelerated jumping

Property / Feature	Type / Signature	Description
		intervals when clicking the right canvas edge.
State	`dial.state("disabled")` **OR** `dial.configure(state="disabled")`	**Dual-Routing State Pipeline:** Handles both syntaxes natively. Freezes canvas mouse-wheel scrolling, disables click jump hooks, and shifts visual themes out of `disabled_map` guidelines via a strict sequential re-binding engine.

Constructor

Initialize a custom stepped rotary selector switch instance. Properties like `labels` support raw string array text list configurations natively for absolute Pygubu inspector panel compatibility. Custom attributes from Pygubu builder allocations (like string `translator` tracks) are automatically intercepted and sanitized by the `ThemeableWidget` mixin layer before the native constructor fires.

```
# Instantiate a 5-position operating mode rotary switch selector
mode_switch = sCTkDialSelector(
    master=control_panel,
    labels=["AM", "FM", "LSB", "USB", "CW-N"],
    arc_angle=180,
    command=on_operating_mode_changed,
    left_click_callback=my_custom_left_click,
    right_click_callback=my_custom_right_click
)
```

Callback Signature & Usage

Dispatches the current absolute active list item integer index directly to runtime configuration listeners.

Command

```
# Fires automatically on valid mouse scrolling, touchpad rolling, or click-drag actions
def on_operating_mode_changed(active_index: int):
    # active_index maps directly to items in your labels block list (0, 1, 2, etc.)
    print(f"Active Selected Option Index position tracker = {active_index}")
```

Centralized Stylesheet Setup (`sCTkThemes.json`)

```
{
    "sCTkDialSelector": {
```

```
            "fg_color": ["#F1F5F9", "#0A0A0A"],
            "text_color": ["#1A4375", "#FF9100"],
            "shadow_color": ["#CBD5E1", "#02040A"],
            "dial_color": ["#9E9E9E", "#2A2F3D"],
            "dial_highlight_color": ["#E4E8EC", "#42454B"],
            "dial_shadow_color": ["#5C6165", "#050507"],
            "dial_rim_light_color": ["#FFFFFF", "#8E949C"],
            "dial_rim_shadow_color": ["#3E4245", "#000000"],
            "pointer_color": ["#1A4375", "#FF9100"],
            "disabled_map": {
                "text_color": ["#94A3B8", "#4B5563"],
                "dial_color": ["#E2E8F0", "#1A1D24"]
            }
        }
    }
```

Every key above is required — construction raises `KeyError` naming any that are missing. See the base class page for the shared contract.

`pointer_color` is **specific to this variant and its Range sibling**, and colours the pointer line. It was present in the theme file for a long time but read by no code path at all — the pointer drew in `text_color` instead. It is now live, so the pointer can differ from the tick labels. It has no `disabled_map` entry; a disabled pointer falls back to the disabled `text_color`.

Other notes

- **Knob rendering:** the body is a shaded dome, marked with a plain straight line from dead centre out to just short of the rim. An earlier version drew an arrowhead and a raised centre cap; both are gone, along with the cap's two hardcoded outline colours. See the base class page.
- `.config()` **now works.** This class previously had no `config = configure` alias, so `.config(...)` bypassed every override and landed on the native widget. If existing code called it expecting no effect, it will now have one.
- **Theme colours are live for the first time.** Colours were previously read from `final_kw`, which never contained them, so every dial rendered in hardcoded fallbacks regardless of the theme file. See reading theme colours.
- **Bypassing the BaseUI Skeletons:** This component avoids all autogenerated Pygubu intermediate templates, connecting the component straight to CustomTkinter's appearance modes via programmatic multiple inheritance tracks.
- **Automated Lifecycle Handshake:** Fires `self._finalize_themeable_lifecycle()` at the absolute end of the constructor initialization track to cleanly pass instance registration hooks straight back up to Pygubu parent controllers.
- **Rolling Selector Loops:** When spinning scroll wheels beyond boundary edges, the index modulo calculates the length of the string array, snapping the cursor back around to index 0 smoothly.

Implementation Example & Test Harness

Below is a complete, self-contained test execution script demonstrating how to properly embed an `sCTkDialSelector` alongside custom click jump hooks and an active mode switch control panel display tracker.

```python
#!/usr/bin/python3

# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for Dial Rotary Switch (sCTkDialSelector)
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkLabelSecondary, sCTkDialSelector


if __name__ == "__main__":

    root = sCTk()
    root.geometry("450x350")
    root.title("Rotary Switch Selector Bench")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    # 1. Attach a live telemetry readout label
    lbl_mode_tag = sCTkLabelSecondary(base, text="Selected Mode: AM", font=("Arial", 11, "bold"))
    lbl_mode_tag.pack(pady=15)


    def my_custom_left_click():
        """Accelerated Jump: Moves 2 complete indexing steps left per click tap."""
        if mode_selector.get_state() == "disabled":
            return
        mode_selector.set(mode_selector.get() - 2)


    def my_custom_right_click():
        """Accelerated Jump: Moves 2 complete indexing steps right per click tap."""
        if mode_selector.get_state() == "disabled":
            return
        mode_selector.set(mode_selector.get() + 2)


    # 2. Instantiate with unique radio deck selector labels and selection trackers
    mode_selector = sCTkDialSelector(
        base,
        labels=["AM", "FM", "LSB", "USB", "CW"],
        arc_angle=180,  # Half-circle step selector arc
        command=lambda idx: lbl_mode_tag.configure(text=f"Selected Mode: {mode_selector._labels[idx]}"
        left_click_callback=my_custom_left_click,
        right_click_callback=my_custom_right_click
    )
    mode_selector.pack(expand=True, fill="none", padx=10, pady=10)


    # 3. Standard application dashboard interaction lock toggle simulation
    def toggle_widget_lock():
        current_mode = mode_selector.get_state()
        target = "disabled" if current_mode == "normal" else "normal"

        mode_selector.configure(state=target)
        btn_lock.configure(
            text="UNLOCK CHANNELS" if target == "disabled" else "LOCK SWITCH (Set 'disabled')"
        )
        print(f"Logged Verification Hook -> mode_selector.get_state() = {mode_selector.get_state()}")
```

```
btn_lock = ctk.CTkButton(base, text="LOCK SWITCH (Set 'disabled')", command=toggle_widget_lock)
btn_lock.pack(side="bottom", pady=10)

# Standard test assertions routine verification sequences
print("--- BOOT INITIALIZATION PASSTHROUGH ---")
mode_selector.state("disabled")
print("state (Disabled Pass) =", mode_selector.get_state())  # Output: disabled

mode_selector.state("normal")
print("state (Normal Pass)   =", mode_selector.get_state())  # Output: normal
print("========================================\n")

root.mainloop()
```

Return to Table of Contents

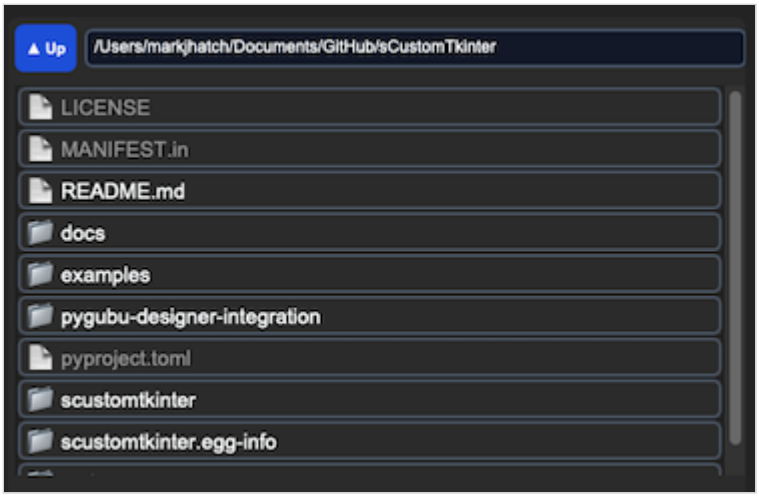
sCTkFileExplorer

(Derived from Separator class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: MoreCustomTkinterWidgets)

Table of Contents

Overview

`sCTkFileExplorer` is a theme-compliant, scrollable file/folder browser — a back button, an editable current-path entry, and a scrollable list of clickable file/folder rows. It inherits `ctk.CTkFrame`, `ScrollBindingMixin`, and `ThemeableWidget`, and builds its own scrolling machinery internally (a raw `tkinter.Canvas` plus a `CTkScrollbar`) rather than composing `sCTkScrollableFrame`.

Dark Mode: 

Light Mode:

Scroll handling comes from `ScrollBindingMixin`**,** the library's single shared implementation, also used by `sCTkScrollableFrame`. This widget supplies two hooks — `_scroll_target()` returns its own internal canvas (no `winfo_parent()` lookup needed, unlike `sCTkScrollableFrame`), and `_scroll_layers()` assembles the widget, canvas, scrollbar, and full row tree.

Three earlier problems are fixed as a result. A global `bind_all("<MouseWheel>", ...)` once affected the entire application rather than this widget, and handled only macOS plus a generic Windows-style delta with no Linux support. A scoped copy of `sCTkScrollableFrame`'s logic then replaced it — but that copy drifted: it walked only *one level* into the row frame, so a row's label or icon was never bound and the wheel did nothing over them, and it had no trackpad accumulator, scrolling on every raw event instead of gating on an accumulated threshold. Consolidating on the mixin fixes both, and trackpad scrolling now matches the rest of the library rather than being noticeably faster and coarser.

Bindings maintain themselves. Activation happens via `after_idle()` and `<Map>`, and a debounced `<Configure>` on the row frame rebinds whenever content changes — so navigating to a new folder, which replaces every row widget, is picked up automatically with no explicit call.

Constructor

```
sCTkFileExplorer(master=None, initialdir=None, type="file", filetypes=None, ...)
```

Parameter	Type	Description
`master`	widget	Parent container.
`initialdir`	`str`	Starting directory.
`type`	`"file"` / `"directory"`	Whether individual files are selectable, or only directories.
`filetypes`	`list[str]`	File extension filter (only meaningful when `type="file"`).
`command`	`callable`	Called with the clicked path (a string) on a single click.
`double_click_command`	`callable`	Called with `(self, path)` on a double click.
`width` / `height`	`int`	Overall widget dimensions.
`**kwargs`	—	Any native `CTkFrame` argument, or a theme-key override (see Theming).

```
explorer = sCTkFileExplorer(control_panel, initialdir="/Users/you/Documents", type="directory", width=
explorer.pack(fill="both", expand=True)
```

`command` **receives a path, not the widget.** An earlier version passed `self` (the widget instance) instead of the clicked path — confirmed and fixed, since the only real-world caller (`sCTkPathChooser`) expected a path string and would have received garbage.

Methods

Method	Returns	Description
`_finalize_split_bindings()`	`None`	Wires the back button, path entry, and canvas resize handling, then loads the initial directory. Auto-scheduled via `self.after(10, ...)` inside `__init__` — you don't need to call it yourself. It no longer governs scroll activation; `ScrollBindingMixin` handles that independently via `after_idle()`, which fires when Tk is actually idle rather than after a guessed delay.
`state(mode=None)` / `get_state()`	`str`	Gets or sets `"normal"` / `"disabled"`, dimming the back button, path entry, scrollbar, and all rows. Disabling also stops scrolling entirely — wheel, trackpad, and scrollbar dragging — matching `sCTkScrollableFrame`.
`configure(**kwargs)`	`None`	Standard configuration, accepting `state`, `type`, `initialdir`, `initialfile`, `filetypes`, and

Method	Returns	Description
		`double_click_command` alongside native options.
`configure(name)`	`tuple`	Pygubu-style single-argument query for any of the six properties above. **Previously broken:** the implementation read `pname = args` rather than `args[0]`, so every comparison tested a tuple against a string and all six queries fell through to the native widget. Pygubu could not read any of them.

There's currently no public method for programmatic navigation from outside the widget — `path_to_show` (a `StringVar`) has no automatic refresh trace of its own (unlike `selected_path`), so navigating externally means setting it *and* explicitly calling the private `_fill_explorer()` afterward, matching the pattern used internally by the back button. This is a real API gap, not a documented feature.

Theming (`sCTkThemes.json`)

```
{
    "sCTkFileExplorer": {
        "btn_font": ["Arial", 11, "bold"],
        "entry_font": ["Arial", 12, "normal"],
        "btn_fg": ["#3B82F6", "#1D4ED8"],
        "btn_hover": ["#2563EB", "#1E40AF"],
        "btn_text_color": ["#FFFFFF", "#F9FAFB"],
        "btn_border_color": ["#1E3A8A", "#1E3A8A"],
        "entry_fg": ["#FFFFFF", "#111827"],
        "entry_text_color": ["#1F2937", "#F9FAFB"],
        "entry_border_color": ["#CBD5E1", "#475569"],
        "row_active_text": ["#1F2937", "#F9FAFB"],
        "row_dimmed_text": ["#94A3B8", "#64748B"],
        "button_color": ["#64748B", "#4B5563"],
        "disabled_map": {
            "btn_fg": ["#CBD5E1", "#334155"],
            "btn_border_color": ["#CBD5E1", "#334155"],
            "btn_text_color": ["#94A3B8", "#64748B"],
            "entry_fg": ["#F3F4F6", "#1F2937"],
            "entry_border_color": ["#CBD5E1", "#475569"],
            "entry_text_color": ["#94A3B8", "#64748B"],
            "row_dimmed_text": ["#5A6672", "#3A4552"],
            "button_color": ["#CBD5E1", "#334155"]
        }
    }
}
```

`button_color` **is required at the top level and in** `disabled_map`**.** This is a harder requirement than it looks: `_process_live_theme_repaint()` is bound to `<Visibility>`, so it fires essentially every time the widget is displayed, not only when explicitly disabled. A theme block missing `button_color` therefore raises `KeyError` on first display, not merely on disable. The values shown above (`["#64748B", "#4B5563"]` normal, `["#CBD5E1", "#334155"]` disabled) are the suggested pair.

`button_color` controls the internal scrollbar's color, distinct from `btn_fg` (the back button).

`row_active_text / row_dimmed_text` control file/folder row text color — `row_active_text` for a normal, selectable row; `row_dimmed_text` for either a row excluded by the current filter, or every row when the whole widget is disabled. Both required at the top level; `row_dimmed_text` is also hard-required in `disabled_map` for the whole-widget-disabled case.

Only `button_color` and `row_dimmed_text` genuinely hard-fail if missing from `disabled_map`. The other six disabled-state keys (`btn_fg`, `btn_border_color`, `btn_text_color`, `entry_fg`, `entry_border_color`, `entry_text_color`) gracefully fall back to their top-level/normal value if `disabled_map` doesn't override them — not a crash risk, just means that specific property simply won't visually change when disabled unless you give it a distinct value.

The internal raw `Canvas`'s background color isn't part of this theme block at all — it's derived from this widget's own `fg_color`, falling back to a fixed neutral pair (`#1C1C1C` dark / `#F3F4F6` light) if `fg_color` is `"transparent"`, since a raw Canvas can't render CTk's transparent pseudo-value. This is a different kind of fallback than the ones above — not a theme gap, but a genuine "needs an actual renderable color" situation, the same accepted pattern used in `sCTkFrameLabeledPrimary`'s scrollbar hiding.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkFileExplorer, sCTkLabelPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("420x480")
    root.title("FileExplorer Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    status = sCTkLabelPrimary(base, text="Selected: (none yet)")
    status.pack(anchor="w", pady=(0, 8))

    explorer = sCTkFileExplorer(
        base, type="directory",
        command=lambda path: status.configure(text=f"Selected: {path}"),
    )
    explorer.pack(expand=True, fill="both")

    root.mainloop()
```

Known Limitations

- No public method for programmatic navigation — see Methods above.
- Missing a required theme key raises `KeyError` at first use, naming exactly which key and whether it's needed at the top level or in `disabled_map`.

- **The debounced rebind also runs on genuine resizes.** `<Configure>` on the row frame doesn't distinguish "rows were added" from "the window was dragged", so resizing rebinds too. One coalesced pass rather than one per event, but on a very large directory it isn't free.
- **The internal `Canvas` is a raw `tkinter.Canvas`,** not a themed widget, so its background is derived rather than themed — see the note at the end of Theming.
- **The scrollbar stays visible when disabled, just inert.** It can't be dragged, but it isn't hidden — CustomTkinter's scrollbar has no native disabled state to lock. Same limitation as `sCTkScrollableFrame`.

Fixed: dragging the scrollbar when the files didn't fill the frame used to push the rows down to the bottom, leaving empty space above them. The scroll region was set straight from `bbox("all")`, which is *shorter* than the visible canvas when content is short, and Tk will still scroll within an undersized region. The region is now grown to at least the canvas height in that case, so `yview` has nowhere to go and scrolling correctly does nothing.

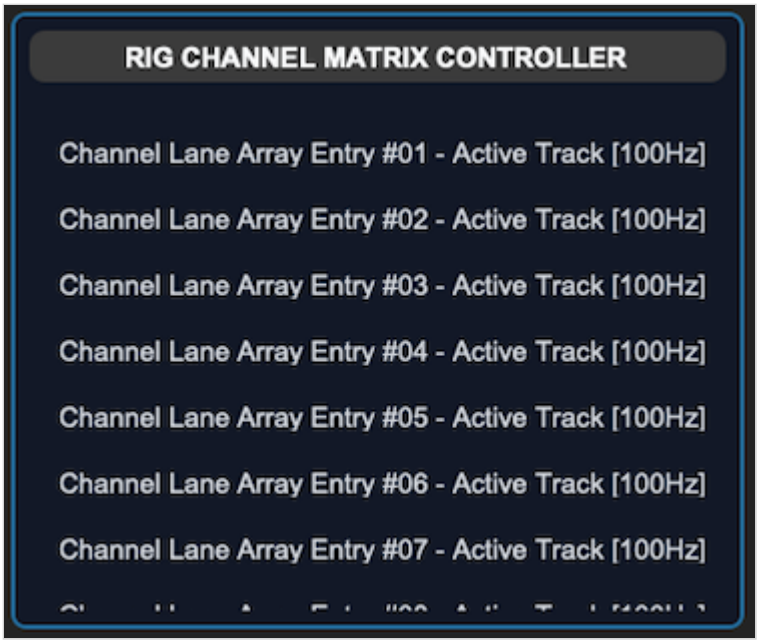
Return to Table of Contents

sCTkFrameLabeledPrimary

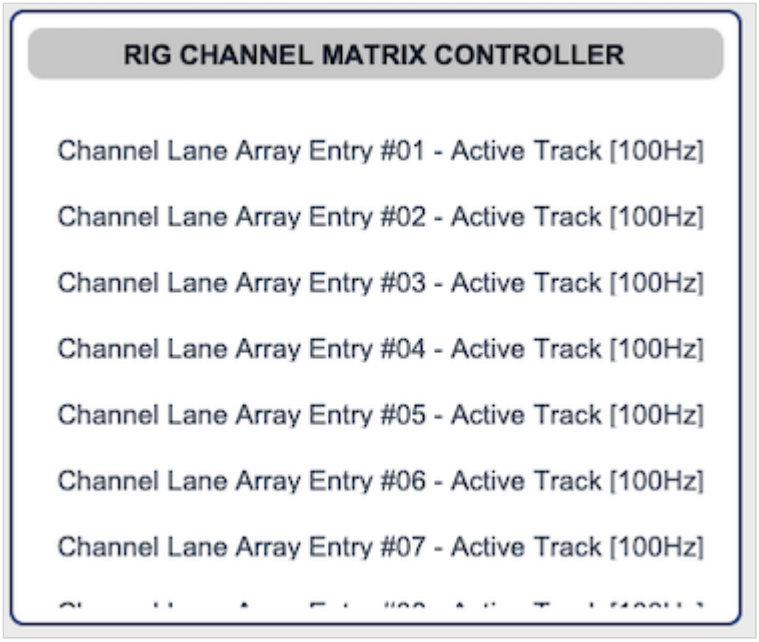
Table of Contents

Overview

`sCTkFrameLabeledPrimary` is a themeable, high-emphasis labeled container panel — the more prominent of the library's two labeled frame tiers (see also `sCTkFrameLabeledSecondary`). It's built on `customtkinter.CTkScrollableFrame`, but deliberately used purely for its native title-label feature — the model here is `ttk.LabelFrame`, which never scrolls. Scrolling is intentionally suppressed; this is a labeled, bordered panel, not a scroll viewport.



Dark Mode: Light Mode:



Constructor

```
sCTkFrameLabeledPrimary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkScrollableFrame` argument (most usefully `label_text`, the panel's title), or an override for one of the theme keys listed under Theming.

```
channel_panel = sCTkFrameLabeledPrimary(
    master=control_root,
    label_text="Channel Settings",
)
channel_panel.pack(expand=True, fill="both", padx=25, pady=25)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's visual "disabled" state. This is purely cosmetic — plain frame-family widgets have no native interactivity to lock. Disabling this panel does **not** automatically disable widgets placed inside it; that's the caller's responsibility, the same pattern used in this project's own test harness (loop over the panel's children and call `.configure(state=...)` on each one).
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()`. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `border_color`, and `label_text_color`.
`winfo_children(include_private=False)`	`list`	By default, filters out children whose exact class name is `"CTkLabel"`, `"Label"`, `"CTkFrame"`, or `"Frame"` — internal furniture `CTkScrollableFrame` creates for its own title row and canvas wrapper. Pass `include_private=True` for the raw, unfiltered list. **Known limitation:** this is a class-name check, not an identity check — a plain, un-themed `customtkinter.CTkLabel` / `CTkFrame` added directly as a child would be filtered out too, since its class name matches. Themed `sCTk`-prefixed widgets are unaffected.
`get_children()`	`list`	Equivalent to `winfo_children(include_private=False)`.

Method	Returns	Description
`get_all_children()`	`list`	Equivalent to `winfo_children(include_private=True)`.
`get_container()`	`self`	Returns the widget itself. Provided for API symmetry with composite widgets (like `sCTkOptionMenuSecondary`) that wrap a separate inner container.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block, including `label_font` and `corner_radius`, is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `label_text_color`, `border_width`, and `label_font` are recomputed from the theme's normal values or its `disabled_map`.

```
{
    "sCTkFrameLabeledPrimary": {
        "border_width": 2,
        "border_color": ["#1A4375", "#2471A3"],
        "fg_color": ["#FFFFFF", "#111827"],
        "corner_radius": 8,
        "label_font": ["Arial", 15, "bold"],
        "label_text_color": ["#111827", "#F9FAFB"],
        "disabled_map": {
            "border_color": ["#CBD5E1", "#374151"],
            "label_text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

On the internal scrollbar: since this widget is built on `CTkScrollableFrame`, a scrollbar exists internally even though scrolling isn't the intent. It's suppressed by matching its colors to the frame's background and collapsing its width to `0`. This is a workaround, not a true disable — confirmed by direct investigation, CustomTkinter's native scrollbar has no disabled state to lock in the first place, even on an unwrapped `CTkScrollableFrame`. Matching colors and zeroing width is the closest achievable approximation.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Safe to use as a base class for your own composite widgets. If you build a composite widget by inheriting `sCTkFrameLabeledPrimary` directly, construction is protected on two fronts: a run-once guard in `ThemeableWidget.__init__` stops your composite's own `final_kw` from being silently overwritten if your widget explicitly calls `ThemeableWidget.__init__` before `super().__init__()`; and this widget's own constructor only forwards the specific keys native `CTkScrollableFrame` actually accepts (confirmed directly against CustomTkinter's

source, which has no fallback `**kwargs` at all — every parameter is explicitly named, so this matters more here than for most widgets). This only matters for that composition pattern — constructing a plain `sCTkFrameLabeledPrimary` directly is unaffected either way.

Example

```
from scustomtkinter import sCTkButtonPrimary, sCTkLabelSecondary, sCTk, sCTkFrameLabeledPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x450")
    root.title("FrameLabeledPrimary Example")

    channel_panel = sCTkFrameLabeledPrimary(root, label_text="Channel Settings")
    channel_panel.pack(expand=True, fill="both", padx=25, pady=25)

    for i in range(1, 6):
        item = sCTkLabelSecondary(channel_panel, text=f"Setting {i}")
        item.pack(pady=4, fill="x", padx=10)

    def toggle_panel_state():
        target = "disabled" if channel_panel.get_state() == "normal" else "normal"
        channel_panel.configure(state=target)

        # Disabling the panel is purely cosmetic -- cascade to children explicitly.
        for child in channel_panel.get_children():
            if hasattr(child, "configure"):
                child.configure(state=target)

        toggle_btn.configure(text="Enable Panel" if target == "disabled" else "Disable Panel")

    toggle_btn = sCTkButtonPrimary(root, text="Disable Panel", command=toggle_panel_state)
    toggle_btn.pack(pady=15)

    root.mainloop()
```

Known Limitations

- Disabling this widget is purely cosmetic — it does not lock interactivity, and does not cascade to child widgets automatically.
- The internal scrollbar cannot be truly disabled (a CustomTkinter limitation, confirmed by direct investigation, not something this wrapper can work around) — only visually hidden via color-matching and zero width.
- `winfo_children()` 's default filtering is a class-name check, not an identity check — see the Methods table above for the specific edge case this can miss.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.

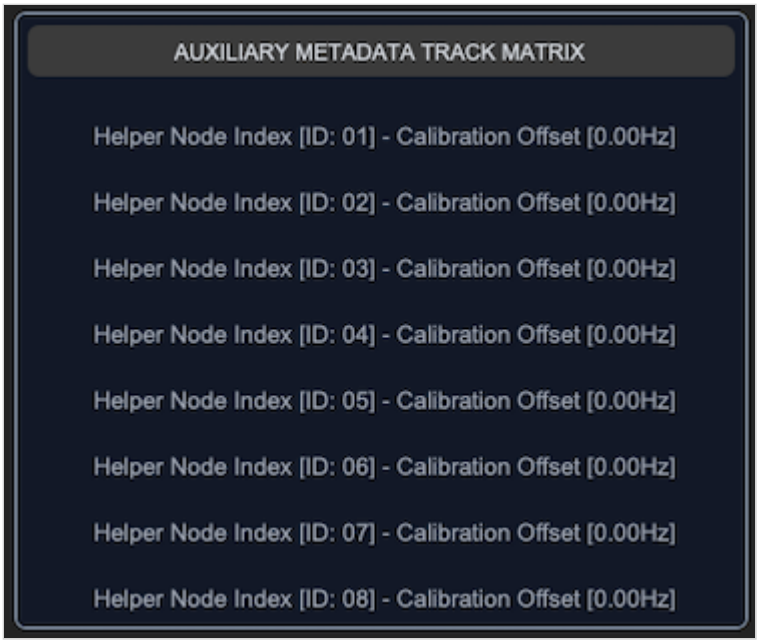
Return to Table of Contents

sCTkFrameLabeledSecondary

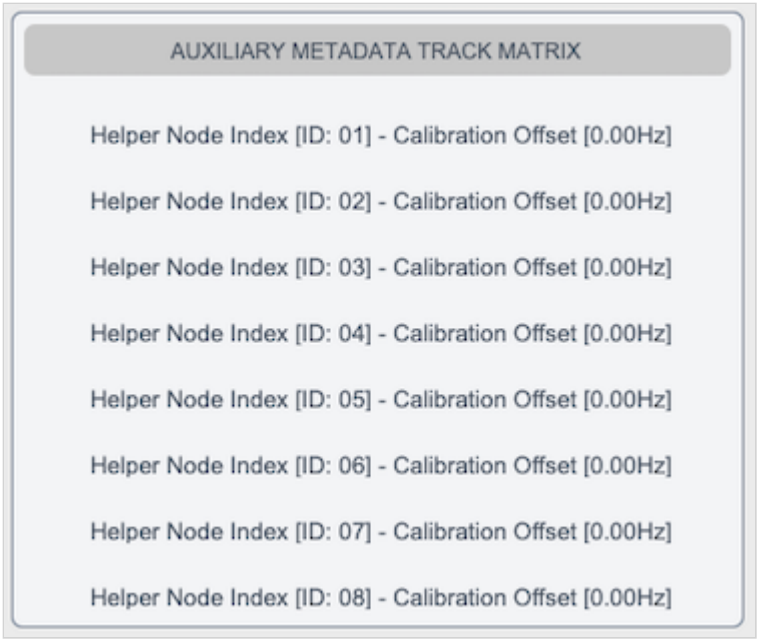
Table of Contents

Overview

`sCTkFrameLabeledSecondary` is a themeable, lower-emphasis labeled container panel — see also `sCTkFrameLabeledPrimary`. It's built on `customtkinter.CTkScrollableFrame`, but deliberately used purely for its native title-label feature — the model here is `ttk.LabelFrame`, which never scrolls. Scrolling is intentionally suppressed; this is a labeled, bordered panel, not a scroll viewport.

Dark Mode: 

Light Mode:



Constructor

```
sCTkFrameLabeledSecondary(master=None, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`**kwargs`	—	—	Any native `CTkScrollableFrame` argument (most usefully `label_text`, the panel's title), or an override for one of the theme keys listed under Theming.

```
notes_panel = sCTkFrameLabeledSecondary(
    master=control_root,
    label_text="Notes",
)
notes_panel.pack(expand=True, fill="both", padx=25, pady=25)
```

Methods

Method	Returns	Description
`state(mode=None)`	`str`	Gets or sets the widget's visual "disabled" state. Purely cosmetic — does not lock interactivity, and does not cascade to child widgets automatically.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus: passing `state=...` routes to `state()`. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `fg_color`, `border_color`, and `label_text_color`.
`winfo_children(include_private=False)`	`list`	By default, filters out children whose exact class name is `"CTkLabel"`, `"Label"`, `"CTkFrame"`, or `"Frame"` — internal furniture `CTkScrollableFrame` creates for its own title row and canvas wrapper. Pass `include_private=True` for the raw, unfiltered list. Same class-name-based known limitation as `sCTkFrameLabeledPrimary` — see that widget's docs for the specific edge case.
`get_children()`	`list`	Equivalent to `winfo_children(include_private=False)`.
`get_all_children()`	`list`	Equivalent to `winfo_children(include_private=True)`.
`get_container()`	`self`	Returns the widget itself. Provided for API symmetry with composite widgets that wrap a separate inner container.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key in the widget's theme block is merged with any matching keyword arguments and applied when the widget is built.
- **Re-applied on every `state()` change** — `fg_color`, `border_color`, `label_text_color`, `border_width`, and `label_font` are recomputed from the theme's normal values or its `disabled_map`.

```
{
    "sCTkFrameLabeledSecondary": {
        "border_width": 1,
        "border_color": ["#64748B", "#94A3B8"],
        "fg_color": ["#F3F4F6", "#111827"],
        "corner_radius": 6,
        "label_font": ["Arial", 12, "normal"],
        "label_text_color": ["#4B5563", "#D1D5DB"],
        "disabled_map": {
            "border_color": ["#CBD5E1", "#374151"],
            "label_text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

On the internal scrollbar: same situation as `sCTkFrameLabeledPrimary` — a scrollbar exists internally since this is built on `CTkScrollableFrame`, even though scrolling isn't the intent. It's suppressed by matching its colors to the frame's background and collapsing its width to `0`, since CustomTkinter's native scrollbar has no disabled state to lock in the first place.

Colors are stored and passed through as raw `(light, dark)` tuples rather than resolved to a single value ahead of time, so they should correctly follow system/app appearance-mode changes automatically — the same approach validated on `sCTkComboBox`, `sCTkSegmentedButton`, and the button family, though not separately re-confirmed for this specific widget.

Safe to use as a base class for your own composite widgets. Same protection as `sCTkFrameLabeledPrimary` — see that widget's docs for the full reasoning (the run-once guard in `ThemeableWidget.__init__`, plus this widget's own constructor filtering keys down to only what native `CTkScrollableFrame` actually accepts before its own constructor call).

Example

```
from scustomtkinter import sCTkButtonPrimary, sCTkLabelSecondary, sCTk, sCTkFrameLabeledSecondary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x450")
    root.title("FrameLabeledSecondary Example")

    notes_panel = sCTkFrameLabeledSecondary(root, label_text="Notes")
    notes_panel.pack(expand=True, fill="both", padx=25, pady=25)

    for i in range(1, 6):
        item = sCTkLabelSecondary(notes_panel, text=f"Note {i}")
        item.pack(pady=4, fill="x", padx=10)
```

```
    def toggle_panel_state():
        target = "disabled" if notes_panel.get_state() == "normal" else "normal"
        notes_panel.configure(state=target)

        # Disabling the panel is purely cosmetic -- cascade to children explicitly.
        for child in notes_panel.get_children():
            if hasattr(child, "configure"):
                child.configure(state=target)

        toggle_btn.configure(text="Enable Panel" if target == "disabled" else "Disable Panel")

    toggle_btn = sCTkButtonPrimary(root, text="Disable Panel", command=toggle_panel_state)
    toggle_btn.pack(pady=15)

    root.mainloop()
```

Known Limitations

- Disabling this widget is purely cosmetic — it does not lock interactivity, and does not cascade to child widgets automatically.
- The internal scrollbar cannot be truly disabled (a CustomTkinter limitation, confirmed by direct investigation) — only visually hidden via color-matching and zero width.
- `winfo_children()` 's default filtering is a class-name check, not an identity check — see `sCTkFrameLabeledPrimary` 's docs for the specific edge case this can miss.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.

Return to Table of Contents

sCTkMessagebox

(Derived from Separator class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: MoreCustomTkinterWidgets)

Table of Contents

The `sCTkMessagebox` is an advanced, themeable dialog window system designed to provide critical messages to the user. It replaces standard OS message alerts with modular, center-positioned dialogue boxes featuring dynamic text-

wrapping, automated parent window tracking calculations, custom asset handling, and support for dual high-contrast action selection layouts that return boolean runtime parameters.

Dark Mode: 

Light Mode:

API Constructor Reference

```
sCTkMessagebox(title, message, typ, master=None, buttons="ok", ok_text="Ok", yes_text="Yes", no_text='
```

Parameter Name	Data Type	Default Value	Description
`title`	`str`	*Required*	Text displayed inside the top operating window header bar title deck.
`message`	`str`	*Required*	Body text string message container paragraph to display inside the prompt panel.
`typ`	`str`	*Required*	Alert asset track type classification identifier. Accepts `"info"`, `"warning"`, or `"error"`.
`master`	`any`	`None`	Reference pointer tracking your root window or parent `sCTkFrame` to calculate centering bounds.
`buttons`	`str`	`"ok"`	Layout selection control mapping. Accepts `"ok"` (single center prompt) or `"yes_no"` (twin balanced selections).
`ok_text`	`str`	`"Ok"`	Custom display string label mapped to the single button layout option track.
`yes_text`	`str`	`"Yes"`	Display string assigned to the primary confirmation button choice track.
`no_text`	`str`	`"No"`	Display string assigned to the secondary dismissal button choice track.
`width`	`int`	`400`	Manual window width boundary tracking restriction limit measured in pixels.

Global Shortcut Function Handlers

To launch modal dialog blocks quickly inside callback triggers without handling complete class instantiations manually, utilize these pre-wired shortcuts via the `messagebox` namespace proxy:

Standard Alert Prompts (Returns `True` upon closure)

```
sCTkMessagebox.showinfo(title, message, ok_text="Ok", width=400, master=root)
sCTkMessagebox.showwarning(title, message, ok_text="Ok", width=400, master=root)
sCTkMessagebox.showerror(title, message, ok_text="Ok", width=400, master=root)
```

Confirmation Prompt Shortcuts (Returns primitive Python `True` or `False` boolean states)

```
sCTkMessagebox.askyesno(title, message, yes_text="Yes", no_text="No", width=400, master=root)
sCTkMessagebox.askwarningyesno(title, message, yes_text="Yes", no_text="No", width=400, master=root)
```

```
sCTkMessagebox.askerroryesno(title, message, yes_text="Yes", no_text="No", width=400, master=root)
```

Simple Syntax Quick-Reference Guide

Below are clean, minimal use-cases showcasing how to call each convenience shortcut using the standardized `messagebox` proxy engine.

1. `sCTkMessagebox.showinfo`

Used for general application notifications, status confirmations, and completions.

```
from scustomtkinter import sCTkMessagebox

# Displays a standard informative dialog popup
sCTkMessagebox.showinfo("System Init", "Satellite link successfully established.", master=root)
```

2. `sCTkMessagebox.showwarning`

Used to display alert parameters, non-fatal operational boundary breaches, or layout cautions.

```
from scustomtkinter import sCTkMessagebox

# Displays a warning alert box with a custom approval button text
sCTkMessagebox.showwarning("Battery Low", "Backup power source dropped below 15%.", ok_text="Acknowle
```

3. `sCTkMessagebox.showerror`

Used to halt operations when a severe terminal failure or unhandled exception block is triggered.

```
from scustomtkinter import sCTkMessagebox

# Displays a fatal critical error box
sCTkMessagebox.showerror("TX Failure", "Transmitter hardware thermal overload detected.", master=root
```

4. `sCTkMessagebox.askyesno`

Launches a standard query dialogue window, returning a boolean flag based on the user's action.

```
from scustomtkinter import sCTkMessagebox

# Captures true/false verification states
if sCTkMessagebox.askyesno("Log Session", "Do you wish to save the active telemetry log files?", maste
    print("User clicked YES: Executing write loop...")
else:
    print("User clicked NO: Dropping record data...")
```

5. `sCTkMessagebox.askwarningyesno`

Launches a critical query box carrying high-visibility alert graphics for destructive actions.

```
from scustomtkinter import sCTkMessagebox

# Captures permission states for hazardous overrides
override_allowed = messagebox.askwarningyesno(
    "Frequency Sync",
    "VFO phase lock is currently unstable. Force manual override?",
    yes_text="Force Override",
    no_text="Abort Scan",
    master=root
)
```

6. `sCTkMessagebox.askerroryesno`

Launches an error-status confirmation panel, typical for prompt actions following a hard code drop.

```
from scustomtkinter import sCTkMessagebox

# Captures choice states to run system self-healing scripts
if sCTkMessagebox.askerroryesno("Cascade Failure", "Buffer buffer overflow hit. Attempt a cold reset?'
    # Execute recovery sequence...
    pass
```

Centralized Stylesheet Setup (`sCTkThemes.json`)

```
{
    "sCTkMessagebox": {
        "fg_color": ["#F1F5F9", "#1C1C1C"],
        "font": ["Arial", 14],
        "text_color": ["#1A1A1A", "#E5E5E5"]
    }
}
```

Every key above is required. Construction raises `KeyError` naming the missing one, rather than substituting a plausible default that would make an incomplete block look merely slightly-off.

`font` and `text_color` style the message label. `fg_color` is the dialog window background, and is **new** — this widget previously forwarded its raw constructor keywords to native `CTkToplevel` rather than the resolved theme keywords, so the theme block never reached the window at all and the dialog rendered in CustomTkinter's own default background. `ThemeableWidget` 's resolution work was discarded for everything except the two label keys read back manually.

Keyword filtering. Theme keywords are now filtered against a whitelist before the native constructor sees them, because `CTkToplevel` names only `fg_color` explicitly and passes everything else through to

`tkinter.Toplevel`, which raises `TclError` on any option it doesn't recognise. This closes a latent crash as well: a caller passing `font=` to this widget would previously have had it forwarded straight through.

There is no `disabled_map` and no `state()`. This is a modal dialog — it grabs input on construction and destroys itself on dismissal, so there is no interval in which a disabled appearance would mean anything.

Layout & Text Wrapping Integration Rules

To completely bypass CustomTkinter's internal multi-line font calculation limitations, this widget uses Python's native `textwrap` module to inject hard newline coordinates before passing layout parameters to your primary text components.

Observe these implementation traits:

- **Horizontal Capsule Brackets**: When `buttons="yes_no"` is active, Column 0 and Column 1 utilize an interlocking `uniform="dialog_buttons"` constraint map. This completely locks both buttons to an identical layout grid pixel width, regardless of text length mismatches.
- **Vertical Safety Gutter**: Text layout nodes use `padx=(10, 35)` paired alongside a calculated character width subtraction map. This forces word bounds to drop downwards well before interacting with the physical window frame margin boundary.
- **Autonomous Resizing**: The `_center_window` geometry calculations lock your custom manual `width` pixel profile constraint, but query the active required widget layout height parameters dynamically via `winfo_reqheight()`. This allows window frames to expand or shrink vertically based on your text content volume requirements automatically.

Configuration

`configure()` and `config()` behave as they do elsewhere in the library: keyword arguments are applied normally, a single positional dict is merged into them, and any other single positional value is forwarded to the native widget.

Three separate defects were fixed here, all silent:

- `super().configure(args)` **passed the whole tuple** as one positional argument instead of unwrapping it, so every single-argument call forwarded a malformed value.
- `if args and isinstance(args, dict)` — `args` is always a tuple, so that branch could never fire and the dict form of `configure()` was dead code.
- **No `config = configure` alias existed,** so `.config(...)` bypassed the override entirely and landed on the native widget. Tkinter binds `.config` as a separate class attribute; it does not track a subclass's override.

Implementation Example & Test Harness

Below is a complete, self-contained test execution script demonstrating how to properly map shortcut handlers, custom text boundaries, and dynamic boolean feedback out of an interactive transceiver dashboard setup.

```python
#!/usr/bin/python3
# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for Messagebox
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary,sCTk, sCTkMessagebox

if __name__ == "__main__":
    root = sCTk()
    root.geometry("300x520")
    root.title("Message Example")

    long_msg = "Warning: The VFO phase lock loop has lost lock synchronization with the master synthes

    # 🚀 Clean functional callbacks using the messagebox namespace!
    def trigger_info_ask():
        print(f"Feedback: {sCTkMessagebox.askyesno('Info Query', 'Log parameter data?', yes_text='Log

    def trigger_warning_ask():
        print(f"Feedback: {sCTkMessagebox.askwarningyesno('Band Switch', long_msg, yes_text='Override

    def trigger_error_ask():
        print(f"Feedback: {sCTkMessagebox.askerroryesno('Fatal Error', 'Attempt buffer cold reset?', y

    # 🚀 Native drop-in style execution pass!
    sCTkButtonPrimary(root, text="Test Info (OK)", width=200, command=lambda: sCTkMessagebox.showinfo
    sCTkButtonPrimary(root, text="Test Info (Yes/No)", width=200, command=trigger_info_ask).pack(pady=
    sCTkButtonPrimary(root, text="Test Warning (OK)", width=200, command=lambda: sCTkMessagebox.showwa
    sCTkButtonPrimary(root, text="Test Warning (Yes/No)", width=200, command=trigger_warning_ask).pack
    sCTkButtonPrimary(root, text="Test Error (OK)", width=200, command=lambda: sCTkMessagebox.showerro
    sCTkButtonPrimary(root, text="Test Error (Yes/No)", width=200, command=trigger_error_ask).pack(pa

    root.mainloop()
```

sCTkPathChooser

Table of Contents

Overview

`sCTkPathChooser` is a theme-compliant single-line path entry paired with a "Browse..." button that opens an `sCTkFileExplorer` in a modal popup. It inherits `ctk.CTkFrame` directly, composing an internal

`sCTkEntryPrimary` and `sCTkButtonPrimary`.

Dark Mode: 

Light Mode:

Every property this widget forwards to its internal entry (`justify` , `width` , `height`) has been confirmed valid against CustomTkinter's own real `CTkEntry` source, the same verification done for `sCTkSpinbox` . There's no risk of this widget sending an unrecognized property to its own entry.

Constructor

```
sCTkPathChooser(master=None, initialdir=None, initialfile=None, type="file",
                filetypes=None, title=None, defaultextension=None,
                justify="left", entry_height=None, browser_width=None,
                browser_height=None, btn_text="Browse...", **kwargs)
```

Parameter	Type	Description
`master`	widget	Parent container.
`initialdir` / `initialfile`	`str`	Starting directory/filename for the browser popup.
`type`	`"file"` / `"directory"`	Whether individual files are selectable, or only directories.
`filetypes`	`list[str]`	File extension filter.
`justify`	`str`	Text alignment inside the entry.
`btn_text`	`str`	The browse button's label.
`**kwargs`	—	Any native `CTkFrame` argument, or a theme-key override (see Theming).

```
save_path = sCTkPathChooser(control_panel, type="directory", initialdir="/Users/you/Documents")
save_path.pack(fill="x", padx=20, pady=10)
```

Methods

Method	Returns	Description
`get()`	`str`	Current path text.
`set(path)`	`None`	Sets the displayed path, normalizing and expanding it.
`state(mode=None)` / `get_state()`	`str`	Gets or sets `"normal"` / `"disabled"`, dimming both the entry and the browse button.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard configuration, accepting `state`, `type`, `title`, `justify`, `btn_text`, `entry_height`, `btn_width` and `btn_height` as first-class properties.
`configure(name)`	`tuple`	Pygubu-style single-argument query for any of the eight properties above. **Previously broken:** the implementation read `pname = args` rather than `args[0]`, so every comparison tested a tuple against a string and failed — all eight queries fell through to the native widget and Pygubu could read none of them. The dict form of

Method	Returns	Description
		`configure()` was dead for the same reason (`isinstance(args, dict)` on a tuple is never true).

Clicking "Browse..." opens an `sCTkFileExplorer` in a modal popup; selecting a path there calls `self.set(...)` on this widget automatically.

Theming (`sCTkThemes.json`)

```
{
    "sCTkPathChooser": {
        "entry_font": ["Arial", 13],
        "entry_fg": ["#F9F9FA", "#343638"],
        "entry_border_color": ["#979DA2", "#565B5E"],
        "entry_text_color": ["#000000", "#FFFFFF"],
        "btn_font": ["Arial", 13, "bold"],
        "btn_fg": ["#3B8ED0", "#1F6AA5"],
        "btn_hover": ["#2C74B3", "#144E75"],
        "btn_text_color": ["#DCE4EE", "#F9F9FA"],
        "btn_border_color": ["#3B8ED0", "#1F6AA5"],
        "disabled_map": {
            "entry_fg": ["#EAEAEA", "#2B2B2C"],
            "entry_border_color": ["#D3D3D3", "#3A3A3C"],
            "entry_text_color": ["#A0A0A0", "#7C7C7C"],
            "btn_fg": ["#D3D3D3", "#2D2F31"],
            "btn_border_color": ["#D3D3D3", "#2D2F31"],
            "btn_text_color": ["#A0A0A0", "#5A5C5E"]
        }
    }
}
```

Every key the code references is present in both the top-level block and `disabled_map` — confirmed by direct cross-check against the actual source, nothing missing.

Every top-level key is now required and validated at construction, matching `sCTkFileExplorer` / `sCTkTableview` / `sCTkSpinbox` / `sCTkSelector` — missing any raises immediately, naming the exact key. `disabled_map` entries deliberately keep their original, more lenient behavior: gracefully falling back to the top-level/normal value if not overridden, rather than hard-failing, since that's intentional and already correct.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkPathChooser, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("450x200")
    root.title("PathChooser Example")

    base = sCTkFrame(root)
```

```
base.pack(expand=True, fill="both", padx=20, pady=20)

chooser = sCTkPathChooser(base, type="directory")
chooser.pack(fill="x", pady=10)

def toggle_disabled():
    target = "disabled" if chooser.get_state() == "normal" else "normal"
    chooser.state(target)
    toggle_btn.configure(text="Enable" if target == "disabled" else "Disable")

toggle_btn = sCTkButtonPrimary(base, text="Disable", command=toggle_disabled)
toggle_btn.pack(pady=10)

root.mainloop()
```

Known Limitations

- **No readonly support** — unlike `sCTkSpinbox`, this widget only has `"normal"` / `"disabled"`, even though the same design opportunity applies (the entry could be readonly-locked while "Browse..." stays clickable, since it's the intended alternative way to set the value). Identified as a genuine future enhancement, not implemented yet.

Return to Table of Contents

sCTkScrollArea

`sCTkScrollArea` is a scrollable viewport container built on a raw `tkinter.Canvas`, offered as an alternative to `ctk.CTkScrollableFrame` for cases where you want to supply your own external scrollbar and control child event binding explicitly. It inherits `ctk.CTkFrame` and `ScrollBindingMixin`.

Its companion is `sCTkScrollbar`. Scroll handling comes from `ScrollBindingMixin`, which is the reference for how scrolling works across this library.

Table of Contents

Constructor

```
scroll_area = sCTkScrollArea(master=None, **kwargs)
```

Parameter	Type	Description
`master`	widget	Parent container.
`**kwargs`	—	Passed to `ctk.CTkFrame`. Note `fg_color="transparent"` and `border_width=0` are set internally and can't be overridden.

Attributes

Attribute	What it is
`scroll_content`	The frame your content goes into. A raw `tkinter.Frame`, not a themed widget.
`canvas`	The raw `tkinter.Canvas` providing the scrolling viewport.

Methods

Method	Description
`hook_scrollbar(scrollbar_widget)`	Connects a scrollbar to the canvas in both directions, and registers it as a scroll layer so the wheel keeps working while the pointer is over the bar itself.
`propagate_scroll_events(target_widget)`	Registers a widget outside `scroll_content` to receive scroll events, along with its descendants. **Rarely needed now** — see below.
`process_incoming_scroll(event)`	Compatibility shim. Scroll events are dispatched by the mixin directly; this remains only for external callers that bound this method themselves.

`propagate_scroll_events()` **is no longer required for ordinary content.** Anything placed inside `scroll_content` is bound automatically and re-bound whenever it changes, so the per-item call shown in older examples is redundant. It's still useful for widgets that sit *outside* that tree.

Its behavior also changed: registered widgets are now **remembered** and re-bound on every subsequent pass. The previous implementation bound once and forgot, so any later rebind lost them.

Wiring it up

```
scroll_view = sCTkScrollArea(container)
scroll_view.pack(fill="both", expand=True)

scrollbar = sCTkScrollbar(container, orientation="vertical")
scrollbar.pack(side="right", fill="y")
```

```
scroll_view.hook_scrollbar(scrollbar)

for i in range(25):
    sCTkLabelSecondary(scroll_view.scroll_content, text=f"Row {i}").pack(anchor="w")
```

No `propagate_scroll_events()` call is needed — the rows are inside `scroll_content`, so the content rebind picks them up.

Theming

This widget is not part of the theme system. It doesn't inherit `ThemeableWidget`, doesn't read `sCTkThemes.json`, and has no theme block. An earlier version of this page showed an `sCTkScrollArea` JSON block — nothing reads it, and adding it has no effect.

The canvas and content-frame backgrounds are hardcoded: `#FAFAFA` in light mode, `#1A1A1A` in dark, switched by the widget's own `_set_appearance_mode()` hook. `scroll_content` is a raw `tkinter.Frame`, which cannot render CustomTkinter's transparent pseudo-value or `(light, dark)` tuples, so it needs a literal color.

Bringing this widget into the theme system is an open item, tied to the Pygubu Designer integration work.

Example

```
#!/usr/bin/python3
import customtkinter as ctk
from scustomtkinter import (sCTk, sCTkFrame, sCTkButtonPrimary, sCTkLabelSecondary,
                            sCTkScrollbar, sCTkScrollArea)

if __name__ == "__main__":
    root = sCTk()
    root.geometry("480x480")
    root.title("sCTkScrollArea Validation Bench")
    root.configure(fg_color=("#F1F5F9", "#1C1C1C"))

    lower_tray = sCTkFrame(root, fg_color="transparent")
    lower_tray.pack(side="bottom", fill="x", padx=15, pady=(0, 15))

    main_layout = sCTkFrame(root, border_width=2)
    main_layout.pack(expand=True, fill="both", padx=15, pady=15)

    status_monitor = sCTkLabelSecondary(main_layout, text="STATUS: viewport online")
    status_monitor.pack(fill="x", padx=10, pady=(5, 10))

    def toggle_appearance_skin():
        ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

    btn_theme = sCTkButtonPrimary(lower_tray, text="Toggle Light/Dark", command=toggle_appearance_ski
    btn_theme.pack(fill="x", expand=True, padx=5)

    scrollbar = sCTkScrollbar(main_layout, orientation="vertical")
    scrollbar.pack(side="right", fill="y", padx=(5, 10), pady=10)
```

```
content_chassis = sCTkFrame(main_layout, border_width=0, fg_color="transparent")
content_chassis.pack(side="left", fill="both", expand=True, padx=(10, 0), pady=10)

scroll_view = sCTkScrollArea(content_chassis)
scroll_view.pack(fill="both", expand=True)

for i in range(25):
    sCTkLabelSecondary(
        scroll_view.scroll_content,
        text=f"Transceiver channel {100 + i} [OK]"
    ).pack(anchor="w", padx=10, pady=4)

scroll_view.hook_scrollbar(scrollbar)

root.mainloop()
```

Known Limitations

- **Outside the theme system entirely** — see Theming. It's the only widget in this library in that position.
- **No** `_finalize_themeable_lifecycle()` **handshake,** so Pygubu Designer gets no registration signal from it.
- **No disabled state.** Unlike `sCTkScrollableFrame` and `sCTkFileExplorer`, there's no `state()` here and no way to make it inert.
- `scroll_content` **is a raw** `tkinter.Frame`**,** so widgets placed in it don't inherit CustomTkinter background propagation. Themed `sCTk` children render correctly; plain `tk` children may need their `bg` set to match.
- **The debounced rebind also runs on genuine resizes** — see the mixin page.

Behavior changed when this widget adopted the shared mixin. Three corrections, all bringing it in line with the rest of the library: Windows wheel travel halved (it previously doubled the `/120` delta), the two's-complement boundary at exactly 32768 was fixed, and the packed touchpad delta is now decoded by bit-shifting rather than reading `event.delta_y`. It also gained the nested-frame boundary guard and the automatic content rebind.

Return to Table of Contents

ScrollBindingMixin

The single shared implementation of cross-platform mouse wheel and macOS trackpad scroll handling for this library. Used by `sCTkScrollableFrame` (and therefore `sCTkTableview`), `sCTkFileExplorer`, and `sCTkScrollArea`.

This page is the reference for how scrolling works. The individual widget pages describe only their own hooks and link here.

Table of Contents

Why it exists

This logic previously existed as three independent copies, each adapted by hand from the first. They drifted, as duplicated code does:

- **The two's-complement sign correction disagreed.** `sCTkScrollableFrame` used `>= 0x8000`; `sCTkScrollArea` used `> 32768`. Those differ at exactly 32768 — the smallest *negative* value in a signed 16-bit field — which one read as –32768 and the other as +32768, inverting direction at that value.
- `sCTkScrollArea` **decoded the packed touchpad delta differently again**, reading `event.delta_y` when present and applying a 16-bit correction to what may be a 32-bit packed value, rather than bit-shifting out the signed components.
- **Windows wheel scaling disagreed:** `/120` unscaled in `sCTkScrollableFrame`, `/120 * 2` in `sCTkScrollArea` — twice the travel per notch.
- `sCTkFileExplorer` **had no touchpad accumulator at all**, scrolling on every raw event instead of gating on an accumulated threshold. Trackpad scrolling there was markedly faster and coarser than everywhere else.
- `sCTkFileExplorer` **walked only one level** into its row frame, so a row's label or icon was never bound and the wheel did nothing over them.
- **The nested-scrollable boundary guard existed in exactly one of the three.**

Every fix had to be made three times, and none of them were. Where the copies disagreed, `sCTkScrollableFrame`'s version — the maintainer-verified reference, confirmed smooth in live testing on macOS with both an Apple mouse and a trackpad — is the one that won.

Platform behavior

Three genuinely different platform models are handled:

Platform	Mechanism
Windows	`<MouseWheel>` with a `/120`-scaled delta
Linux	Discrete `<Button-4>` / `<Button-5>` events — no continuous delta exists
macOS	Its own `<MouseWheel>` scaling, **plus** a separate higher-precision `<TouchpadScroll>` synthetic event

macOS `<TouchpadScroll>` packs a two-axis delta into a single 32-bit integer — X in the high 16 bits, Y in the low 16 — each an *unsigned* field that must be converted back to signed, or every upward scroll reads as a large positive number.

Trackpad events arrive far more frequently and with far finer deltas than wheel notches, so scrolling on each one is unusably fast. Deltas accumulate and move the view only once a threshold is crossed. The accumulator resets on a direction reversal, so reversing responds immediately instead of first cancelling out what had built up.

Tuning constants

Class attributes on the mixin, so they can be overridden per subclass or per instance without any additional machinery. **These are macOS-tuned**; other platforms may want different values.

Constant	Default	Meaning
`MAC_SCROLL_SENSITIVITY`	`3`	Amplification for macOS wheel deltas, which are much smaller than Windows' `/120` steps
`MAC_SCROLL_MAX_STEP`	`5`	Ceiling on units travelled per macOS wheel event
`TOUCHPAD_ACCUMULATION_THRESHOLD`	`12.0`	Accumulated trackpad delta required before the view moves

`MAC_SCROLL_MAX_STEP` **exists because macOS reports wildly different delta magnitudes depending on hardware.** An Apple Magic Mouse sends fine-grained values near 1; a conventional wheel mouse sends a large value per detent — around 38 in live testing. Multiplying that by the sensitivity gave 114 units from a single wheel click, jumping a 100-row list end to end. The amplification is still correct for fine-grained hardware, so rather than dropping it, the result is clamped: small deltas scale normally, large ones saturate.

Resulting travel per event:

`event.delta`	Hardware	Units
0.4	Magic Mouse	3
1	Magic Mouse	3
2	Magic Mouse	5 (clamped)
38	Wheel detent	5 (clamped)

Setting `MAC_SCROLL_MAX_STEP` to 3 gives the conventional three-lines-per-notch that matches macOS defaults. Values below 3 slow the Magic Mouse too, since its fine deltas already scale to 3 before the clamp applies.

Whether these should move somewhere more discoverable than class attributes is an open question.

Activation and rebinding

Bindings are automatic and self-maintaining. No activation call is needed, and content added after a widget is placed is picked up on its own.

That reliability takes four mechanisms, each covering a gap the others don't. All are idempotent — bindings are always torn down before being rebuilt — so overlapping coverage costs nothing.

Mechanism	Covers
`<Map>` on the host	Later remaps, e.g. `pack_forget()` then re-placement
`<Map>` on `extra_map_widget`	The widget the geometry manager actually sees
`after_idle()` at construction	Initial activation, independent of mapping semantics
`<Configure>` on `content_widget`, debounced	Content added *after* activation

Why `<Map>` alone isn't enough. `CTkScrollableFrame` is not the widget that gets placed: it builds an internal `_parent_frame` plus a canvas, inserts *itself* into that canvas via `create_window()`, and overrides `pack()` / `grid()` / `place()` to operate on `_parent_frame`. The widget is therefore a canvas-window child and may never receive `<Map>` the way an ordinarily-managed widget does. `after_idle()` is what actually establishes bindings in practice.

Why the content rebind is needed. Activation happens once, at a moment when the container is usually still empty — callers construct, place, and *then* populate. Confirmed by live testing: an `sCTkTableview` bound at activation time collected 16 layers (frame, canvas, header cells) because `load_dataset()` hadn't run yet; the 32 data cells created afterwards were never bound, so it scrolled beside its rows but not over them.

`<Configure>` fires when children change the container's layout, so it catches every content-adding path. It's debounced through `after_idle` because building a table fires it once per cell — one rebind instead of 32, run after the burst rather than during it, so it sees the finished tree.

`<Configure>` can't distinguish "children were added" from "the window was dragged", so **resizing rebinds too**. Coalesced, but not free on a very large content tree.

> **Do not replace `tk.Misc.bind(self, ...)` with `self.bind(...)`.** CustomTkinter overrides `CTkScrollableFrame.bind()` to forward every binding to `self._parent_canvas` instead of attaching it to the widget. An earlier version used `self.bind()`, and bindings never landed on the frame — scroll handling was silently never installed, and widgets only appeared to scroll where native CustomTkinter's own global `bind_all` handler happened to cover for it. `tk.Misc.bind` called unbound reaches the real Tkinter implementation. This looks like a needless complication and is not.

Disabling scroll

When a host's `_scroll_permitted()` returns `False`, the mixin doesn't merely unbind — it installs **blocking** handlers. Two separate mechanisms are involved, because neither alone is sufficient.

Wheel and trackpad events. Unbinding is not enough: native `CTkScrollableFrame.__init__` installs its own application-global `bind_all("<MouseWheel>")` handler that survives any `unbind()`. Calling `unbind_all()` would disable scrolling for *every other* scrollable widget in the application. Instead a handler returning `"break"` is installed on each layer. Tk dispatches bindings by bindtag in order — widget, class, toplevel, then `all` — and `bind_all` lands on that final tag, so a widget-level `"break"` halts the chain before the global handler is reached. Confirmed by live testing: with two independent scrollable frames side by side, disabling one left the other scrolling normally with both a mouse wheel and a trackpad.

Scrollbar dragging. `unbind()` is actively dangerous here — Tk's `unbind()` removes *every* binding for an event on a widget, so calling it on a scrollbar's `<Button-1>` would destroy CustomTkinter's own drag handler permanently, with no way to restore it. Binding a blocker with `add="+"` doesn't work either: handlers fire in the order added, and CustomTkinter's was added during its own construction, so `"break"` at that point is too late. A private, per-instance bindtag is inserted at the **front** of the widget's tag list instead, so the blocker runs before CustomTkinter's bindings. Re-enabling removes the tag; CustomTkinter's bindings are never modified.

The tag name embeds `id(self)`, so disabling one host has no effect on any other in the same application.

The scrollbar stays visible when blocked, just inert. CustomTkinter's scrollbar has no native disabled state to lock, so there's no greyed-out appearance to switch to.

Nested scrollable frames

The descendant walk stops at any nested `CTkScrollableFrame` boundary — covering `sCTkScrollableFrame` and anything built on it, such as `sCTkSelector` and `sCTkTableview`. Without this, an inner scrollable frame placed inside an outer one would have its canvas, scrollbar, and entire content tree bound to the *outer* host's handler as well as its own, and since bindings use `add="+"`, both fire on the same event and scroll both at once. Native CustomTkinter guards the same boundary in its own `_check_if_valid_scroll`.

The guard applies to descendants only, so a scrollable host still binds its own layers.

Not yet live-tested. The logic mirrors CustomTkinter's own guard and is straightforward, but an actual nested case hasn't been exercised against it.

A separate scrolling region built directly on a plain `Canvas` is **not** guarded — the check keys on `CTkScrollableFrame` specifically. Guarding that would need an explicit opt-out convention, since a plain `Canvas` has no way to declare itself an independent scroll region.

Host contract

A host class must implement two methods and may override two more:

Method	Required	Returns
`_scroll_target()`	yes	The widget to call `yview_scroll()` on, or `None` if scrolling isn't currently possible
`_scroll_layers()`	yes	The ordered, deduplicated list of widgets to bind
`_scroll_permitted()`	no	`False` to install blocking handlers instead of scroll handlers. Default `True`
`_scroll_drag_targets()`	no	Widgets whose click-drag should also be blocked when not permitted. Default none

Hosts call two setup methods from `__init__`:

```
self._init_scroll_state()                    # must run before any binding
self._install_scroll_activation(
    extra_map_widget=...,                    # optional
    content_widget=...,                      # optional
)
```

`content_widget` **matters.** It defaults to `self`, which is correct only when content is added directly to the host. A host that puts content in a separate inner frame **must** pass that frame — adding rows to an inner frame doesn't resize the outer widget, so `<Configure>` on `self` would never fire and the content rebind would silently never happen.

Hosts may also define `_USE_CUSTOM_SCROLL_BINDING = False` as a kill switch, falling back to whatever native CustomTkinter provides. It's checked inside `_toggle_scroll_bindings()` rather than at the call sites, so it can't be bypassed by reaching that method through a different entry point — the exact bug that made the toggle ineffective in an earlier revision.

Host implementations

Host	`_scroll_target()`	Notes
`sCTkScrollableFrame`	Parent canvas via `winfo_parent()`	Wrapped by a native `CTkScrollableFrame` that owns the canvas. Passes `_parent_frame` as `extra_map_widget`. Disabling stops scrolling.
`sCTkFileExplorer`	`self.canvas`	Builds its own canvas, so no lookup needed. Passes `explorer_frame` as `content_widget`. Disabling stops scrolling.
`sCTkScrollArea`	`self.canvas`	Builds its own canvas. Passes `scroll_content` as `content_widget`. No disabled state.

Return to Table of Contents

sCTkSelector

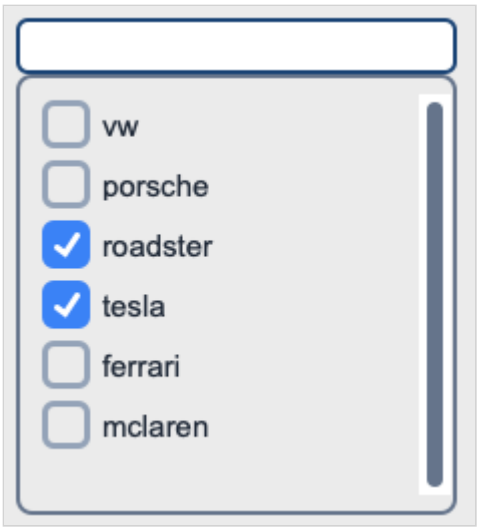
(Derived from Separator class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: MoreCustomTkinterWidgets)

Table of Contents

Overview

`sCTkSelector` is a theme-compliant, scrollable multi-select (or single-select) list of checkboxes, with an optional live-filtering search field. It's built by composing a themed frame, an `sCTkScrollableFrame` for the checkbox list, and one `sCTkCheckBox` per item — not by subclassing a single native CustomTkinter widget.

Dark Mode: 

Light Mode: 

This widget inherits `sCTkFrame` directly (rather than raw `ctk.CTkFrame`) — a composition pattern that previously carried a real risk of `ThemeableWidget.__init__` running twice per instance and silently corrupting this widget's own resolved theme data. That risk is now fully closed: `ThemeableWidget` has a run-once guard preventing the double-init, and `sCTkFrame` itself filters its inbound kwargs down to only what native `CTkFrame` actually accepts before its own constructor call. Neither fix required any change to this widget.

Constructor

```
sCTkSelector(master, items=None, multiple_choices=True, searchBox=True, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	—	Parent container.
`items`	`list[str]`	`None`	Initial list of checkbox labels. Must not contain duplicates — raises `ValueError` if it does.
`multiple_choices`	`bool`	`True`	If `False`, selecting one item automatically deselects any other currently-selected item.
`searchBox`	`bool`	`True`	Whether the live-filtering search field is shown above the checkbox list.
`**kwargs`	—	—	Any native `CTkFrame` argument, or an override for one of the theme keys listed under Theming.

```
channel_selector = sCTkSelector(control_panel, items=["Ch 1", "Ch 2", "Ch 3"], multiple_choices=False
channel_selector.pack(expand=True, fill="both", padx=20, pady=20)
```

Methods

Method	Returns	Description
`get_all_items()`	`list[str]`	Every checkbox's label text, regardless of current search filter or selection state.
`state(mode=None)`	`str`	Gets or sets the widget's visual state. `"disabled"` locks and dims every checkbox and the search field (routed to the search field's own `"readonly"`, not `"disabled"` — see Known Limitations); anything in `("normal", "enabled", "active")` re-enables both.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard widget configuration, plus `items`, `multiple_choices`, `searchBox`, `pack_propagate`, `grid_propagate`, and `state` are all handled as first-class properties, matching the constructor. Calling `configure("propname")` with a single property name returns a Tkinter-style query tuple for `state`, `multiple_choices`, `searchBox`, `items`, `pack_propagate`, `grid_propagate`, and `fg_color` / `border_color` / `text_color`.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — `fg_color` and `corner_radius` control the outer frame; the checkbox-related keys and `border_color` are validated and applied once at construction time (not repeatedly on every state change).
- **Re-applied on every `state()` change** — the checkbox colors below are recomputed from normal values or `disabled_map` every time you call `state()`.

```
{
    "sCTkSelector": {
        "fg_color": "transparent",
        "corner_radius": 6,
        "text_color": ["#1F2937", "#F9FAFB"],
        "checkbox_fg_color": ["#1A4375", "#1F6AA5"],
        "checkbox_hover_color": ["#112A4B", "#1A5885"],
        "border_color": ["#94A3B8", "#4B5563"],
        "checkmark_color": ["#FFFFFF", "#FFFFFF"],
        "disabled_map": {
            "text_color": ["#808080", "#666666"],
```

```
            "checkbox_fg_color": ["#CBD5E1", "#475569"],
            "border_color": ["#CBD5E1", "#334155"],
            "checkmark_color": ["#F1F5F9", "#94A3B8"]
        }
    }
}
```

`checkbox_fg_color` / `checkbox_hover_color` **are dedicated keys, not reused from** `fg_color`**.** An earlier version derived each checkbox's accent color from this widget's own `fg_color` — the same key that controls the outer frame's background — falling back to a hardcoded generic blue whenever `fg_color` was `"transparent"` (a common, legitimate choice for a frame, not a theme gap). Reusing one key for two different visual purposes didn't work well; dedicated keys fix that cleanly. All five top-level keys (`text_color` , `checkbox_fg_color` , `checkbox_hover_color` , `border_color` , `checkmark_color`) and four `disabled_map` keys (all but `checkbox_hover_color` — disabled checkboxes reuse `checkbox_fg_color` for hover too, since hover can't meaningfully trigger while disabled) are required; missing any raises immediately at construction.

`border_color` **is also shared with this widget's two internal sub-widgets** (the search field and the checkbox-list frame), passed in once at construction so their *normal*-state border visually matches this widget's own border — confirmed by direct testing that these two sub-widgets' own independent default themes can otherwise visibly mismatch, especially in dark mode. This only establishes the shared normal-state value; each sub-widget's own state-driven color changes (the search field's readonly/disabled coloring in particular) are left completely untouched afterward.

Every color is passed through as a raw `(light, dark)` tuple, letting CustomTkinter's native appearance-mode tracking handle repaints automatically, consistent with the approach used throughout this project.

Example

```python
from scustomtkinter import sCTk, sCTkFrame, sCTkSelector, sCTkButtonPrimary, sCTkLabelPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x420")
    root.title("Selector Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    selector = sCTkSelector(base, items=[f"Item {i}" for i in range(1, 21)])
    selector.pack(expand=True, fill="both", pady=10)

    status = sCTkLabelPrimary(base, text=f"state: {selector.get_state()}")
    status.pack(pady=5)

    def toggle_disabled():
        target = "disabled" if selector.get_state() == "normal" else "normal"
        selector.state(target)
        status.configure(text=f"state: {selector.get_state()}")
        toggle_btn.configure(text="Enable" if target == "disabled" else "Disable")

    toggle_btn = sCTkButtonPrimary(base, text="Disable", command=toggle_disabled)
    toggle_btn.pack(pady=10)
```

```
root.mainloop()
```

Known Limitations

- **Disabling this widget routes the search field to `"readonly"`, not `"disabled"`** — deliberate, so its text remains selectable/copyable, but worth knowing if you expected a uniform `"disabled"` state across every sub-component.
- Calling `configure("fg_color")` (or similar) returns `str(value)` where `value` may itself be a `(light, dark)` tuple rather than a single resolved color. Known gap shared with the wider Pygubu single-argument query investigation set aside elsewhere in this project.
- `.config()` **previously bypassed this widget entirely.** Tkinter binds `.config` to `.configure` as a separate class attribute rather than tracking a subclass's override, and this class had no `config = configure` line — so `.config(...)` skipped the `items` / `searchBox` / `multiple_choices` / `state` handling and landed on `sCTkFrame`'s `configure()` instead. Fixed. Note this widget uses the older `(self, cnf=None, **kwargs)` signature rather than `*args`; that's correct here and is *not* the shape that caused the tuple-comparison bugs found elsewhere in the library, since `cnf` is a real parameter holding the value itself.
- `items` must not contain duplicate labels — `configure(items=[...])` raises `ValueError` if it does, since selection tracking is index-based and duplicate labels would make search filtering ambiguous.

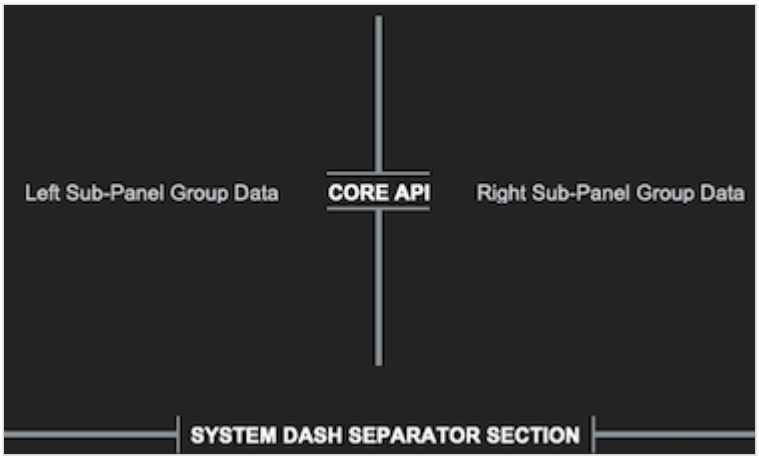
Return to Table of Contents

sCTkSeparator

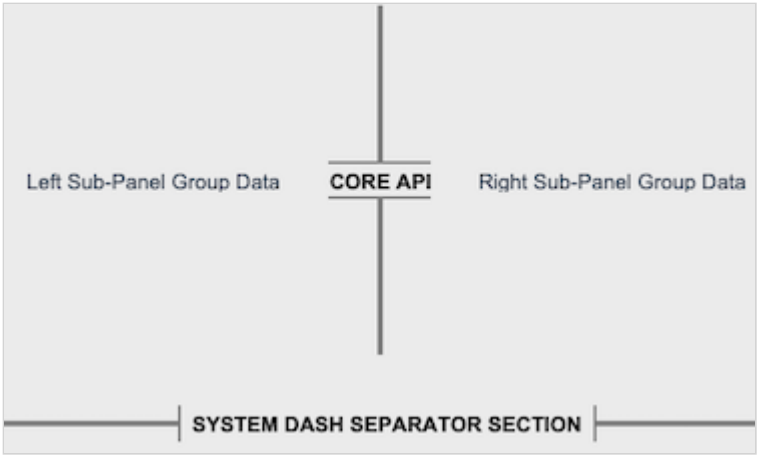
(Derived from Separator class by Fastattack, 2024. This widget was made available to the community via the MIT License. Source Repository: MoreCustomTkinterWidgets)

Table of Contents

The *sCTkSeparator* is an advanced, themeable divider widget for CustomTkinter. It provides dynamic scaling via layout managers, vector-drawn customizable corner radiuses, dashed/dotted line styles, and automated line-splitting centered section text headers with bounding capsule brackets.

Dark Mode: 

Light Mode:



System Architecture Overview

The component functions as a structural vector drawing lane subclassed from `ctk.CTkBaseClass`. Rather than forcing a static line width or texture file, it wraps a native Tkinter canvas object to paint partitions programmatically.

The visual update matrix implements two important enhancements:

1. **Dynamic Layout Dimension Adapters**: To prevent text characters from clipping, the instantiation block monitors initialization properties. If text section banners are provided, the widget automatically stretches its bounding frame vertical or horizontal thickness out to `28px` to give text canvas regions clear physical space while leaving the split line itself perfectly thin.
2. **Skin Preference Broadcaster Interceptor**: Features an explicit `_set_appearance_mode` connection loop. This forces text header strings, dashed patterns, and line fills to recalculate active palettes instantly during global theme swaps without any pixel lag.

API Property Reference

Property Name	Data Type	Default Value	Description
`master`	`any`	*Required*	Parent container instance (e.g., `sCTkFrame` or `ctk.CTk`).

Property Name	Data Type	Default Value	Description
`length`	`int`	`100`	The total span length of the line track in pixels (corresponds to widget height if vertical, width if horizontal).
`width`	`float`	`4`	The visual thickness profile of the divider line in pixels.
`corner_radius`	`int` or `None`	`6` (from theme)	Defines roundness sharpness of divider line endpoints (defaults to stylesheet configuration).
`orientation`	`str`	`"vertical"`	Sets spatial directional positioning alignment. Accepts `"vertical"` or `"horizontal"`.
`text`	`str`	`""`	Appends a centered section header label text directly inside a computed line split zone.
`font`	`tuple` or `CTkFont`	`("Arial", 11, "bold")`	Text font profile style parameters for the embedded header tag.
`text_color`	`str` or `Tuple[str, str]`	Central theme default	Font hex palette token string mapping. Supports appearance mode tuples.
`dash`	`tuple` or `None`	`None`	Integer stroke sequence array tuple mapping out dashed/dotted rendering rules (e.g., `(5, 5)`).

State

Method	Description
`state(mode=None)`	Getter with no argument; setter with `"normal"` or `"disabled"`.
`get_state()`	Equivalent to `state()` with no argument.
`configure(state=...)`	Same effect. Both routes are supported.
`cget("state")`	Reads the current state.
`configure("state")`	Pygubu-style single-argument query.

A separator has nothing to interact with, so disabling only repaints it from `disabled_map` — the line and any header text dim together. It exists so a panel can disable every widget it contains uniformly.

Centralized Stylesheet Setup (`sCTkThemes.json`)

The component queries your centralized theme sheet profile matrix using standard `self._resolve_color()` lookup calls, ensuring that indicator dots and canvas borders translate colors smoothly across appearance updates.

To satisfy the framework configuration guidelines, ensure your theme matrix includes this structured asset block:

```
{
    "sCTkSeparator": {
        "fg_color": ["#808080", "#8A9296"],
        "bg_color": "transparent",
        "corner_radius": 6,
        "font": ["Arial", 11, "bold"],
        "text_color": ["#1A1A1A", "#FFFFFF"],
        "disabled_map": {
            "fg_color": ["#CBD5E1", "#475569"],
            "text_color": ["#94A3B8", "#64748B"]
        }
    }
}
```

Every key above is required, including `disabled_map`. Construction raises `KeyError` naming the missing key and whether it belongs at the top level or in `disabled_map`.

This matters because the disabled colours were previously unreachable. `_draw()` read them with hardcoded fallbacks — `.get("fg_color", ["#CBD5E1", "#475569"])` and `.get("text_color", ["#94A3B8", "gray50"])` — and since the theme block had no `disabled_map` at all, those fallbacks were **always** taken. A disabled separator never used the configured theme. The values shown above are those same fallbacks promoted into the theme file, so the appearance is unchanged; the one exception is dark-mode disabled text, where the Tk colour name `gray50` is replaced by `#64748B`, matching the disabled text colour used across the rest of the library.

`font` and `corner_radius` lost their fallbacks for the same reason. `text_color` additionally used to fall back to `ctk.ThemeManager.theme["CTkLabel"]["text_color"]` — borrowing another widget class's colour, which would now only mask a theme gap.

Structural parameters are not required in the theme. `orientation`, `length`, `width`, `text` and `dash` are read from the resolved keywords, so the theme *can* supply them, but they are constructor arguments with sensible defaults and requiring them would push layout decisions into the stylesheet.

Layout Manager Integration

Mixing layout manager tracking loops within the same immediate frame layer is completely blocked. When handling automated expansion parameters across scaling monitor resolutions, enforce the following geometry behaviors:

Grid Configurations (`.grid()`)

- **Horizontal Mode Line**: Must use `sticky="ew"` to allow the vector path to grow horizontally.
- **Vertical Mode Line**: Must use `sticky="nswe"` to stretch across columns and rows evenly without crushing string lines.
- **Parent Frame Setup**: The container frame track columns/rows **must** have their weights configured to let the engine allocate expanding window real estate:

```
# Column 0 and Column 2 hold widgets and expand; Column 1 isolates the separator line track
grid_Frame.grid_columnconfigure(0, weight=1)
grid_Frame.grid_columnconfigure(1, weight=1)
grid_Frame.grid_columnconfigure(2, weight=1)
```

Pack Configurations (`.pack()`)

- **Horizontal Mode Line**: Must use `fill="x"` alongside `expand=False` so it hugs adjacent frames tightly instead of expanding into empty background rows.
- **Vertical Mode Line**: Must use `fill="y"` inside layout columns.

Pygubu Designer Properties Guide

When configuring layouts visually within the Pygubu Designer editing workspace panel strip, observe these property formatting rules:

1. `orientation` : Select `vertical` or `horizontal` from the choice dropdown list pane. The preview canvas will immediately adjust orientations without flattening.
2. `text` : Type any section title banner sequence string directly into the entry field (e.g., `AUDIO CONTROLS`). The line will cleanly break around the text boundaries.
3. `dash` : Enter raw comma-separated lists of numerical values directly into the input strip **without using quote symbols or brackets**.
    - Type `5,5` for standard clean dash blocks.
    - Type `2,6` for clean dotted layout maps.
    - Leave blank or type `None` to restore solid rounded vector shapes.

Event Binding

`bind()` and `unbind()` are overridden to route to the internal canvas, which is what actually receives events — `CTkBaseClass` filters direct binds on the widget itself.

Both previously discarded an argument, and both were fixed:

- `bind()` **accepted** `add` **and ignored it,** hardcoding `add=True` in the forwarded call. A caller passing `add=False` to *replace* an existing binding would silently accumulate one instead.
- `unbind()` **accepted** `funcid` **and discarded it,** so it removed *every* binding for that sequence rather than the one identified. This is the same destructive behaviour that made `unbind()` unusable for blocking scrollbar drags in `sCTkScrollableFrame` : Tk's `unbind()` with no `funcid` wipes bindings this widget never installed, with no way to restore them.

If existing code depended on the old behaviour it will change — though neither method did what its signature promised.

Implementation Example & Test Harness

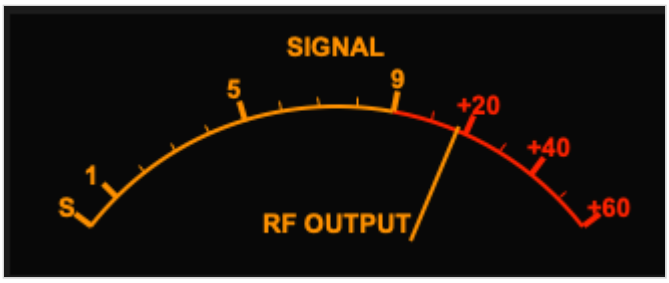
Below is a complete, self-contained test execution script demonstrating how to layout horizontal, vertical, and dashed separators inside an interactive telemetry deck panel while exercising lock states and skin sweeps.

```
#!/usr/bin/python3
# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for Separator
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTkLabelSecondary, sCTk, sCTkSeparator

if __name__ == "__main__":
    root = sCTk()
    root.title("sCTkSeparator Production Test Environment")
    root.geometry("600x450")

    grid_Frame = sCTkFrame(root)
    grid_Frame.pack(side="top", fill="both", expand=True, padx=20, pady=15)
    grid_Frame.grid_columnconfigure(0, weight=1); grid_Frame.grid_columnconfigure(1, weight=1); grid_

    lbl_left = sCTkLabelSecondary(grid_Frame, text="Left Sub-Panel Group Data")
    lbl_left.grid(row=0, column=0, sticky="nswe")

    sep_vertical_text = sCTkSeparator(grid_Frame, orientation="vertical", text="CORE API", width=4)
    sep_vertical_text.grid(row=0, column=1, sticky="nswe", padx=10, pady=10)

    lbl_right = sCTkLabelSecondary(grid_Frame, text="Right Sub-Panel Group Data")
    lbl_right.grid(row=0, column=2, sticky="nswe")

    sep_horizontal_text = sCTkSeparator(root, orientation="horizontal", text="SYSTEM DASH SEPARATOR SI
    sep_horizontal_text.pack(side="top", fill="x", padx=20, pady=10)

    def toggle_separator_lock():
        target = "disabled" if sep_vertical_text.get_state() == "normal" else "normal"
        sep_vertical_text.configure(state=target)
        sep_horizontal_text.configure(state=target)
        btn_lock.configure(text="Lock Separators" if target == "normal" else "Unlock Separators")

    def toggle_skin_mode():
        current_skin = ctk.get_appearance_mode()
        ctk.set_appearance_mode("Light" if current_skin == "Dark" else "Dark")

    btn_lock = sCTkButtonPrimary(root, text="Lock Separators", command=toggle_separator_lock)
    btn_lock.pack(pady=5)
    btn_theme = sCTkButtonPrimary(root, text="Simulate Global Theme Shift", command=toggle_skin_mode)
    btn_theme.pack(pady=(5, 20))

    root.mainloop()
```

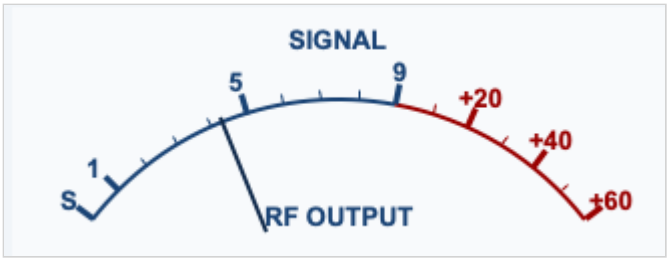
Return to Table of Contents

sCTkSMeter

The `sCTkSMeter` is a standalone, theme-adaptive analog S-Meter/Power Output gauge instrument designed specifically for ham radio transceiver desktop interfaces. Natively inheriting container footprints from `customtkinter.CTkFrame`, it delivers smooth telemetry tracking sweeps without the overhead of extraneous nesting modules.

Dark Mode: 

Light Mode:



🛠️ Core Gauge Geometry & Scale Mechanics

The instrument face is split mathematically to mirror classic analog transceiver gauge divisions perfectly:

- **The S-Unit Scale (Ticks 0–9):** Maps incoming telemetry values from `0.0` to `9.0` linearly across the first 60% of the visual arc container, rendered in your high-contrast brand or amber theme palettes.
- **The Decibel Over S9 Scale (Ticks 9–15):** Maps advanced signal parameters from `9.0` up to `69.0` across the remaining 40% of the dial arc track (where `+20dB` sits at coordinate 29, `+40dB` at 49, and `+60dB` at 69). This region is permanently framed by your crimson/redline alert warning colors.
- **Unified Pivot Axis Integration:** The inner rendering engines calculate lines, arcs, labels, and needle sweeps using a singular synchronized mathematical pivot point (`center_x = width * 0.48`). This entirely eliminates off-axis tracking drift or floating pointer artifacts when live data streams update.

📋 API Constructor Reference

```
sCTkSMeter(master=None, width=250, height=130, state="normal", **kw)
```

Parameter Name	Data Type	Default Value	Description
`master`	`any`	`None`	Reference pointer tracking your root window or parent `sCTkFrame` container layout layer.
`width`	`int`	`250`	Panel width in pixels. Supports Pygubu geometry-default reset queries.

Parameter Name	Data Type	Default Value	Description
`height`	`int`	`130`	Panel height in pixels. Supports Pygubu geometry-default reset queries.
`state`	`str`	`"normal"`	`"normal"` or `"disabled"`. See State below.

⚡ Global Object Instance Methods

To drive the needle tracking sweep fluidly inside background receiver threads, automatic VFO frequency scanning loops, or telemetry data parsing hooks, utilize this direct public setter:

Update Instrument Needle Position

```
# Updates pointer positioning dynamically. Expects a float value clamped between 0.0 and 69.0.
smeter.set(value)
```

State

Method	Description
`state(mode=None)`	Getter with no argument; setter with `"normal"` or `"disabled"`.
`get_state()`	Equivalent to `state()` with no argument.
`configure(state=...)`	Same effect. Both routes are supported.
`cget("state")`	Reads the current state.
`configure("state")`	Pygubu-style single-argument query.

Disabling dims, it does not freeze. `state("disabled")` changes only the palette. `set()` continues to update the needle, and the gauge keeps tracking live values in the dimmed colours. This is deliberate for an output-only instrument: a meter that held its last reading while greyed out would be indistinguishable from one showing a current value, which on a radio panel is actively misleading. There is no input to lock out — the state exists so a panel can disable every widget it contains uniformly.

The background is deliberately **not** dimmed; the face and needle carry the signal, matching `sCTkScrollableFrame` and the dial family.

🎨 Centralized Stylesheet Integration (`sCTkThemes.json`)

```
{
    "sCTkSMeter": {
        "fg_color": ["#F4F7FA", "#0A0A0A"],
```

```
        "text_color": ["#1A4375", "#FF9100"],
        "alarm_color": ["#990000", "#FF2200"],
        "needle_color": ["#112A4B", "#FF9100"],
        "font": ["Arial", 10, "bold"],
        "scale_font": ["Arial", 10, "bold"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#4B5563"],
            "alarm_color": ["#CBD5E1", "#4B5563"],
            "needle_color": ["#94A3B8", "#4B5563"]
        }
    }
}
```

Every key above is required. Construction raises `KeyError` naming the missing key and whether it belongs at the top level or in `disabled_map`. This replaced a pattern of `.get(key, ("#hex", "#hex"))` throughout the draw code, which silently substituted a plausible guess and made an incomplete theme block look merely slightly-off rather than broken.

`font` is used for the "SIGNAL" and "RF OUTPUT" captions; `scale_font` for the numeric tick labels and the "S" marker. They're separate keys because the widget makes that distinction, even though the default values happen to match.

> **Font size has layout consequences.** Label positions are computed from fixed pixel offsets tuned for 10pt text. A noticeably larger font will overlap the tick marks and the arc — the widget does not measure text and adjust. Change these values in small steps and look at the result.

Fixed: the configured `fg_color` never actually rendered. It was popped out of the resolved defaults in the constructor (correctly — the native frame takes it separately) and then read back afterwards from the dictionary it had been removed from, so the background always fell through to a hardcoded value. Light mode is where this was visible.

Implementation Example & Test Harness

Below is a complete, self-contained interactive test execution script demonstrating how to use `sCTkSMeter`.

```
#!/usr/bin/python3
# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for S Meter
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkSMeter
import random


if __name__ == "__main__":

    root = sCTk()
    root.title("sCTk Standalone Analog Gauge")
    root.geometry("450x260")
    root.configure(fg_color=("#F1F5F9", "#1C1C1C"))
```

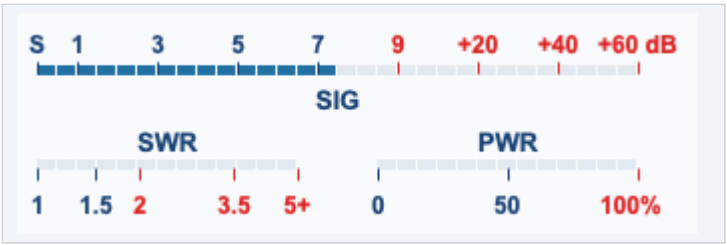
```
dashboard_frame = sCTkFrame(root, fg_color="transparent", border_width=0)
dashboard_frame.pack(padx=20, pady=20)

smeter = sCTkSMeter(dashboard_frame, width=340, height=130)
smeter.pack(padx=10, pady=10)


class SignalSimulator:
    def __init__(self, root_win, meter):
        self.root, self.meter = root_win, meter
        self.target, self.needle = 6.0, 0.0

    def shift_vfo(self):
        self.target = random.uniform(1.5, 65.0)
        self.root.after(random.randint(2500, 5000), self.shift_vfo)

    def physics_loop(self):
        jitter = random.uniform(-1.5, 1.5)
        sig = max(0.0, min(69.0, self.target + jitter))
        self.needle += (sig - self.needle) * 0.25
        self.meter.set(self.needle)
        self.root.after(25, self.physics_loop)


sim = SignalSimulator(root, smeter)
sim.physics_loop()
sim.shift_vfo()


def toggle_theme():
    ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")


sCTkButtonPrimary(root, text="Toggle Theme mode", command=toggle_theme).pack(pady=5)
root.mainloop()
```

sCTkSMeterBar

The `sCTkSMeterBar` is a standalone, low-profile horizontal discrete 30-segment LED bar instrumentation widget displaying independent telemetry tracks for incoming receiver S-Units, transmitter SWR ratio levels, and forward RF Power output percentage. Like all sCTk widgets, it is fully theme-adaptive.

Dark Mode: 

Light Mode:



🛠️ Subsystem Layout & Multi-Track Physics

The discrete LED matrix map shifts automatically based on the device operational path constraints:

- **The S-Meter Track (Top Row):** Maps incoming telemetry values across 30 linear segments. Signals from `0.0` to `9.0` utilize the first 60% of the bar, while advanced signal ranges up to `+60dB` expand into the remaining 40% redline warning zone.
- **The Transmitter Track (Split Bottom Row):** Splices the lower segment path down the center into two separate monitoring zones. The left half maps a logarithmic SWR reflection track up to your custom `swr_max_value`, while the right half tracks forward RF power from `0%` to `100%`.
- **Post-Boot Geometry Flattening:** Overrides native internal grid layout constraints programmatically to force all 30 LED rectangles to sit perfectly flush. This completely removes horizontal spacing holes, keeping your panel elements locked into a solid hardware console bar.

📋 API Constructor Reference

```
sCTkSMeterBar(master=None, swr_max_value=5.0, swr_visible=True, pwr_visible=True,
              hide_lower_row=False, width=320, height=110, state="normal", **kw)
```

Parameter Name	Data Type	Default Value	Description
`master`	`any`	`None`	Reference pointer tracking your root window or parent `sCTkFrame` container layout layer.
`swr_max_value`	`float`	`5.0`	The explicit maximum scale boundary representing the far right edge limit tracking your transmitter's SWR track.
`swr_visible`	`bool`	`True`	Visibility flag for the SWR cluster. `False` greys its text, ticks and LEDs to `inactive_color`. Distinct from widget state — see State.

Parameter Name	Data Type	Default Value	Description
`pwr_visible`	`bool`	`True`	Visibility flag for the PWR cluster. `False` greys its text, ticks and LEDs to `inactive_color`. Distinct from widget state — see State.
`hide_lower_row`	`bool`	`False`	Layout override command. When `True`, the entire lower instrumentation cluster collapses and vanishes, pushing the `SIG` bar to the true vertical center of the card footprint.
`width`	`int`	`320`	Panel width in pixels. Supports Pygubu geometry-default reset queries.
`height`	`int`	`110`	Panel height in pixels. Supports Pygubu geometry-default reset queries.
`state`	`str`	`"normal"`	`"normal"` or `"disabled"`. See State below.

⚡ Global Object Instance Methods

Update Instrument Telemetry Channels

```
# Pass parameters to update any of the 3 telemetry rows independently on the fly.
# Expects floats matching your radio data streams.
led_bar_gauge.set(s_value=9.2, swr_value=1.4, pwr_value=45.0)
```

Live Layout Configuration Modifier

```
# Updates layout presentation properties on the fly without reconstruction overhead.
led_bar_gauge.configure_visibility(swr_visible=False, pwr_visible=True, hide_lower_row=False)
```

State

Method	Description
`state(mode=None)`	Getter with no argument; setter with `"normal"` or `"disabled"`.
`get_state()`	Equivalent to `state()` with no argument.
`configure(state=...)`	Same effect. Both routes are supported.
`cget("state")`	Reads the current state.
`configure("state")`	Pygubu-style single-argument query.

Disabling dims, it does not freeze. `state("disabled")` changes only the palette. `set()` continues to update all three telemetry rows, and the bar keeps tracking live values in the dimmed colours. This is deliberate for an output-only instrument: a meter that held its last reading while greyed out would be indistinguishable from one showing a current value, which on a radio panel is actively misleading. There is no input to lock out — the state exists so a panel can disable every widget it contains uniformly.

State and row visibility are independent. `state("disabled")` dims the whole widget via `disabled_map`. `configure_visibility(swr_visible=False)` greys just that cluster to `inactive_color`. A row can be hidden on an enabled widget, and a disabled widget still shows whichever rows are visible. Both can apply at once.

The background is deliberately **not** dimmed; the LEDs and labels carry the signal.

🎨 Centralized Stylesheet Integration (`sCTkThemes.json`)

```
{
    "sCTkSMeterBar": {
        "fg_color": ["#FFFFFF", "#0A0A0A"],
        "text_color": ["#1A4375", "#FF9100"],
        "alarm_color": ["#DC2626", "#FF2200"],
        "led_on_color": ["#2471A3", "#FF9100"],
        "led_off_color": ["#E2E8F0", "#1A1D20"],
        "inactive_color": ["#94A3B8", "#334155"],
        "font": ["Arial", 10, "bold"],
        "scale_font": ["Arial", 9, "bold"],
        "disabled_map": {
            "text_color": ["#94A3B8", "#4B5563"],
            "alarm_color": ["#CBD5E1", "#4B5563"],
            "led_on_color": ["#CBD5E1", "#374151"],
            "led_off_color": ["#F1F5F9", "#1A1D20"]
        }
    }
}
```

Every key above is required. Construction raises `KeyError` naming the missing key and whether it belongs at the top level or in `disabled_map`. This replaced a pattern of `.get(key, ("#hex", "#hex"))` throughout the draw code, which silently substituted a plausible guess and made an incomplete theme block look merely slightly-off rather than broken.

`inactive_color` greys the SWR or PWR cluster when that row is switched off via `configure_visibility()`. It has no `disabled_map` entry because it is not a state colour — see State. This value was previously hardcoded in the draw routine with no theme lookup at all, the only colour in this widget the theme could not reach.

`font` is used for the "SIG", "SWR" and "PWR" section labels; `scale_font` for the numeric scale markings — S units, SWR values and power percentages. These were previously hardcoded across eight separate `create_text` calls and never consulted the theme.

> **Font size has layout consequences.** Label positions are computed from fixed pixel offsets tuned for 9pt and 10pt text. A noticeably larger font will overlap the tick marks and the LED rows — the widget does not measure text and adjust. Change these values in small steps and look at the result.

Fixed: the configured `fg_color` never actually rendered. It was popped out of the resolved defaults in the constructor (correctly — the native frame takes it separately) and then read back afterwards from the dictionary it had been removed from, so the background always fell through to a hardcoded value. Light mode is where this was visible.

Implementation Example & Test Harness

Below is a complete, self-contained interactive test execution script demonstrating how to use `sCTkSMeterBar`.

```
#!/usr/bin/python3
# ======================================================================
# 🛠️ TESTING HARNESS IMPORTS & SETUP for S Meter Bar
# ======================================================================

import customtkinter as ctk
from scustomtkinter import sCTkFrame, sCTkButtonPrimary, sCTk, sCTkSMeterBar
import random


if __name__ == "__main__":

    app = sCTk()
    app.title("sCTk Bar Instrument Test Harness")
    app.geometry("440x240")
    app.configure(fg_color=("#F1F5F9", "#1C1C1C"))

    panel_container = sCTkFrame(app, fg_color="transparent", border_width=0)
    panel_container.pack(padx=20, pady=15, fill="both", expand=True)

    led_bar_gauge = sCTkSMeterBar(panel_container, width=340, height=110)
    led_bar_gauge.pack()

    class HarnessSimulator:
        def __init__(self, root_win, bar):
            self.root, self.bar = root_win, bar
            self.s_target, self.s_curr = 4.0, 0.0
            self.swr_target, self.pwr_target = 1.0, 0.0
            self.swr_curr, self.pwr_curr = 1.0, 0.0
            self.tx_active = False

        def tuning_cycle(self):
            self.s_target = random.uniform(0.5, 13.5)
            if not self.tx_active and random.random() > 0.4:
                self.tx_active = True
                self.swr_target = random.uniform(1.1, 4.5)
                self.pwr_target = random.uniform(35.0, 95.0)
                self.root.after(random.randint(1500, 3000), self._release)
            self.root.after(random.randint(4000, 8000), self.tuning_cycle)

        def _release(self):
            self.tx_active = False
            self.swr_target, self.pwr_target = 1.0, 0.0

        def physics_tick(self):
            self.s_curr += ((max(0.0, min(15.0, self.s_target + random.uniform(-1.2, 1.2)))) - self.s
            self.swr_curr += (((max(1.0, min(5.0, self.swr_target + random.uniform(-0.15, 0.15))) if
            self.pwr_curr += (((max(0.0, min(100.0, self.pwr_target + random.uniform(-2.5, 2.5))) if
```

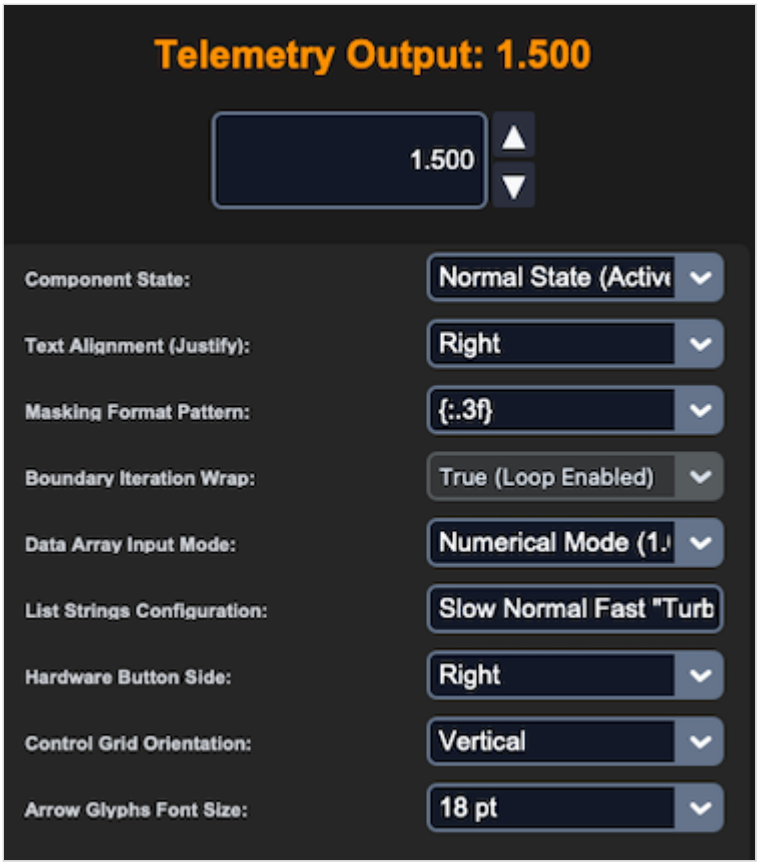
```
            self.bar.set(s_value=self.s_curr, swr_value=self.swr_curr, pwr_value=self.pwr_curr)
            self.root.after(20, self.physics_tick)

    sim = HarnessSimulator(app, led_bar_gauge)
    sim.physics_tick()
    sim.tuning_cycle()

    def toggle_theme():
        ctk.set_appearance_mode("Light" if ctk.get_appearance_mode() == "Dark" else "Dark")

    sCTkButtonPrimary(app, text="Toggle Theme", command=toggle_theme).pack(pady=5)
    app.mainloop()
```

sCTkSpinbox

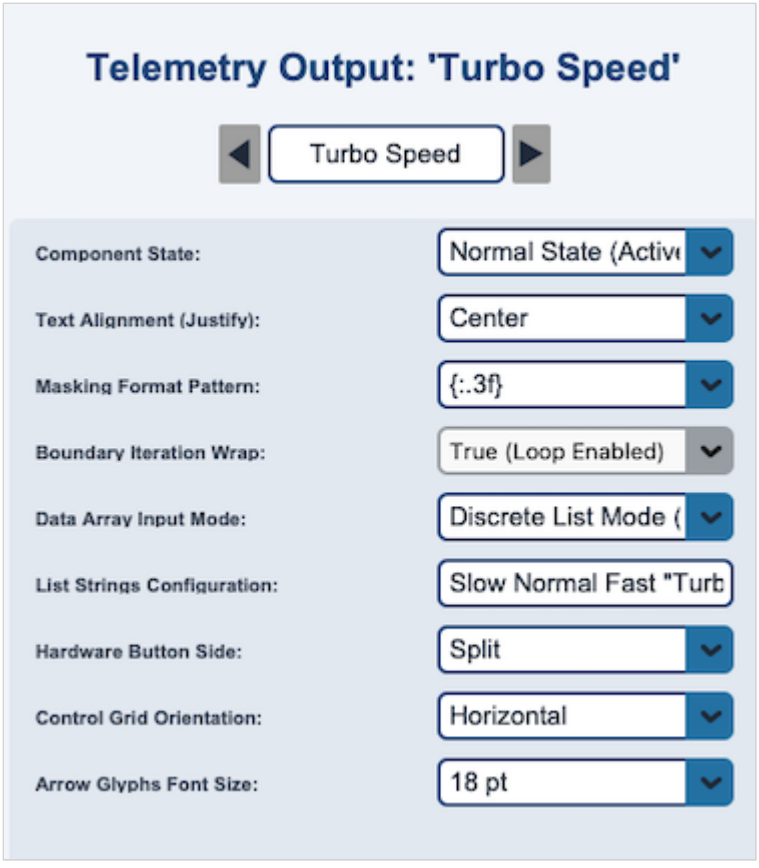
Table of Contents

Overview

`sCTkSpinbox` is a theme-compliant subclass of `customtkinter.CTkFrame` that composes an internal `sCTkEntryPrimary` alongside two increment/decrement buttons. It supports two operating modes: stepping a numeric value between `from_` and `to` in increments of `step_size`, or cycling through a fixed list of string values.

Dark Mode: 

Light Mode:



Like `sCTkEntryPrimary`, this widget has a genuine three-state model — normal, readonly, and disabled — matching real `ttk.Spinbox` semantics. The key distinction: in readonly mode, the entry can't be typed into directly, but the increment/decrement buttons stay fully clickable, since they're the intended alternative way to change the value. The buttons themselves only ever report `"normal"` or `"disabled"` — `"readonly"` isn't a real native `CTkButton`

concept, and there's no reason for the buttons to look any different in readonly mode, since nothing about their own behavior changes.

Every property this widget forwards to its internal entry (`width` , `height` , `justify` , `show` , `exportselection` , `placeholder_text` , `fg_color` , `border_color` , `text_color` , `state`) has been confirmed valid against CustomTkinter's own real `CTkEntry` source — both its explicit named constructor/configure parameters and its `_valid_tk_entry_attributes` whitelist. There's no risk of this widget sending an unrecognized property to its own entry.

Constructor

```
sCTkSpinbox(master=None, from_=0.0, to=100.0, step_size=1.0, command=None,
            state="normal", wrap=False, justify="left", show=None,
            placeholder_text=None, exportselection=True, width=140, height=32, **kw)
```

Parameter	Type	Default	Description
`master`	widget	`None`	Parent container.
`from_`	`float`	`0.0`	Lower bound of the numeric range. Ignored in discrete-value mode (see `values` below).
`to`	`float`	`100.0`	Upper bound of the numeric range. Ignored in discrete-value mode.
`step_size`	`float`	`1.0`	Amount added or subtracted per click of the up/down buttons.
`command`	`callable`	`None`	Called with the new value whenever it changes, via either button click or `set()` .
`state`	`str`	`"normal"`	Initial state: `"normal"` , `"readonly"` , or `"disabled"` . Any other value is treated as `"normal"` .
`wrap`	`bool`	`False`	If `True` , stepping past either bound (or past either end of a discrete value list) wraps around instead of stopping.
`justify`	`str`	`"left"`	Text alignment inside the entry: `"left"` , `"center"` , or `"right"` .
`show`	`str`	`None`	Character-masking string, e.g. `"*"` for password-style display.
`placeholder_text`	`str`	`None`	Placeholder shown when the entry is empty.
`exportselection`	`bool`	`True`	Standard Tkinter selection-to-clipboard behavior.

Parameter	Type	Default	Description
`width` / `height`	`int`	`140` / `32`	Overall widget dimensions in pixels.
`**kw`	—	—	`button_width`, `button_height`, `button_side` (`"left"` / `"right"` / `"split"`), `orientation` (`"vertical"` / `"horizontal"`), `arrow_font_size`, `format`, `values` — see Theming for where these come from — plus any theme-key override.

```
freq_spinbox = sCTkSpinbox(control_panel, from_=88.0, to=108.0, step_size=0.1, placeholder_text="MHz"
freq_spinbox.pack(pady=10)
```

Methods

Method	Returns	Description
`get()`	`str`	Current entry text.
`set(value)`	`None`	Sets the displayed value (a number, or a string matching one of `values` in discrete mode), and calls `command` if one was given. Temporarily re-enables the entry to update its text if it isn't currently `"normal"`, then restores whatever state it was actually in — including `"readonly"`, not just `"normal"` / `"disabled"`.
`set_values(list_of_strings)`	`None`	Switches to discrete-value mode with the given list, or back to numeric mode if given an empty list.
`state(mode=None)`	`str`	Gets or sets the widget's normal/readonly/disabled state. The entry receives the full three-way state (routed through its own `state()`); the up/down buttons only ever receive `"normal"` or `"disabled"`.
`get_state()`	`str`	Equivalent to calling `state()` with no argument.
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard configuration, plus all the constructor's custom keywords (`button_width`, `orientation`, `format`, `values`, etc.) can be changed at runtime the same way.
`cget(attribute_name)`	varies	Supports querying `state`, `from_`, `to`, `step_size`, `button_width`, `button_height`, `button_side`, `orientation`, `arrow_font_size`, `format`, `wrap`,

Method	Returns	Description
		and `values` directly, in addition to standard native properties.

Theming (`sCTkThemes.json`)

```
{
    "sCTkSpinbox": {
        "font": ["Arial", 15, "normal"],
        "arrow_font": ["Arial", 8, "normal"],
        "arrow_up_char": "▲",
        "arrow_down_char": "▼",
        "arrow_right_char": "►",
        "arrow_left_char": "◄",
        "border_width": 1.5,
        "corner_radius": 6,
        "entry_color": ["#FFFFFF", "#111827"],
        "border_color": ["#1A4375", "#64748B"],
        "text_color": ["#1F2937", "#F9FAFB"],
        "placeholder_text_color": ["#5A6E7F", "#526071"],
        "button_color": ["#9E9E9E", "#2A2F3D"],
        "button_hover_color": ["#7D7D7D", "#374151"],
        "disabled_map": {
            "entry_color": ["#F3F4F6", "#1F2937"],
            "border_color": ["#CBD5E1", "#475569"],
            "text_color": ["#94A3B8", "#64748B"],
            "button_color": ["#CBD5E1", "#334155"]
        },
        "readonly_map": {
            "entry_color": ["#F8FAFC", "#1F2937"],
            "border_color": ["#64748B", "#94A3B8"],
            "text_color": ["#1F2937", "#F9FAFB"]
        }
    }
}
```

`entry_color` / `border_color` / `text_color` override the internal entry's own colors for all three states, a deliberate design choice: this widget controls its entry's look via its own theme keys rather than the entry's independent defaults. `readonly_map` requires `entry_color` , `border_color` , and `text_color` whenever readonly is actually requested — missing any raises immediately. No readonly-specific `button_color` exists or is needed, since buttons always use normal `button_color` / `button_hover_color` whenever they aren't disabled — they're meant to look completely ordinary in readonly mode.

`arrow_font` is read in full — family, size, and weight — and applied to both increment/decrement buttons. It can also be overridden at runtime via `configure(arrow_font=(...))` , or `configure(arrow_font_size=...)` to change just the size without respecifying the full tuple.

Example

```python
from scustomtkinter import sCTk, sCTkFrame, sCTkSpinbox, sCTkButtonPrimary, sCTkLabelPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x250")
    root.title("Spinbox Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    freq_spinbox = sCTkSpinbox(base, from_=88.0, to=108.0, step_size=0.1, placeholder_text="MHz")
    freq_spinbox.pack(pady=10)

    status = sCTkLabelPrimary(base, text=f"state: {freq_spinbox.get_state()}")
    status.pack(pady=5)

    def cycle_state():
        order = ["normal", "readonly", "disabled"]
        current = freq_spinbox.get_state()
        next_state = order[(order.index(current) + 1) % len(order)]
        freq_spinbox.state(next_state)
        status.configure(text=f"state: {freq_spinbox.get_state()}")

    cycle_btn = sCTkButtonPrimary(base, text="Cycle State", command=cycle_state)
    cycle_btn.pack(pady=10)

    root.mainloop()
```

Known Limitations

- **The disable/enable-cycle cursor-position fix is not independently confirmed for readonly transitions** — the underlying entry inherits this caveat from `sCTkEntryPrimary`; see that widget's docs for the full explanation.
- `readonly` **mode's placeholder behavior follows** `sCTkEntryPrimary` **'s** — a readonly field showing placeholder text never clears it on focus, since native CustomTkinter deliberately never deactivates a placeholder while `state` is `"readonly"`.
- Calling `configure("propname")` for most single-argument property queries returns a Tkinter-style tuple whose `current` value may be `str()` of a `(light, dark)` color tuple rather than a single resolved color — the same known gap as elsewhere in this project's Pygubu-query investigation.

Return to Table of Contents

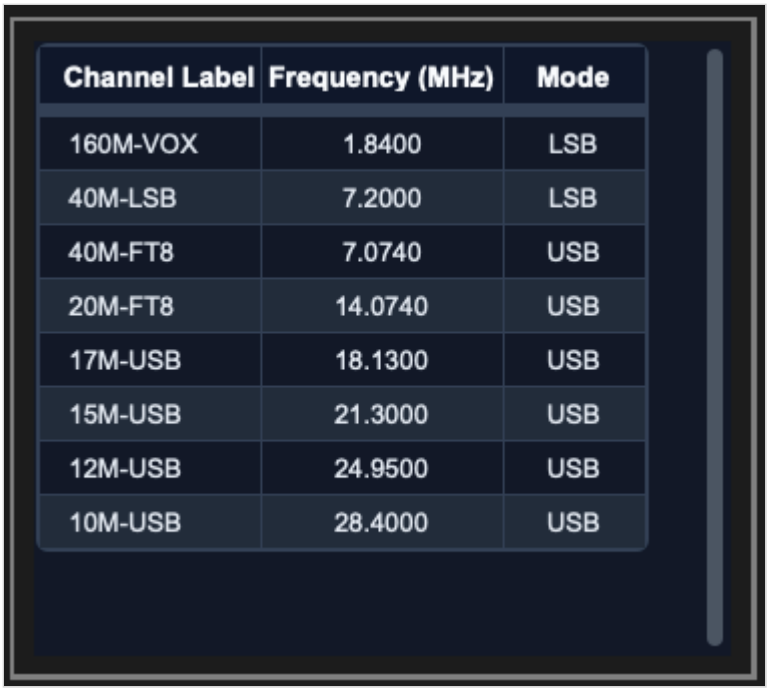
sCTkTableview

Table of Contents

- Known Limitations

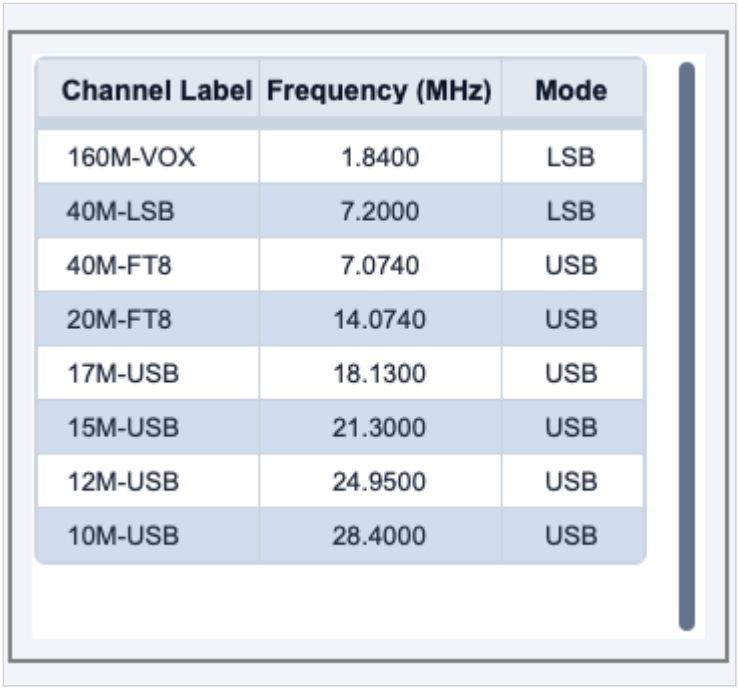
Overview

`sCTkTableview` is a theme-compliant, scrollable grid of labeled cells — a simple spreadsheet-like table, with optional zebra-striped rows, click and edit callbacks, and in-place cell editing. It's built by inheriting `sCTkScrollableFrame` directly, using its scrolling and label feature, then laying out its own header row and cell grid on top.

Dark Mode: 

Channel Label	Frequency (MHz)	Mode
160M-VOX	1.8400	LSB
40M-LSB	7.2000	LSB
40M-FT8	7.0740	USB
20M-FT8	14.0740	USB
17M-USB	18.1300	USB
15M-USB	21.3000	USB
12M-USB	24.9500	USB
10M-USB	28.4000	USB

Light Mode:



Channel Label	Frequency (MHz)	Mode
160M-VOX	1.8400	LSB
40M-LSB	7.2000	LSB
40M-FT8	7.0740	USB
20M-FT8	14.0740	USB
17M-USB	18.1300	USB
15M-USB	21.3000	USB
12M-USB	24.9500	USB
10M-USB	28.4000	USB

This widget inherits `sCTkScrollableFrame` directly — the same composition pattern used by `sCTkSelector` — and previously needed a fragile workaround for it: temporarily overwriting its own `self.__class__.__name__` during construction, to trick `sCTkScrollableFrame` 's internal `ThemeableWidget.__init__` call into reading a harmless theme block instead of corrupting this widget's own. That workaround has been removed entirely. `ThemeableWidget` 's

run-once guard now prevents the double-init outright, and `sCTkScrollableFrame` itself filters its inbound kwargs down to only what native `CTkScrollableFrame` actually accepts — confirmed directly against CustomTkinter's source to have no `**kwargs` catch-all at all, so this filtering matters more here than for almost any other widget in this project.

Constructor

```
sCTkTableview(master, columns=None, width=500, height=300, grid_mode="zebra",
              header_line_width=2, outline_width=1.0, outline_radius=4,
              state="normal", num_columns=3, num_rows=1, show_headers=True,
              cell_bg_color=None, cell_alt_bg_color=None, *args, **kwargs)
```

Parameter	Type	Default	Description
`master`	widget	—	Parent container.
`columns`	`list[str]` or comma-separated `str`	`None`	Column header labels.
`width` / `height`	`int`	`500` / `300`	Overall widget dimensions in pixels.
`grid_mode`	`"zebra"` / `"grid"` / `"none"`	`"zebra"`	Row background styling.
`header_line_width`	`int`	`2`	Header row's bottom border thickness.
`outline_width` / `outline_radius`	`float` / `int`	`1.0` / `4`	Outer table border thickness and corner rounding.
`state`	`"normal"` / `"disabled"`	`"normal"`	Initial state.
`num_columns` / `num_rows`	`int`	`3` / `1`	Initial grid size when `columns` isn't given.
`show_headers`	`bool`	`True`	Whether the header row is shown.
`cell_bg_color` / `cell_alt_bg_color`	color	`None`	Overrides the theme's cell background colors for this instance specifically — see Theming for how this interacts with the theme file.
`**kwargs`	—	—	Any native `CTkScrollableFrame` argument, or an override for one of the other theme keys listed under Theming.

```
readings_table = sCTkTableview(control_panel, columns=["Time", "Frequency", "Signal"], num_rows=8)
readings_table.pack(expand=True, fill="both", padx=20, pady=20)
```

Methods

Method	Returns	Description
`state(mode=None)` / `get_state()`	`str`	Gets or sets `"normal"` / `"disabled"` .
`configure(**kwargs)` / `config(**kwargs)`	varies	Standard configuration, plus `state=...` triggers a full color/font re-application across every header and cell.

Theming (`sCTkThemes.json`)

- **Applied once, at construction** — every key below, plus `cell_bg_color` / `cell_alt_bg_color` (which can also come from the constructor, see below).
- **Re-applied on every** `state()` **change.**

```
{
    "sCTkTableview": {
        "header_bg_color": ["#E2E8F0", "#0F172A"],
        "header_text_color": ["#0F172A", "#F8FAFC"],
        "header_font": ["Arial", 14, "bold"],
        "cell_bg_color": ["#FFFFFF", "#111827"],
        "cell_alt_bg_color": ["#D1DCEE", "#222C3A"],
        "cell_text_color": ["#1E293B", "#E2E8F0"],
        "cell_font": ["Arial", 13, "normal"],
        "grid_line_color": ["#CBD5E1", "#334155"],
        "disabled_map": {
            "header_bg_color": ["#CBD5E1", "#1E293B"],
            "header_text_color": ["#94A3B8", "#64748B"],
            "cell_bg_color": ["#F1F5F9", "#1F2937"],
            "cell_alt_bg_color": ["#E2E8F0", "#263241"],
            "cell_text_color": ["#94A3B8", "#64748B"],
            "grid_line_color": ["#E2E8F0", "#293548"]
        }
    }
}
```

All six colors are required both at the top level and in `disabled_map` — missing any raises immediately at construction, naming the exact key. `header_font` / `cell_font` are required only at the top level; no widget in this project uses a disabled-state font variant.

`cell_bg_color` / `cell_alt_bg_color` **are the two exceptions** — they can come from either the theme block *or* the constructor kwarg of the same name, so it's only a hard failure if *neither* provides a value. Whichever one this instance resolves to at construction is remembered and correctly restored on every return to `"normal"` — an earlier version

always reverted to the theme's value on re-enable, silently discarding a constructor override after a disable/enable cycle.

Colors are passed through as raw `(light, dark)` tuples, letting CustomTkinter's native appearance-mode tracking handle repaints — an earlier version resolved disabled-state colors to a single fixed string while leaving enabled-state colors as tuples, meaning a disabled table would stop following light/dark mode changes while an enabled one kept working correctly. Both branches are now consistent.

Example

```
from scustomtkinter import sCTk, sCTkFrame, sCTkTableview, sCTkButtonPrimary

if __name__ == "__main__":
    root = sCTk()
    root.geometry("400x300")
    root.title("Tableview Example")

    base = sCTkFrame(root)
    base.pack(expand=True, fill="both", padx=20, pady=20)

    table = sCTkTableview(base, columns=["Time", "Frequency", "Signal"], num_rows=6)
    table.pack(expand=True, fill="both", pady=10)

    def toggle_disabled():
        target = "disabled" if table.get_state() == "normal" else "normal"
        table.configure(state=target)
        toggle_btn.configure(text="Enable Table" if target == "disabled" else "Disable Table")

    toggle_btn = sCTkButtonPrimary(base, text="Disable Table", command=toggle_disabled)
    toggle_btn.pack(pady=10)

    root.mainloop()
```

Known Limitations

- Missing a required theme key raises `KeyError` at construction, naming exactly which key and whether it's needed at the top level or in `disabled_map` — check the exact message if construction fails after a theme file change.
- Calling `configure("propname")` for most single-argument property queries falls through to the native widget's `configure()`, which doesn't support arbitrary single-argument queries — the same known gap as elsewhere in this project's Pygubu-query investigation.

Return to Table of Contents